# Daily Notes - 2026 January

Jan 30., Fri
Agents

Jan 29., Thu
Agents

Jan 28., Wed
Agents Team

Jan 27., Tue
Agents

Jan 26., Mon
Agents

Jan 23., Fri
Agents Team

Jan 22., Thu
Agents Team

Jan 21., Wed
Agents Team

Jan 20., Tue
Agents Team

Jan 19., Mon
Agents

Jan 16., Fri
Agents

Jan 15., Thu
Agents

Jan 14., Wed
Agents

Jan 13., Tue
Agents

Jan 12., Mon
Agents

Jan 9., Fri
Agents

Jan 8., Thu
Agents

Jan 7., Wed
Agents

Jan 6., Tue
Agents

Jan 5., Mon
API/MCP · Agents Team



↑ Daily Notes - 2026 January



## Jan 30., Fri

↑ Jan 30., Fri

# Agents

## Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Gyula's release plan** - Planning to release and create a showcase video for Craft Agents functionalities. Also working on Google OAuth improvements and robustness for refresh handling (addressing daily re-login issues).
- **Levy's MCP v2 constraints discussion** - Before publishing with Craft KB data, wants to add constraints: "No more than 1 deletion per tool call, No more than 2 documents edited per tool call." Team should align on these limits.
- **Balint's CDC/analytics pipeline** - Major infrastructure work on sampling, time-travel queries, and Athena integration. May need async query interface for agents. Worth discussing architecture implications.
- **Christoph improving agent UX** - Working on making prompts "more flexible and forgiving" rather than requiring precise terms. Good to share learnings on what works.
- **Cross-team**: Gyula had first AI knowledge transfer with Benedek (App Team) - gaining insights on using AI for UI challenges.

## Christoph Locher

- Implemented repeating tasks configs in the HTML parser and validation, so we now have a natural language of setting repeating tasks
- Implemented all page-level styling properties (Text Color, Document Color, all Backdrop types and settings, Separators with all washi settings, Cover image, wide page, default font)
  - Through the agent users can now set custom light and dark mode text and document colors and even backdrops (including different gradients for light and dark mode). It even works for the pattern style washi's colors.
  - I also checked page styling works with subpages, so the agent can also override subpage styling and go back to inheriting from parent

### Jan 30., Fri

- Start tweaking the system prompts and HTML documentation to help the agent better understand user requests – right now it can do everything, but for some actions it needs precise prompts using the right terms. I want to make it more flexible and forgiving.
- If there is time in the afternoon, I might look into tables or collections editing

## Balint Bartfai

Since yesterday's rollups had to be ditched from RisingWave, I looked at other solutions for longer-term storage and how w (agent) can query them.

Initially I was planning to store all CDC changes to cold storage, then build query capabilities over that dataset. This proved to be way too much data.

So I approached this from 2 angles

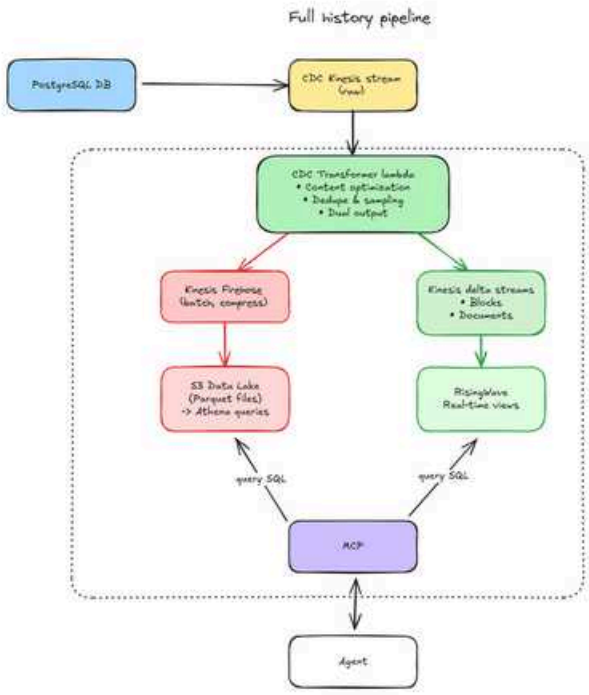
- how can we reduce the CDC stream to distill the deltas to high-fidelity updates only
- how can we ensure the capability for an agent to be able to write queries and "rewind time", being able to get a full history of document edits

First, our current setup blindly takes every update (batch) from CDC, deflates, transforms into plain JSON, and sends it to a Kinesis queue consumed by RW. To reduce volume, I implemented a content transformer on the CDC consumer that

- dedupes CDC records. There were noops coming - same content hash
- implements sampling for hi-fi events only, time and significance based. For events on docs
    - keep first occurrence (this is our baseline)
    - keep deletes (for reconstruction of a doc based on deltas)
    - keep significant changes (phrases, sentences, etc)
    - keep periodic snapshots (30s intervals during active editing)
    - ...skip everything else

Once this was done and working, I looked at how we could bring back the rollups and have agents querying longer-term information, as this can not come from RisingWave OR our DB (we simply don't have this information).

I built a pipeline like this:




The debezium CDC stream gets consumed by the consumer which samples, enriches, etc. (above), and sends the events to 2 Kinesis streams: one ingested by RisingWave, and a firehose that is piped to a long-term storage data lake: parquet on S3. Parquet format is a compressed format designed for 'classic' big-data analysis, and can be queried by AWS Athena (which if I understand correctly is Presto/Trino under the hood). I built this with a goal in mind: given updates that the agent can query, can it time-travel? And if it can, can it recreate a document from deltas? Currently it can, but **only for block/doc contents**, not styling. If needed we can expand on this later.

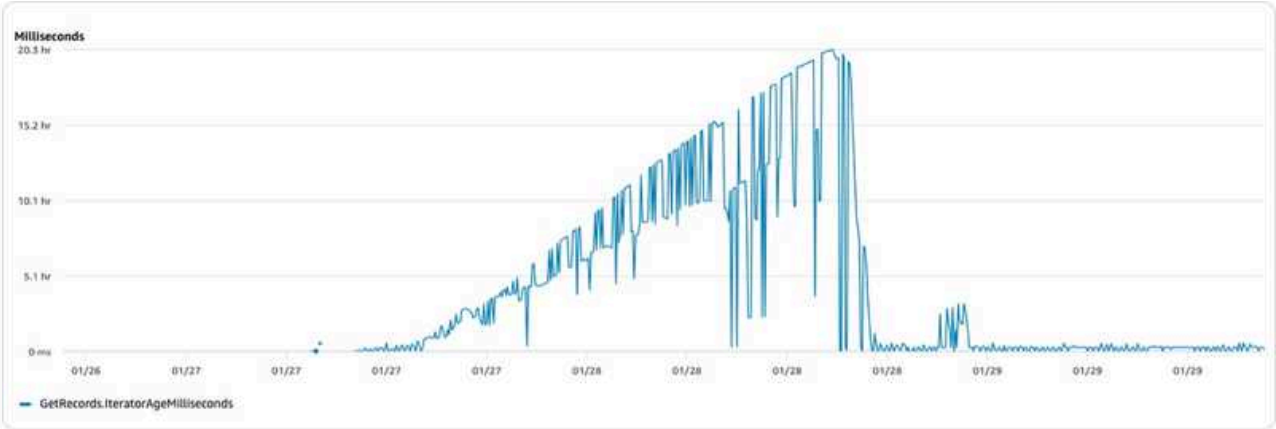
Rollups happen every day, every month, and older data (30d+) is piped to Glacier for even cheaper storage. Queries will take time for this, so we will have to build an async query interface - where the agent receives a query ID, and can poll an endpoint for results of that query. Athena seems to perform well, so for now I am keeping online queries for it.

> Currently I am running 2 separate content enrichers - working on a single one which outputs to 2 streams. This should be ready early tomorrow.

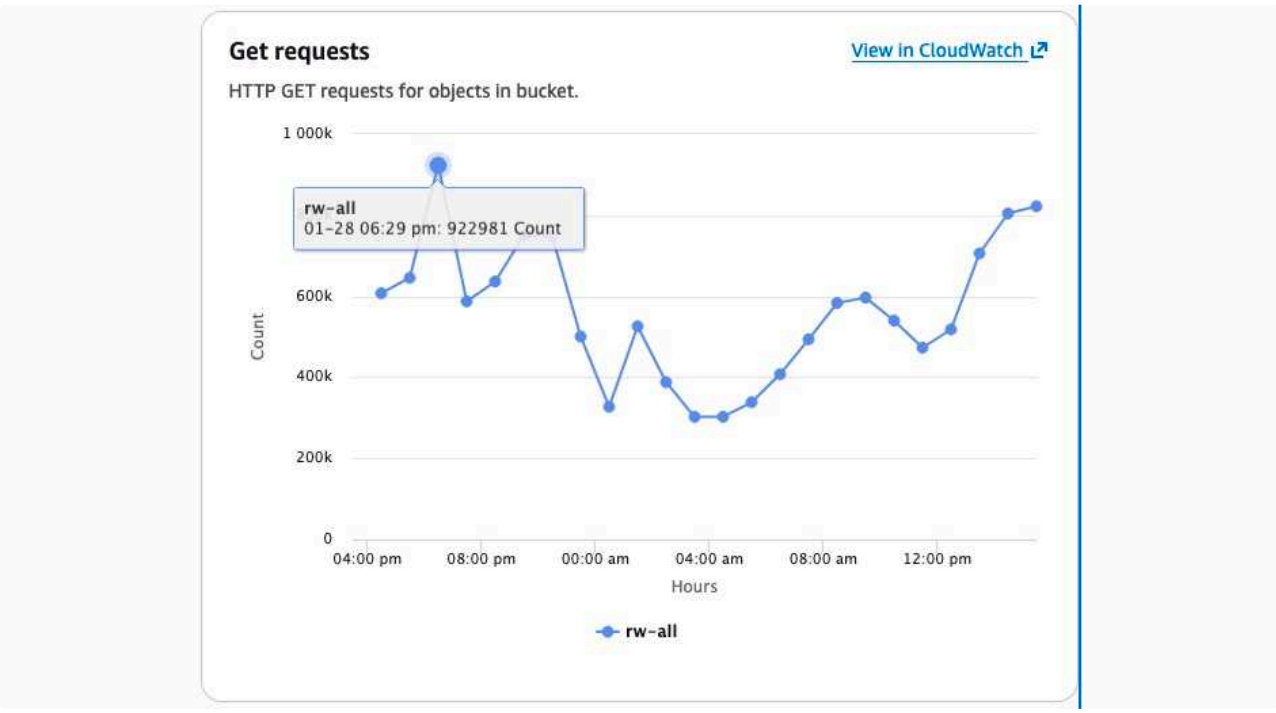
I built out tools for agents to be able to query this information. Both 'specialized' ones (doc history) and a more generic SQL interface like with presence information. Testing is tricky as both Marci and myself are deploying to `luki-workflow-links` sandbox, so I focused more on the infra side.

### More data from RisingWave in prod

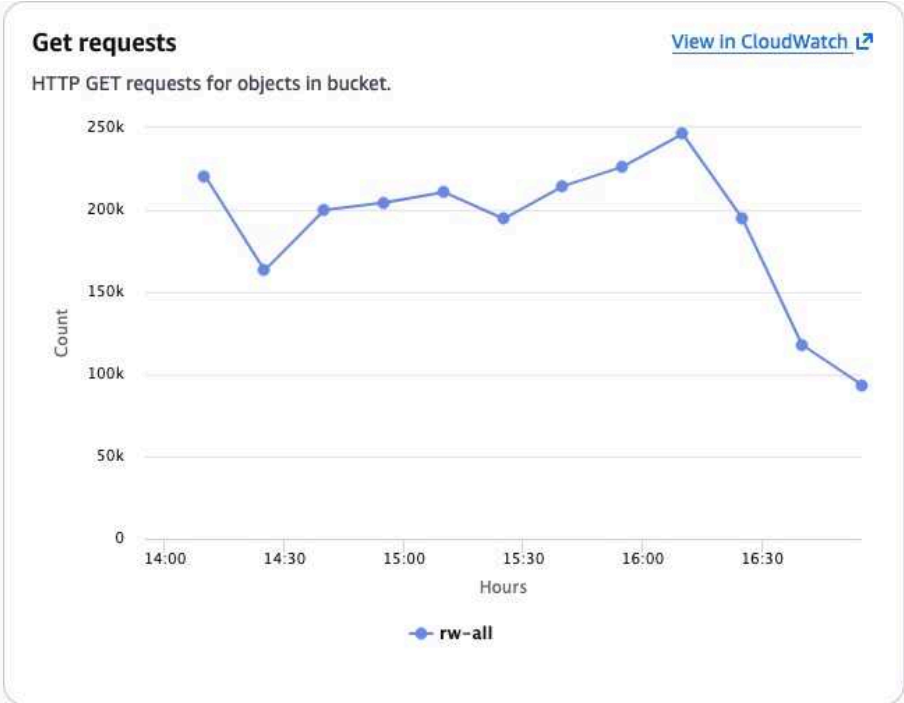
After solving yesterdays Kinesis backlog issue:




I took a closer look at how many S3 ops are taking place. This is the last 24h




Way too much. Reducing the volume further would mean losing important updates, so I looked at implementing the ECS disk cache to cut S3 R/W - RisingWave recommends this if the disk IO can handle it. Fargate ephemeral storage perf is similar to gp2 - ~125MB/s baseline throughput. I configured RW to go with 200 MB/s max, if this is overwhelming (IOPS queueing up) I will reduce this.




> Unfortunately this turns out to be a premium only feature in RisingWave. We can run on 4 vCPUs as a "trial" - currently we're running 4. So this stays, but any further scaling of CPU will stop this feature. Testing.

## Next up

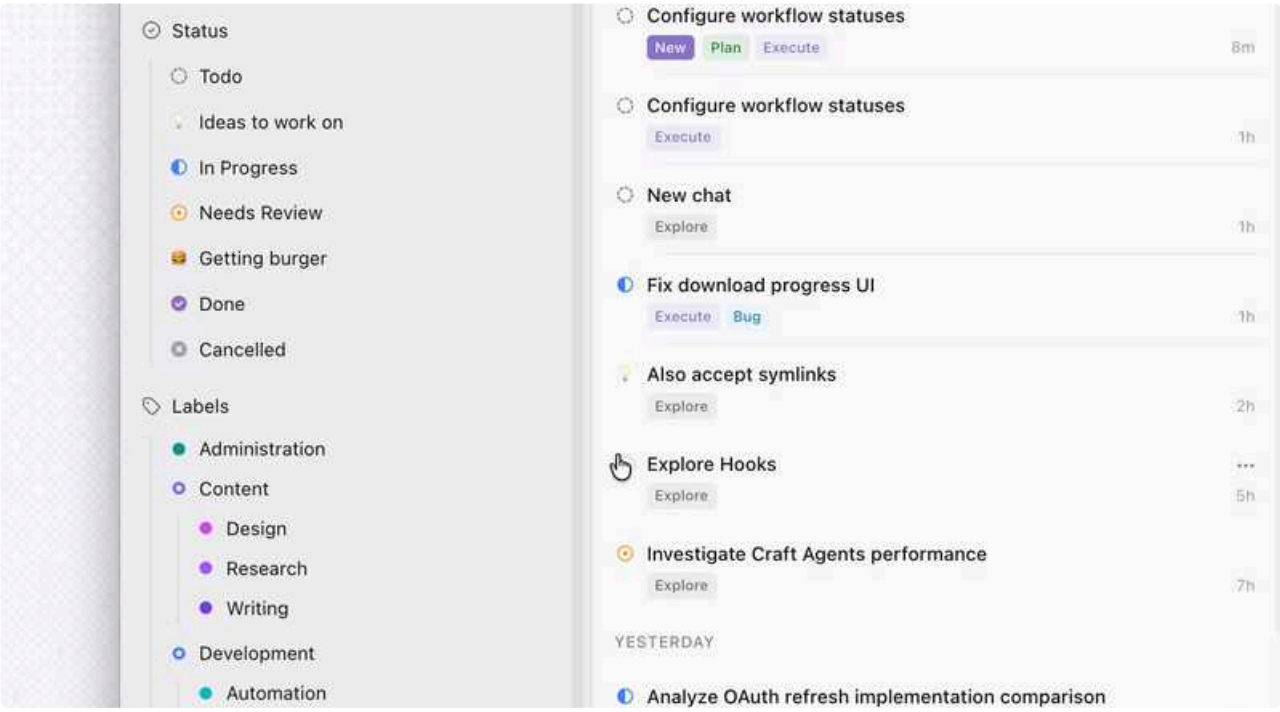
Based on todays changes to RW and stream events, I want to finally push this to 100% samples.

Merging agent tools to query this information should come tomorrow as well.

Gyula Halmos

## Craft Agents

- Finalized the search function spent quite some time on refactoring and polishing it
- Had our first AI knowledge transfer with Benedek Gobolos, gained some insights how is he using the AI to tackle UI challenges
  - Also told Bene to dog food Craft Agents more haha
- Also wrapped up the “Minis” - for all popovers where the users can change the UI by invoking agentic sessions now we spawn a lightweight Haiku session with limited tool access and also trimmed down sys prompt (no Claude preset, no Craft preset, just a basic sys prompt)
- I will assist the Maestro in making my UI components better looking by adding his magic
- Lesson is learned that the SDK still provides all tool descriptions though so i couldnt go below 14k tokens but it is already better than the 50k with presets
- After the functionality got in place i also started working on the UI to have an inline execution instead of a separate chat window




- Can be polished, but not bad
- Also fixed some smaller bugs (like code blocks did not render in turn card’s output)

## Next steps

- Plan is to release
- I put together a script and show case the Craft Agents functionalities in a video
- I have a WIP solution to impro im ve Google OAuth and also want to focuse on the OAuth related robustness - proper refresh to handle these cases where people have to login to everything every morning

- Some cases may be on our side some maybe on the provider side
- Also there were some suggestions on how to handle Sources wanna spend a bit on that and Linux ARM64 build if i get there

Jan 31., Sat morning up to 11 will be hectic i have to run errands, bank, barber... will be on and off and after 11 going to the office.

Zsolt Varnai

Jan 29., Thu

**Assistant Editing**

- **Infra Update**
  - **Environment**
    - Recreated the services from the latest service template
    - Using commonJS modules and node runtime
    - It turned out, that the **Claude Agent SDK also requires ESM module resolution**, so the compilation failed
    - Discussed the options with Tamas and went through a bunch of trial and error like last time with different module configurations and got to the same conclusions
    - Bun was still a viable option, but we tried to avoid using bun in AWS
    - At the end we got a solution, thats good enough:
      - **Upgraded Node from v22 to v24**, which solved the ESM resolution errors in AWS and local debugging without using bun
      - For the unit test execution ts-node/mocha does not support ESM resolution, so **we use bun only for test execution**
  - **Common library**
    - Reverted back the common library changes and deployment scripts to standard
    - Created service specific utils where needed
- **Existing Tools**
  - Updated existing tools to return HTML instead of markdown content
  - As the HTML conversion is the on the server side the client side tools just return the block json in the tool result and the server executes the conversion and replacement of the content
- **Improvements**
  - Improved validation for card image properties

Jan 30., Fri

- Continue with editing support

↑ Daily Notes - 2026 January

## Jan 29., Thu

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · Block editing ahead of schedule...



↑ Jan 29., Thu



## Agents

## Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Block editing ahead of schedule** — Christoph completed almost all block types (images, separators, code, whiteboard, sketch, files, smart links, cards, tasks) — originally planned for Friday. Moving to page-level properties today. Great progress to celebrate!
- **Image and file URL issues** — Christoph mentioned "remaining issues with image and file urls" that Zsolt is looking at. Status check needed.
- **RisingWave stream consumer struggles** — Balint had to drop 5 expensive materialized views with 3-way JOINs due to stream consumer not keeping up. Learning: only keep MVs that are actively queried; others should be JOINed on-demand by agents.
- **Presence debouncing tradeoffs** — Balint implemented heartbeats every 60sec, but initially disabled no-update events (suboptimal for presence accuracy). Discuss if current approach is acceptable long-term.
- **New agent tooling for presence/activity** — Balint created tools (`presence_active`, `presence_editors`, `activity_query`, etc.) for agents to query real-time analytics. Cross-team awareness — these may be useful for other teams' agents.

## Christoph Locher

Jan 28., Wed

I continued to work on getting the agent to edit the blog types, and we are now at a point where not just text blocks, but almost all block types are supported:

- images
- separators (including all washi settings)
- code blocks (including setting language and theme) & formulas
- whiteboard & sketch
- file blocks
- smart links (changing layout, title, description)
- cards (including all background configurations and sizes)
- tasks can now be rescheduled by the agent

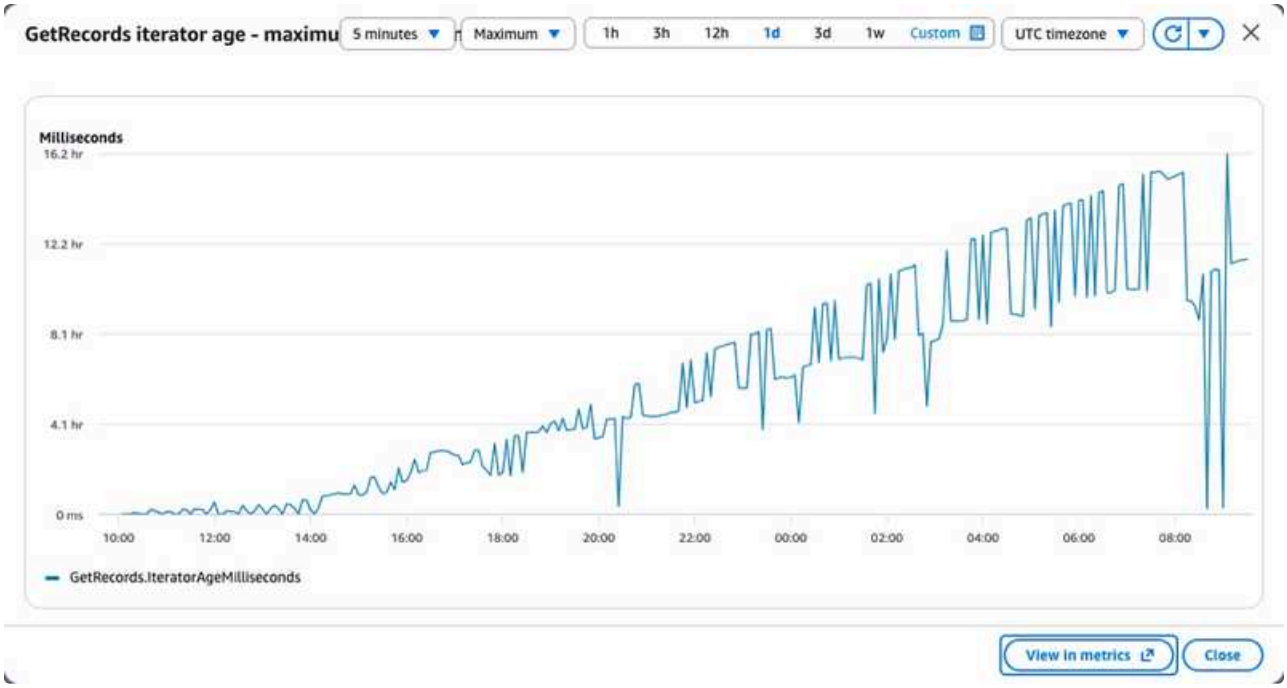
There are some remaining issues with image and file urls, that Zsolt is looking at.

Jan 29., Thu

Because on the block basis we are now already much further along than planned – all block types were planned for Friday and cards not at all – tomorrow, I plan to look into page level properties, which will also enable styling and theming.

## Balint Bartfai

### Stream consumer




The stream consumer for presence events was not keeping up, as can be seen above.

→ Further debounce needed, scaling RW to 4 vCPU as it is clearly CPU-bound at this stage.

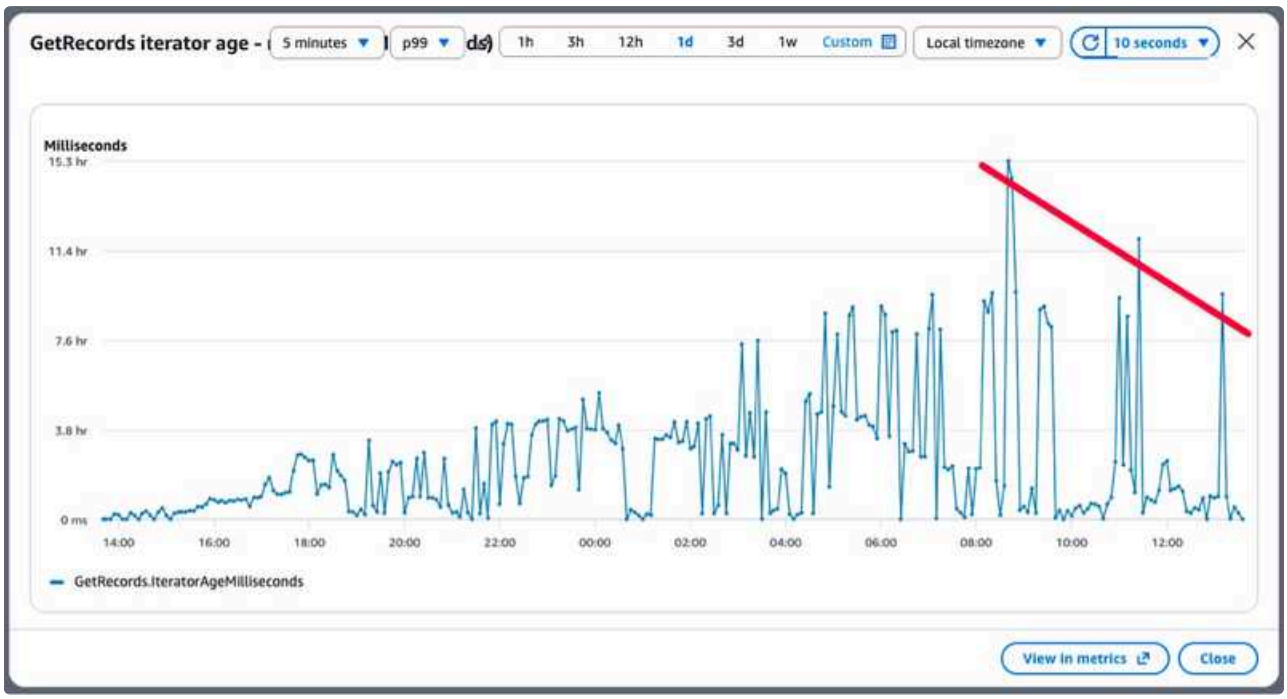
First, disregard no-update events. This is suboptimal: it doesn't provide a heartbeat, so if a user has a doc open but doesnt move the cursor, it would eventually not send a presence event. I will keep this until the stream is in a better shape.

Next, I implemented heartbeats every 60sec if theres no change. This provides better realtime presence information.

Further debounce for presence
#4226 · Opened by balintb

Open ready-for-ci ready-for-review

RW lag spikes are reducing (p99), meaning it is now catching up:




I had to drop expensive MVs with 3-way JOINs in them

- `user_edit_sessions`
- `unified_activity_stream`
- `unified_session_activity`
- `user_activity_hourly`
- `user_activity_daily`

These can be recreated **on-demand by the agent** when it queries - no need for materialized views.

The question then became why max time would be dropping and rising? This has to do with how RW reads into MVs. Max metric shows whichever is farthest behind.

So now, we have significant backpressure on the enriched edits table


So I dropped the `document_edits_enriched` MV as well. This would be trivial for an agent to JOIN on demand.

Going through these MVs, it's clear that we need to only keep those MVs which we will be atively querying against. All others, which can be queried on the fly, should not be built as the stream is ingested. Reading up on RisingWave internals, each source in a MV should be up-to-date with another - otherwise we have iterators that wait for hours on the stream, as seen with presence.

Learning is rollups have to be managed differently. I am looking at how we can do this more effectively.

## Tooling

I created tools for the agents to query data.

Agent tooling for presence
#4229 · Opened by balintb
Open ready-for-ci

Instead of informing the agent through tool descriptions about which tables are available, the agent learns about available tables by reading the `risingwave://schema` resource (we can rename this later!), then uses the `activity_query` tool to run SQL queries against those tables to get real time analytics about doc activity / user presence / collaboration. Filtered by `space_id`.

Besides `activity_query`, some other specialized tools:

- `presence_active`: who's currently viewing doc
- `presence_editors`: who's adctively editig doc
- `presence_collaboration` - full space only: docs with multiple active users
- `activity_timeline`: get activity timeline for space/document
- `activity_user`: get specific users activity

- `activity_document`: get document activity summary

The data we have now is great for realtime collab and recent activity. Long-term analysis will need to come from somewhere else, as maintining rollups on the fly is not something RW seems good at.

Adding a data lake as a sink to RW would solve longer term analysis. I will look at our options on tech. A solution I have in mind is

> Kinesis (as our firehose) → data lake (S3, in parquet) → Athena (or Presto, etc) tables defined over S3

An agent can easily construct a historical query (which might take minutes to run), and does not have to worry about indexes / query perf. If we store deltas, we'll have a full audit log of every change to every document, queryable on cheap storage.

### Next up

- Will be monitoring RW more, and if streams can be ingested on time, will be pushing to 100% sampling rate
- Agent tooling tests on sandbox
- Firehose → data lake ingest tests. Simple kinesis to S3, with Athena queries

Gyula Halmos

## Craft Agents

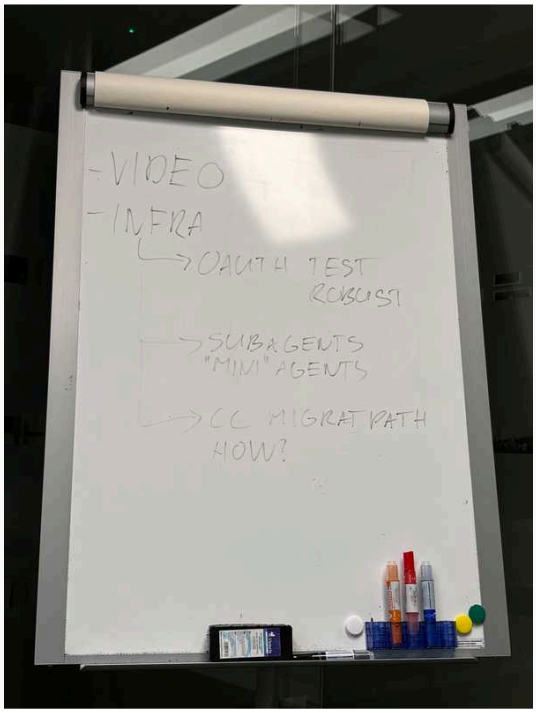
Improve search navigation and highlighting UX
#124 · Opened by rjulius23

Closed

- UI case study for me 🙂was polishing inteh morning still work out some glitches since pulling hte latest main some small changes in the CatDisplay need to be handled
- Spent sometimes on hooks as well

Craft Share Page
View and share Craft Agents session transcripts
https://agents.craft.do/s/XgC0EvJE8zNDeZzUSgHNI

- Defined the focus going forward:




- Also fixed and closed some Open Source issues - fixes are on main - low impact changes

## Next steps

- Shift focus to "Mini" Agent executions - already started the planning phase
- Finish the Search function and prepare for final review (due to being UI impact 🙂)
- Look at Craft OAuth flow (i still use workflow links, so should play around with actual OAuth for Craft)

## Zsolt Varnai

Jan 28., Wed

**Assistant Editing**

- **Block Styling**
  - Fixed some remaining styling update issues base on the end to end tests
- **Block Operations**
  - Added recursive delete block support for subpages
  - Added support for local block reordering based on HTML content
  - Added page styling logic for subpages, when the block becomes a subpage or is reverted back to non-page
  - Fixed new subpage creation flow, where the block has no HTML yet
  - Found and fixed issue with HTML sync flow, where stale HTMLs can break updates, when a page is alternating between having subpage content vs not
  - Cleaned up and fixed issues with new block creation flow
  - Added validation logic for unsupported block moves via HTML sync
- **Agent Infra**
  - Added proper support for parallel tool calls in the message handling loop
- **Undo / Redo**
  - Also added undo redo support into the client tool infra, so non-sync based edit operations can also be undone (eg delete and move blocks tool)
- **Image Support**
  - Added image creation support by triggering metadata updaters to create proper images
  - Investigated issue why Craft S3 asset URLs cannot be inserted like other URLs
    - The metadata updaters didn't inject auth headers for S3 urls, so added AAC handling to the Image and File metadata updaters

Jan 29., Thu

- Continue with editing support
  - Service infra updates

↑ Daily Notes - 2026 January

# Jan 28., Wed

Agents Team
Recommended for discussion · generated by Daily Team Discussion Points Agent · Presence services capacity test r...



↑ Jan 28., Wed



# Agents Team

## Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Presence services capacity test results** — Balint deployed to prod at 50% sampling after RTC optimization (~80-90% less events). S3 storage estimation: ~$700/month worst case, likely ~$350/month with optimizations. Running overnight for 24h+ data collection before moving to 100%.
- **RisingWave cost optimization options** — Balint identified 3 optimization levers: reduce checkpoint frequency (risk: data loss), add memory to 16GB (less compaction), enable elastic disk cache (90% S3 read reduction claimed). Worth discussing trade-offs.
- **Testing with actual agent tomorrow** — Balint wrote tools for agent testing, merging to sandbox tomorrow. Good checkpoint for team awareness.
- **Text block operations reliable** — Christoph reports all text block operations and settings including inline styles now work reliably with strict HTML validation. Lists still slightly flaky. Moving to other block types today (Code, Link, File, Drawing, Whiteboard).
- **Hooks/event system design in progress** — Gyula evaluating hooks architecture with event bus pattern. Looking at SQLite + nanoevents but wants to research how successful desktop apps handle this. Good for early input if anyone has experience.
- **Gyula half-day off** — Gyula will be off after noon today.
- **Cross-team: CX sequential execution question** — Tamas (CX) encountered issue where agent parallelized despite plan specifying sequential 2-second delays. Open question for Agents Team: how to enforce sequential execution when tools want to parallelize?

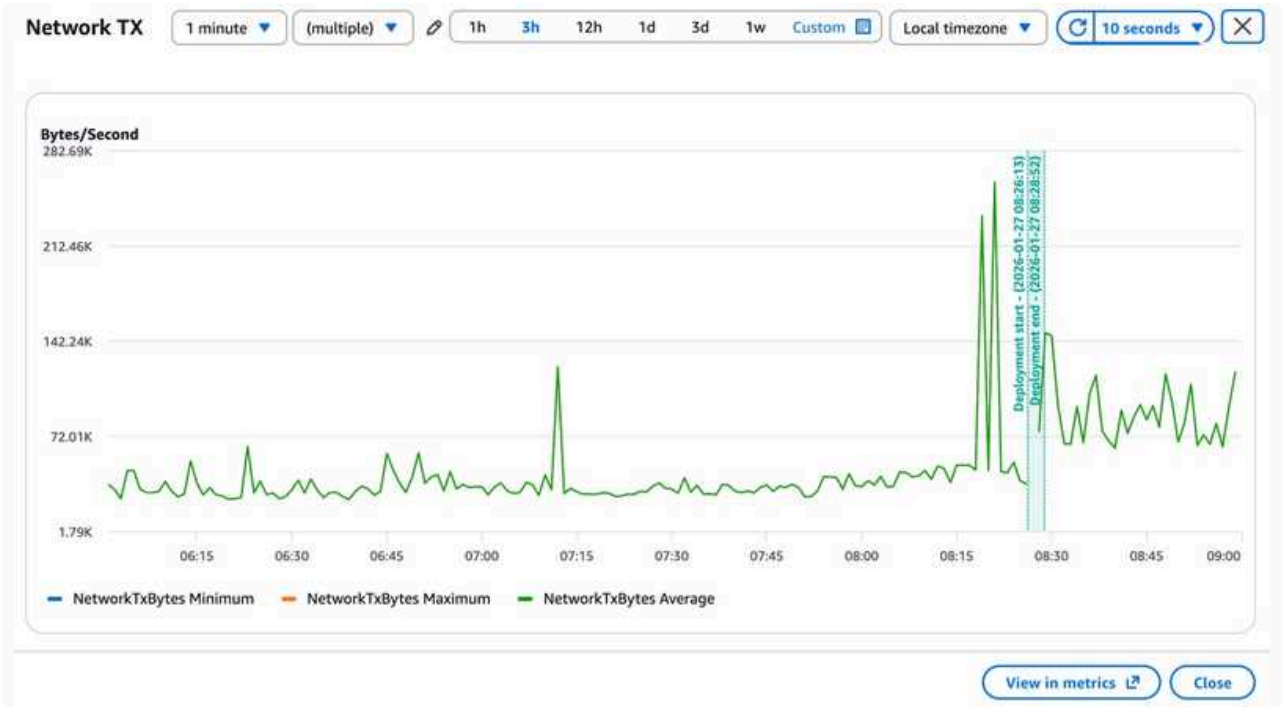
## Balint Bartfai

Summary:

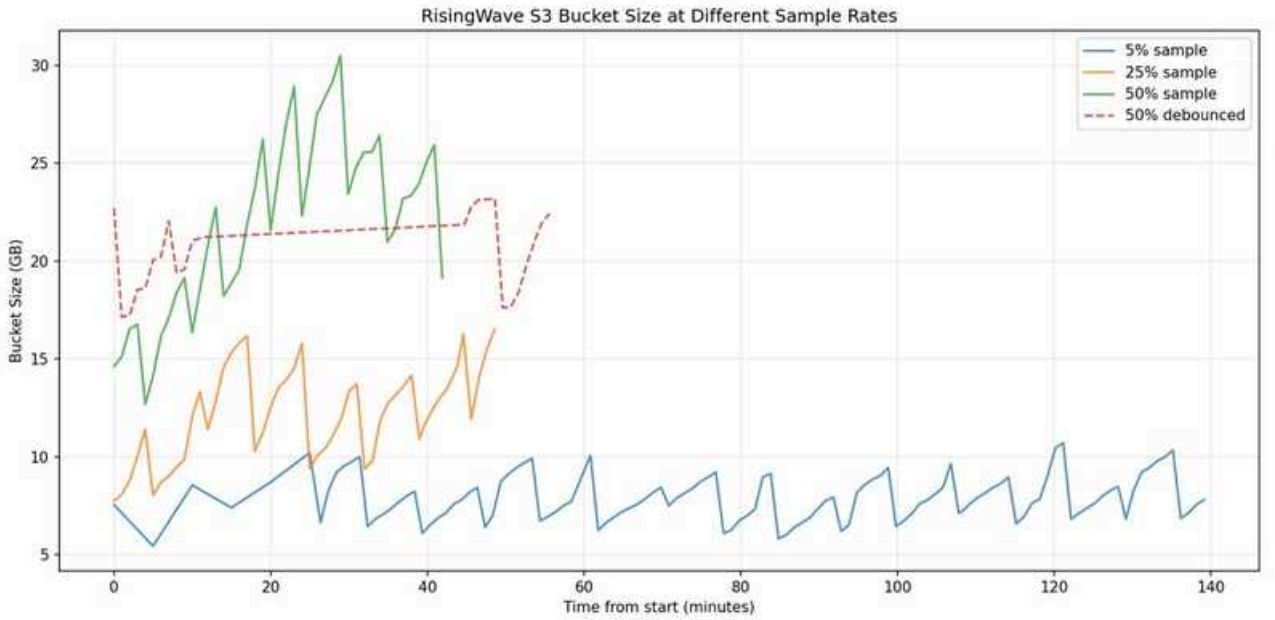
- deployed presence services to prod, did capacity test
- optimized RTC event emission, ~80-90% less events emitted
- estimation from all datapoints: S3 storage planning

We have RTC events flowing in:


And network from `luki-rtc` increase as expected:




Deployed initially at 5% sample rate, monitored. See below for S3 bucket size:




Then gradually ramped up to 50% sampled. At 50% the bucket was growing *very* fast at around 2GB / minute, so I decided to look at RTC to reduce events coming in.

RTC debounce
#4224 · Opened by balintb

Merged ready-for-ci ready-for-review

By debouncing RTC events (updates are capped at 1 / sec) we gained a ~80-90% reduction in events, less work for RTC, less work for RisingWave. CPU graphs showed this nicely:

Learnt a lot about RisingWave today. It works similar to Cassandra: writes go to a LSM tree memtable, memtables are flushed to S3, once they reach an entropy limit they're then compacted by reading them back from S3, compaction in memory, writing it back. The CPU spikes above are compactions happening. RW does a very good job of utilizing 100% CPU for this.

• • •

Once I had all collectors set up, started gathering datapoints for S3 storage, ops, and number of objects.

This was quite noisy, as the bucket would constantly grow - shrink - grow - shrink due to compactions happening. This can be seen on the S3 bucket size graph above.

From the collected datapoints, analyzed the data with Claude and Python and did capacity planning for S3 storage and ops.

See detailed planning at


S3 storage planning

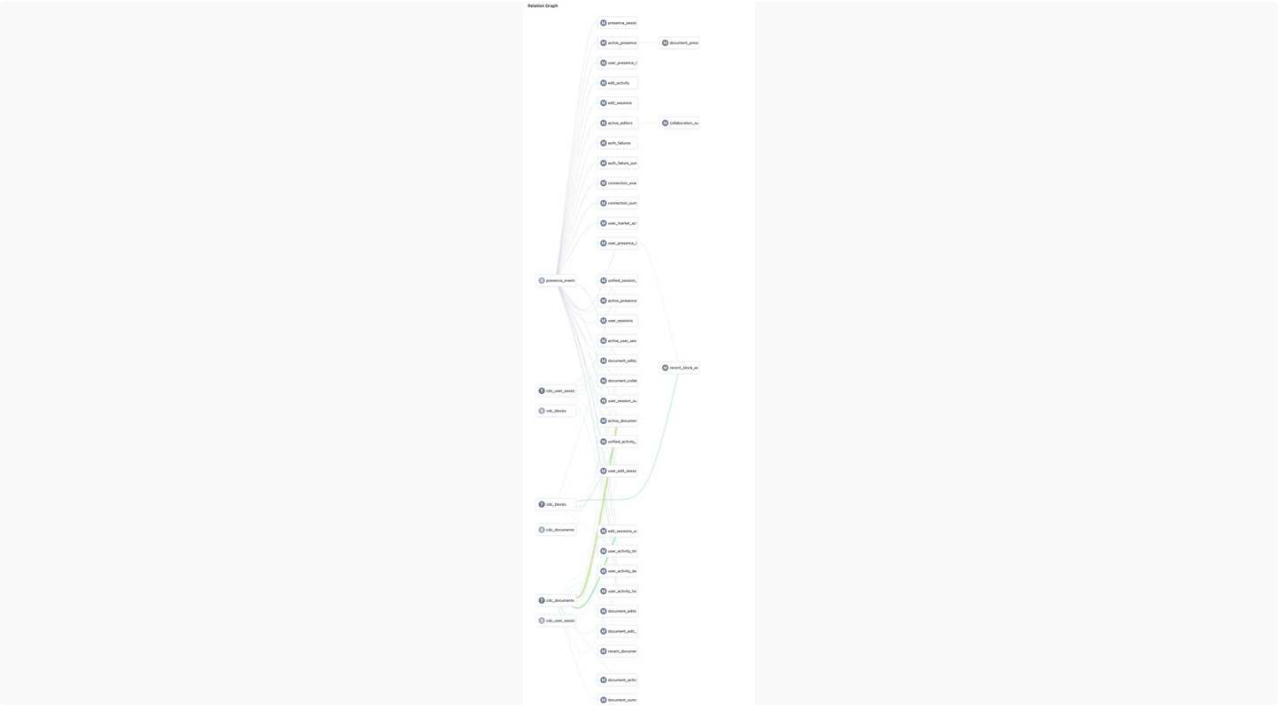
> The total monthly cost without any optimization, for all users, and all expensive materialized views tops at ~700USD / month. My intuition based on today's data and seeing the characteristics of the system is we're going to be able to operate at 50% of this monthly cost.

From capacity planning, came up with a plan to further optimize costs with smaller tweaks to RisingWave:

- reduce checkpoint frequency. This runs the risk of data loss if the service dies.
- add memory. Bumping to 16GB would mean less paging and less compaction - more memory == larger cache == fewer S3 reads.
- Elastic disk cache can be enabled, adding an extra buffer between mem and S3. These ops are free. Compaction can happen faster if reads come from disk. Benchmarks ***claim*** 90% reduction in S3 reads. See below for how this would impact our costs - extra ~40USD / month for more mem.

• • •

RisingWave gives us a nice overview of the dataflow into the materialized views:



But more importantly, we can monitor latency and backpressure on this diagram:


Latency from RTC / CDC event to RisingWave is around 5-10sec. Diagram above shows high latency of 22 seconds before optimizations.

## Next up

- we will keep the system running at 50% (debounced) overnight and gather at least 24h of datapoints. A this stage a further refinement can be made of the capacity and cost estimation
- after longer test with 50%, we will open it to 100%

- → testing with an actual agent. Wrote the tools for this today, will be merging to sandbox tomorrow

## Christoph Locher

Jan 27., Tue

- Worked on getting the agent to handle all text block properties reliably
- Added very strict HTML validation with good error messages, that before the Agent is allowed to write
  - This encodes all of our rules regarding block types, their properties, which options can be combined and which excluded. The HTML cannot contain anything unknown, everything needs to be defined with in the system of Craft
  - This works as we hoped and even better than it did in the protoype, because it is even stricter. Now the agent learns very quickly what to do.
- Did lots of roundtrips and testing to make sure all possibilities are covered

**At the end of the day it seems that all text block operations and settings, including inline styles, work reliably.**

Lists are still a little flaky, but for the moment work well for text blocks.

Because these worked well I started moving on to the other blocks, starting with the simple ones: fixing all properties for Separator blocks (except the detailed Washi textures) and Images.

Jan 28., Wed

As text blocks are already covered, I'll move on to the remaining blocks (Code Block, Link Block, File Block, Drawing Block and Whiteboard Block).

Gyula Halmos

## Craft Agents

- Started a few improvements and bugfix branch: ghalmos/bugfixes-and-search
- Mainly install app updates and proper search inside sessions cc Viktor Pali
- The functionality is finally fine, the UX i am still polishing
- Also started to evaluate how to introduce hooks and what format. I have a working concept, but it is still forming:
  - Have a solid event stream setup with an event bus, it is something that is easy to over engineer
  - Sqlite+nanoevents looks solid, BUT
  - My approach is to use our mantra - “stand on the shoulders of giants” - want to look up how successful desktop apps do it for inspiration
  - I really want it to be simple and robust as a lot of features can be based on it in the future

## Next steps

- Finish the bugfix and search branch to be ready for main
- Continue the above mentioned conceptualization and create a fiest iteration and connect the hook concept i came up with

Jan 28., Wed i will be half a day off(after noon)

Zsolt Varnai

Jan 27., Tue

**Assistant Editing**

- **E2E Test Infrastructure**
  - Created full end to end test setup, that involves the client, service and anthropic server
  - First with sequential test running, then upgraded to parallel test execution with worker coordination
  - Headless version for fast running the whole suite, but selected tests runnable on the current doc / assistant window for investigation
- **Test Cases**
  - Created and executed plan for extensive test suite with several categories
  - Block Creation, Block Updates, Inline Formatting, Block Attributes, Delete Operations, Move Operations, Reorder Operations, Multi-Page, Validation, Edge Cases, Round-Trip, Integration, Block Types, Page Properties
  - Started investigating and fixing various failing test cases
- **Undo / Redo**
  - Added undo / redo support for the document sync operation

Jan 28., Wed

- Continue with editing support
  - Work on block moving / subpage operations

↑ Daily Notes - 2026 January

## Jan 27., Tue

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · Edit Mode milestone deadline (W...

↑ Jan 27., Tue

## Agents

### Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Edit Mode milestone deadline (Wed Jan 28)** — Target: "All editing operations on text blocks work reliably, the agent handles a single page" by tomorrow evening. Christoph and Zsolt are dividing and conquering text edit operations. Status check on progress.
- **OAuth token update shipped** — Gyula shipped the OAuth update removing Claude Code dependency. Notes potential "soso UX" for users with old CLI-based OAuth tokens that expired (restart and re-auth required). Also found breaking behavior on Windows for clean installs without Claude CLI.
- **Presence/RTC infrastructure live on prod** — Balint's RisingWave and RDS infrastructure is now running on prod (not ingesting yet). Load test scheduled for this morning, then: start RTC event stream → start CDC connector consumer → run source init → "fingers crossed". This is a critical milestone.
- **Craft Agents roadmap exploration** — Gyula outlined potential roadmap items including: improved search across sessions, notifications, OSS cleanup, analytics approach (with detailed opt-in principles), shareable sessions for team collaboration, and Hooks feature. Worth prioritizing for next 3 weeks.

## Craft Edit Mode Implementation Plan

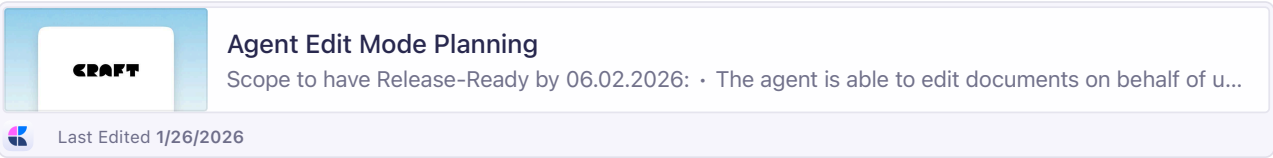
Scope to have Release-Ready by 06.02.2026:

- The agent is able to edit documents on behalf of users.
  - This includes adding, removing and moving all block types [except tables and collections]; changing any property on these blocks that is exposed in the Format sidebar [except Card styling]; creating subpages and moving blocks into subpages; adding and changing inline styling in text blocks.
  - Excluded are all actions regarding Tables and Collections, all actions performed only in the document (comments, reactions, reminders, task schedule date and due date, repeating tasks), all actions performed at the page level (actions in the Style sidebar)
  - Excluded are space-level actions like moving documents, creating folders, etc.
- The agent can switch between Explore mode, where it reads documents and presents a plan for changing it, and Edit mode, where it executes a plan of performs actions immediately
- The plan is presented to the user inside the Chat in the existing markdown format, the user can request changes or approve

### Milestones

- Wednesday 28. Evening: All editing operations on text blocks work reliably, the agent handles a single page
- Friday 30. Evening (pot. buffer monday): All editing operations on all block types (except Tables & Collections) work reliably, the agent can handle subpages
- Tuesday 03. Evening: All existing functionality works reliably with the new agent system
- Wednesday 04 Evening: The agent has an explore mode, where it only reads on doesn't write

More details on the timeline and the individual packages:



## Christoph Locher

Jan 26., Mon

As I was still in the mindset, I quickly finished up the HTML converter prototype, adding full task support, table support and collection support. The Readme now contains a preliminary spec for how these could be implemented in line with our existing principles, that we can come back to later.

README.md
File src/prototypes/craft-html/converters/README.md
lukilabs / document-edit-plan-mode Branch main

Add task support to JSON-to-HTML converter — - Task status: done, undone, cancelled (via isTo...

Add table support to JSON-to-HTML converter — - Add isCode support to inline text rendering...

Add collection support to JSON-to-HTML converter — Co-Authored-By: Claude Opus 4.5 <nore...

Then I worked with Zsolt and Tamas to set up my local system so I can run the new agent service and use the new Agent SDK in my app and I was able to perform fist edit operations with the agent.

I synced with Zsolt several times and we worked out a plan (see above).

Jan 27., Tue

Discussed with Zsolt that we will devide and conquer all basic text edit operations, as this requires a lot of back and forth and testing to make sure everything works correctly.

## Balint Bartfai

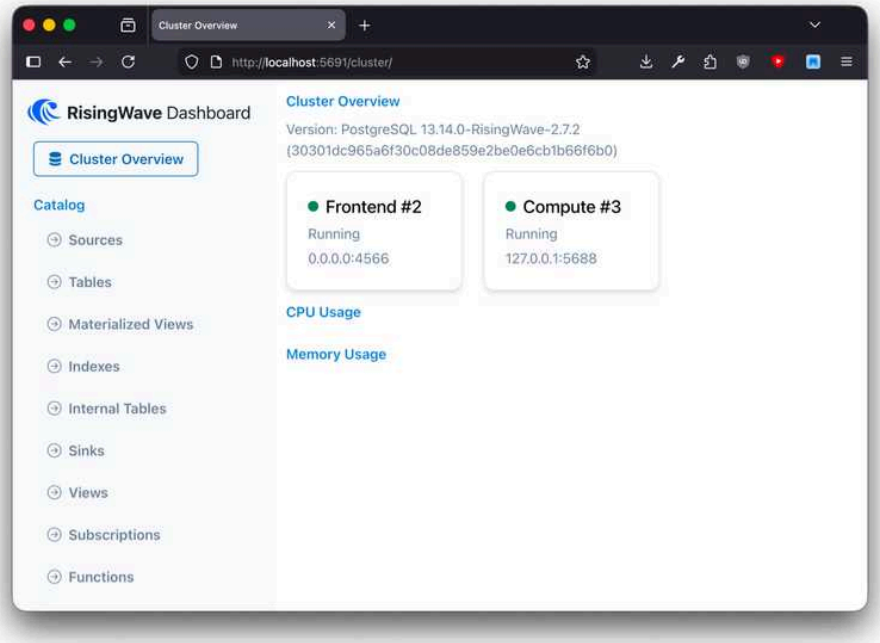
Merged final modifications to presence, RTC and CDC connector. Infra running (but not consuming events yet) on prod.


- As per design meetings with Tom, removed postgres CDC connector (direct from WAL) - it's too risky for production. Instead, rewrote the CDC → RW to use a lambda consumer listening to debezium to inflate (batch) events from that stream, then output to separate plain JSON kinesis streams which then gets ingested into RisingWave. This adds a couple more streams, however, it's much less risky, and allows us to enrich events in the future - this way we only depend on CDC format for *ingest*, so we can use any format (in sync with RWs MVs) as egress to streams
- Redid partition handling as per comments
- Hit rate limits on docker registry pulls → set up ECR registry to pull RisingWave images from. Startup was consuming too many pulls due to network issues (with the new infra) on prod.
- We're now using the ARM image for RisingWave. RW is written in Rust - native ARM64 images available. This allows us to run on Fargate Graviton. It's 20% cheaper per vCPU hour as a base reduction, with spots this can go up to 70% savings. No perf issues expected, streaming is mem+IO-bound.

Modified the RTC service so that it samples RTC events based on a consistent hash of `spaceId`, allowing us to ramp up traffic. Currently it's set to 5% for prod.

> **RisingWave and RDS** - infra for presence - **is running on prod**, currently without ingesting anything. Load test in the morning.
>
> → start RTC event stream
>
> → start CDC connector consumer
>
> → run source init in RisingWave
>
> → fingers crossed

Connection to RisingWave can be made with the same method as connecting to the prod DBs - jumpbox SSH tunnel, then `sql.risingwave.prod.local:4566` as the host. You can also use `:5691` (via SSH tunnel) to view the dashboard:



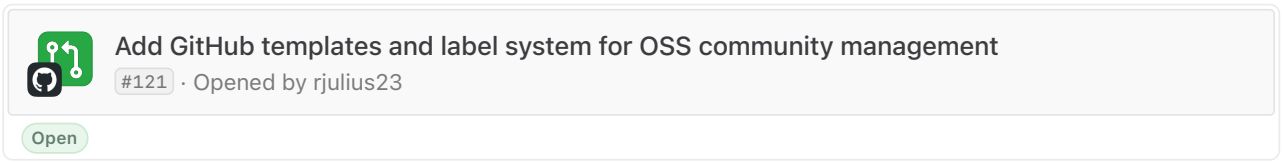
Gyula Halmos

### Release mayhem, and important lessons about hte SDK

- Today shipped the OAuth update for Claude token
  - No more reliance on Claude Code - causing refresh issues due to different oauth flow compared to the native one
  - Also introduced mid-session expiration handling (tested as much i could) - not 100%, but once all user switch to native oauth flow it should be OK
    - there could be some soso UX if a user has an old CLI based oauth token that expired - restart and re-auth cleans it though
- Removing Claude CLI dependency had an unexpected impact on clean installs without Claude CLI - some breaking behaviour on windows from Agent SDK
  - My mistake not checking it, learnt the lesson, but also with the new 2-step release i feel a bit more confident - there is more space to test the final version

### OSS Management

- Started to prepare a PR to organize the OSS project a bit better
  - Labels
  - Issue templates
  - Workflows




### Exploring potential roadmap for Craft Agents

- Small, but quick wins:
  - Improved search across sessions, not just title but inside as well - Unified Workplace concept
  - Notifications - configurable, actionable would be an addition - Better monitoring of the workflows
  - Clean-up issues coming from OSS - Stability
- Important for scaling:
  - Better analytics approach:
    1. **Default off** - Never auto-enable
    2. **Tiered opt-in** - Let users choose comfort level
    3. **Transparent code** - Open source the analytics component
    4. **Public dashboard** - Show aggregated data openly
    5. **Clear purpose** - Explain how data improves the product
    6. **Minimal data** - Only collect what's needed
    7. **No nagging** - Ask once during onboarding, accept "no" gracefully

- New features:
  - Shareable labels like RSS feeds - Collaborative Team Work
    - People can share sessions between each other not jsut hte content, but continue each others works for example, or work within the same context
  - Introduce Hooks - To be aligned with Claude Agent SDK capabilities
    - Challenge: how to make it available for non-power users

## Next steps

- Plan out the roadmap for the next 3 weeks
- Spend some time on the OSS community - issue administration, start discussions, set up machinery, fine tune guides

Zsolt Varnai

Jan 26., Mon

**Assistant Editing**

- **Support**
  - Helped Christoph setting up the new V2 agent service and iOS proto for end to end local testing
- **Planning**
  - Planning with Christoph about the next 2 weeks scope
- **AI Agent Service V2**
  - Created a new version of the AI Agent service and its related AWS resources to isolate it from the current production version and have both available concurrently
  - Discussion with the services team about the plan for productionizing the codebase based on the recently updated services tech stack
- **Edit Flow**
  - Testing with Christoph's usecases and improving the behavior with improved agent instructions
  - Fixed various styling conversion and diffing issues
  - Moved inline HTML conversion from iOS to server side
  - Enabled Edit tool with sync support for incremental HTML editing
  - Improved CSS usage and fixed highlight colors

Jan 27., Tue

- Continue with editing support

↑ Daily Notes - 2026 January

## Jan 26., Mon

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · Presence prod deployment today...

↑ Jan 26., Mon

## Agents

### Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Presence prod deployment today** — Balint pushing prod test to Monday (today), working with Tom. Changes finished late Friday. Sandbox shows 93MB S3 data in 24h (not representative of real usage).
- **RisingWave setup complexity** — Parts of Presence infra still not in CloudFormation. Setting up MVs on RisingWave noted as tricky. Discuss any blockers.
- **Write Tools HTML conversion progress** — Zsolt created roundtrip tests (block → HTML → block), implemented page block handling with subpage titles, validation logic. Continuing today. Consider: need HTML format spec from Agents team (mentioned by Levy in Services).
- **Cross-team dependency with Services** — Levy (Services) needs HTML format from Agents team for MCP V2. Coordinate on this.
- **Gyula's workflow experiments** — Interesting flow discovered: task with Craft Agents → upload to Craft Docs → review/edit in Craft → pull changes via iMessage. Communication with agent is key need.
- **Labels feature with Haiku** — Gyula suggests triggering Haiku by default for editing (fast and accurate for simple name changes). Worth exploring.
- **Auto-update testing infrastructure** — Gyula set up Parallels VMs (Windows 11, MacOS) and Kamatera Cloud for Ubuntu testing. Test docs created.

### Balint Bartfai

- Found parts of the Presence infra still not in CloudFormation. Went through this, verified with Claude, did a test deploy on sandbox to make sure
  - → some parts are still tricky, to setup RisingWave we need to set up MVs on RW
- Prod readiness
  - Currently sandbox created 93MB of S3 data in 24h. This is not representative as usage is just us, but I was expecting much more. We will see with prod
- We will push prod test to Monday, I finished changes late afternoon. Working with Tom on Monday
- ...WIP. Will be working Saturday.

## Zsolt Varnai

Jan 23., Fri

**Write Tools**

- **HTML Conversion Roundtrip Tests**
  - Created test suite and various tests for HTML conversion roundtrip (block → HTML → block)
  - Tests verify that block attributes are preserved through the roundtrip
  - Covers text styles, list styles, indentation, alignment, code blocks, and nested blocks
- **Page Block Handling**
  - Implemented root page vs subpage title handling
  - Subpages with title in <head>, with validation to make it readonly. Title can be only changed from parent page
  - Root pages are exception as they have no parent. They have special title in the body that can be edited.
- **Block Update & Validation**
  - Added all validation logic to pretool hook for correct feedback to the agent
  - Added validation automated tests
- **CSS**
  - Extract CSS to standalone server side
  - Added reference about valid attributes to CSS comments
- **HTML conversion + diffing**
  - Added tests for various block styles and attributes to be converted and updated properly
  - Started fixing attributes diffing and block update issues

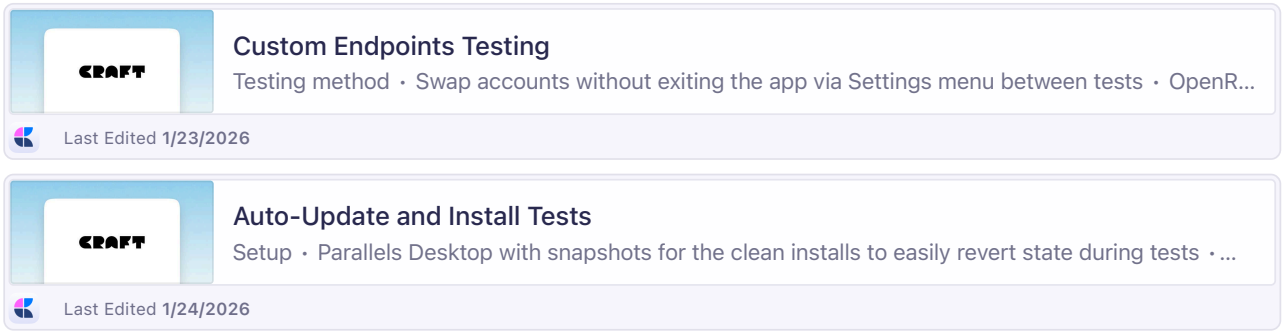
Jan 26., Mon

- Continue with write tools proto

Gyula Halmos

### Lot of manual testing and setup

- Set up Parallels Windows 11 and MacOS VMs (Ubuntu would be arm64 hence not, but probably should revisit what were the build issues with it so i can stay all the way in parallels and get it for the testing platform - parallels CLI is a Pro feature but can be useful)
- Set up Kamatera Cloud for Ubuntu testing
- Test docs:




### Played around with Labels amazing stuff (inspired by Lisa Skorobogatova 😛):

- For editing i think it should trigger default with Haiku - for simple name changing it is really fast and accurate
- Always telling the agent to change the name feels clunky, dont know better solution for now, but thinking about it
- Set up my private set up and created a New skill called sync that syncs my .craft-agent folder to a private Github repo this way i can store my ideas and projects in Craft Agents and can get back to it even if i screw up

### Experiment on how can i adapt my workflows with Craft Agents

- I had a really nice flow:
  - Start a task with Craft Agents and ask it to upload the generated documents to Craft Docs
  - Then I reviewed the docs made changes in Craft Docs and then asked the agent via iMessage to pull my latest changes and continue accordingly
  - It worked quite well, needs more polish but i liked this flow:
    - Nice editing and reading experience via Craft Docs
    - I could do it from my bed while the laptop was on my desk doing the heavy lifting
- I really need to be able to communicate with it, the simplest solution is iMessage as i have full control over it (mostly)
- I wanted a solution without code change because we already have a lot of features and it is hard to keep up so better to reuse new functionalities
- I created a Skill + iMessage source it works great, needs some more polish

- Claude Code hooks should be exposed as well in Craft Agents it would make these features more robust 🙂 it is worth exploring
- My inspirations:
  - Just a moment...
    https://medium.com/@lakshminp/claude-code-hooks-the-feature-youre-ignoring-while-babysitting-y...
    https://medium.com/@lakshminp/claude-code-hooks-the-feature-youre-ignoring-while-babysitting-your-ai-789d39b46f6c
  - mylee04 / code-notify
    Cross-platform desktop notifications for Claude Code/Codex/Gemini - Get alerts when tasks complete
    mylee04 Shell Stars 64
  - karanb192 / claude-code-hooks
    🪝 A growing collection of useful Claude Code hooks. Copy, paste, customize.
    karanb192 JavaScript Stars 92

## Next steps

- Sync with Balint Orosz
- My ideas:
  - Look into OSS issue management, clean up stale/fixed issues at least reply on those
    - Clean up PRs - can discuss
  - Spend some time on how to better conclude multi platform testing
    - Challenges - rich feature list - hard to test all aspects without automation
    - Create a priority list of features to be tested on each release
    - Create a release test suite that should be part of the release process
    - Figure out in which areas do we consider the OSS fixes, can we have a reliable test harness to accept non UI fixes
- Look into ways on how to do scheduled tasks
  - Without code change - using skills, hooks, 3pp solutions
  - Evaluate how can it be showed the users prolly via OSS / youtube video / X post .. etc

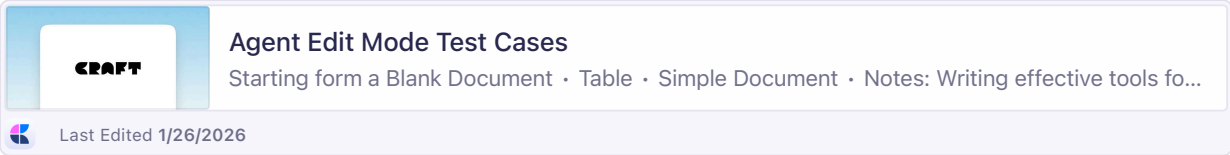
Ideas From Balint Orosz

- Cli sources (mail, calendar etc)
- Onboarding skill packages (marketing, design, frontend devs etc...) - pre packaged, ready to roll

## Christoph Locher

Jan 23., Fri

- Sync with Zsolt on page-level property handling
  - → keep title read-only in subpage html, but handle the page-level props of the block [e.g. backdrop, document color, default separator style, ...] in the subpage html, because this is where they take effect. This means a block's data may be spread throughout two html files (title & block styling in parent html, page props in subpage html), but it is never duplicated (property is always either in one file or the other, never in both). This matches how users see the props in the UI – a block's document color is only shown when you look at the Style sidebar inside the subpage.
- Collected more test cases and prompt examples from a user's POV and shared them with Zsolt for testing




- Started exploring how to handle collections, as they are the last complex case we haven't considered until now:
  - some collection items are subpages, but other's aren't therefore it might be best to display them in a HTML table, with the data-has-children attribute added for the rows representing collection items with children, so they work exactly like subpages.
  - following our "no duplicate data" principle, the collection item's field values would only be present in the table, in the parent page. This makes sense, as there we can also add the possible values as data properties of the table itself (e.g. table knows all possible values of a multi-select field)
  - one drawback is that in the UI, the user sees the collection item's field values both in the parent page in the collection and in the subpage view at the top of the page. This is different from what the agent sees. So we need to make sure the agent understand that when the user refers to a collection item from its subpage view and asks to change a field, that it needs to edit the parent page's html. But it should be smart enough to do that.

Jan 26., Mon

- Extend the proto html converter with tables and collections as a proof of concept
- Start testing prompts in my agent fork with the HTML generated by the Swift implementation

↑ Daily Notes - 2026 January

## Jan 23., Fri

↑ Jan 23., Fri

# Agents Team

## Recommended for discussion

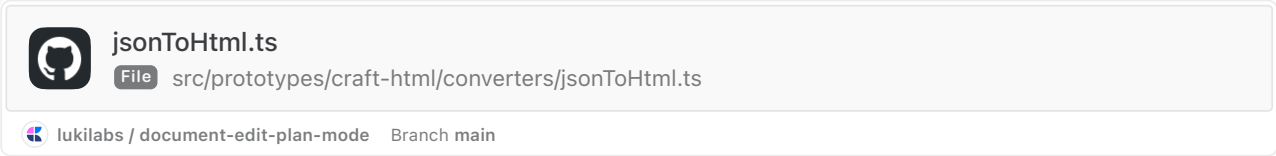
*generated by Daily Team Discussion Points Agent*

- **Balint needs help with AWS LB DNS** — Explicitly stated "I'll need help with this one" — AWS Load Balancer DNS not working to reach RisingWave on dev. Who can assist?
- **RisingWave prod rollout imminent** — Sandbox successful, planning short prod test (10min → 1hr → longer). Balint needs to sync with Tom. Ensure everyone is aware of the test window and monitoring plan.
- **Christoph waiting on Zsolt's feedback** — HTML converter ready for Zsolt to take as input and convert to Swift. Check if Zsolt has reviewed and provided feedback.
- **Tables and collections excluded from HTML converter** — Christoph noted these are "very complex" and excluded for now. Is this blocking anything or acceptable for the proof of concept?

## Christoph Locher

I iterated on our HTML creation, changing the converter to go directly from Craft JSON output to HTML, without the step through my AST format. Then I made it more comprehensive and added to it all the different block types and styling variants (e.g. including the different styling options and background for cards, washi tape configuration and so on). Essentially the HTML should really contain everything the user can see in a document.

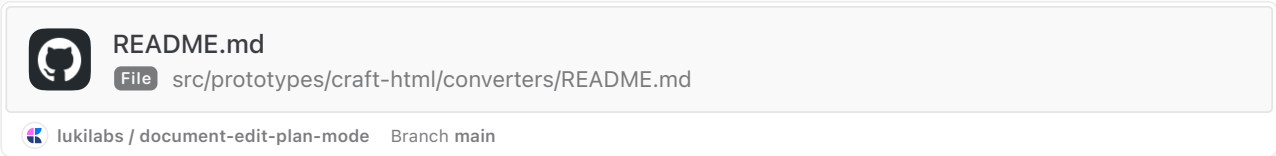
For now I excluded tables and collections, because they are very complex – once the we have the proof everything works, I will can add those as well.




The converter outputs a nicely formatted HTML document that uses semantic tags and classes – just good, high-quality HTML.

The plan is for Zsolt to take this converter as an input and work with Claude to convert it to Swift.

Then I worked on updating the documentation for the parser. Essentially this describes all the possibilities of styling inside a Craft doc, both from a technical point of view of converting but also from a UX point of view how users use the properties.




This will be additional info for the agent to understand how Craft documents work, what is possible and what users mean when they use the Craft UI terms, not HTML (Users will say “Make all subtitles blue”, not “Make all H2’s blue”).

I also worked on developing test cases, collecting example documents and for each document several example prompt of what users could request and what they would expect the agent to do, so we can start testing the correct behavior.

Tomorrow I will continue refining the parser and docs based on Zsolt’s feedback and also create more test and example cases (and travel home).

## Balint Bartfai

Fixing issues (still) with the RisingWave setup

- S3 creds were expiring - and since RW uses OpenDAL for S3 auth (spoiler alert: it's terrible), it can't refresh at all after expiry. So I changed it to an IAM user
- AWS LB DNS still does not work for some reason to reach the RW on dev. I'll need help with this one
- some MVs were not persisted, some infra was not in CloudFormation

All-in-all, rollout to sandbox succesful. Had a chat with Balint, we will be trialling this setup on prod to get some numbers as soon as it's safe.

The plan is to switch on for 10min, observe. If all metrics look good, keep it for 1hr. If it still looks good, keep it longer so we can do capacity planning and cost estimates.

Anticipated largest cost will be S3. Risks are RW on ECS cannot consume blocks CDC with one node - then we will have to scale out (which requires 4 different ECS apps - see https://risingwavelabs.github.io/risingwave/design/architecture-design.html). RW consumes the CDC from DB1s WAL, not Debezium Kinesis - I don't anticipate any issues but it's a different form of logical replication. S3 size and - because there are many files - S3 r/w ops are a big question, especially with small block updates. We won't need all MVs as they are defined now, but for the test we **will be using all of them** to stress-test RW.

## Prod rollout plan

First, `Enabled=false` flag set and cloudformation deploy.

- Hooks and metadata small RDS comes up first
- RisingWave ECS comes up next

Via jumpbox:

- Connect and Init RW RDS
  - set database `risingwave_meta`, set users `risingwave`
- Temporary enable RisingWave `Enabled=true`
  - RW creates meta tables
  - RW materialized views init
- Disable RW `Enabled=false`

It's a go

- Start RisingWave `Enabled=true`

To stop, killswitch:

```
aws ecs update-service \
  --cluster ecs-fargate-cluster \
  --service risingwave-prod \
  --desired-count 0
```

## Next up

- sync with Tom on the test
- if all is prod-ready we will be conducting a short test. If that goes well a slightly longer one.

Zsolt Varnai

Jan 22., Thu

**Write Tools**

- **Subpage creation**
  - Fixed issues with subpage creation flow
  - Added info about the HTML structure to the system prompt
- **HTML parsing**
  - Moved HTML parsing to server side using **htmlparser2**
  - Simplified architecture by removing unnecessary ID mapping from the server side (server only receives mapped IDs)
- **End to end flow**
  - Fixed issues with unreliable tool calling
  - Fixed issues with flaky subpage handling
- **HTML conversion**
  - Ported Christoph's HTML schema and conversion proto
  - Moved the conversion from swift to the service, so now both mapping directions happen server side and the client does not need to know about the HTML
- **Other**
  - Added parentId to search tool result to be able to use the HTML read after finding the block
    - The agent wasn't fully sure the block IDs can be used, so we can improve on this later on
  - Found an issue of the tool results not being streamed properly to the client, so fixed the event handling and ensured matching IDs
  - Tested new "full proxy" AI gateway integration, added anthropic version header

Jan 23., Fri

- Continue with write tools proto

↑ Daily Notes - 2026 January

## Jan 22., Thu

Agents Team
Recommended for discussion · generated by Daily Team Discussion Points Agent · RisingWave prod feasibility - Bali...

↑ Jan 22., Thu

## Agents Team

## Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **RisingWave prod feasibility** - Balint needs 24h of data to calculate if prod can handle block updates via CDC. Currently at 365MB with 743 objects since mid-afternoon. Decision point: if prod can't handle it, will need to aggregate into 5min buckets and discard raw history.
- **RisingWave crash recovery** - Had a cluster ID mismatch after crash that caused MV definitions to be lost. Now fixed with meta stored in DB2, but worth confirming recovery process is documented.
- **0.2.24 release issue** - Gyula mentioned "screwed up a bit 0.2.24" - any follow-up actions needed?
- **CX Team feedback on Craft Agent** - Abner reported context compaction lost work mid-session. Relevant for agent reliability improvements.

### Balint Bartfai

RisingWave is now **S3 backed** - data is stored in its own format and persistent.

I linked up the **event streams with CDC** from RDS sources

- `dev_realm_documentdatamodel` - document metadata changes
- `dev_usersessionlog` - user session lifecycle
- and finally `dev_realm_blockdatamodel` - block-level content changes

This has allowed RW to now provide

- presence MVs from Kinesis stream (RTC) - who's online, who's editing now, where (doc) a user has been, grouped edit events per session
- document MVs from CDC - edits, edit sessions, recent activity, editors
- session MVs - active sessions, per-user session stats, JOIN-d session activity with RT presence
- cross-stream JOIN-s
  - who's viewing what content
  - enriched block edits (user info)
  - active collaborations right now
  - all user activity combined
- live, 1 hour, 24 hour, 7 day, 30 day rollups of document, session, edit activity
- "hook queryable" events - this view is optimized for the hook system queries

Edits have been debounced into "stable" views to provide a better interface for queries (30sec inactive).

In addition to this, RW metadata is now stored in DB2, which means on a crash of the service, all MVs and all server config is preserved. This was needed as RW crashed, came up again, but could not use the S3 storage due to a cluster ID mismatch. MV defs were lost as well (they can be recreated easily as all of them are in code). With meta in DB this will not happen again.

Since CDC from blocks is quite a heavy stream, I will do calculations tomorrow after 24h of running to see if this would be feasible on prod. Currently (at 8:30pm, since mid-afternoon today) we are at 365.38 MB with 743 objects in the S3 bucket that RisingWave stores its data in. This is with all different types of MVs, and full retention (no "raw → rollup → delete" enabled) which we can definitely reduce. But we need to know if prod can handle block updates or not - if not, we could be aggregating them into 5min buckets and throwing out raw history after that.

### Gyula Halmos

#### Craft Agents

- Screwed up a bit 0.2.24 but with positive intent hehe
- Simplified and unified the build and release process now it can be triggered on git tag push to the main also simplified and automated somewhat the version management (it can be further improved)
- Single command release trigger with `bun release`
- Was only half a day but started to collect ideas and also organize my workspace in Craft Agents a bit better:
  - Craft
    https://agents.craft.do/s/yHPMp7pCIA4I1eWoUGENe

#### Next steps

- Do a bit of research in the open source community about similar apps
- Streamline my workflow (started already see above)
- Start to step back a bit and think more strategically about the app

### Christoph Locher

- Bit of a mixed day with lots of meetings
- In the afternoon, worked on understanding and tweaking the agent code and logic
- Edit the Agent UI to make it less technical
  - Removed tool call detail view and turn detail popover, tool parameter display
  - Moved mode selector into the chat input, next to model selector to simulate app UI
  - Simplified message styling, loading indicators, turn header, tool call titles for Craft tools
  - lots of other small tweaks
- Edited the system prompt some more, tried to remove some too restrictive descriptions to make it more universal
- Also pushed a few smaller UI fixes for things I noticed throughout the day to the Agents repo

### Jan 22., Thu

- Continue working on agent ui, tools and system prompt

Zsolt Varnai

Jan 21., Wed

**Write Tools**

- **HTML parser**
  - Did a second round of measurements, now running in node.js and focusing on js libraries.
  - Based on performance I'm planning to use **htmlparser2**, but as the APIs are not that much different we could switch in the future if needed (eg to parse5 if HTML5 compliance becomes relevant)

| Library | Language | Total | Patches | Granularity | Diff Algo | HTML5 |
|---|---|---|---|---|---|---|
| htmlparser2 | JavaScript | 16.28 ms | 2351 | Node | custom | No |
| parse5 | JavaScript | 27.30 ms | 2351 | Node | custom | Yes |
| node-html-parser | JavaScript | 52.08 ms | 2351 | Node | custom | No |
| cheerio | JavaScript | 54.02 ms | 2351 | Node | custom | No |
| diffDOM | JavaScript | 62.69 ms | 51 | Subtree | built-in | Yes |
| linkedom | JavaScript | 94.23 ms | 2351 | Node | custom | Yes |

- **Page level HTML infra**
  - Created new HTML export client API, where a document is exported into multiple HTML files
  - The server creates all of them when the session starts
  - New save API that updates a single page based on the HTML, with deletes/moves restricted
  - We could potentially allow local moves, where the existing subblocks are reordered with a single write
  - Switched to using mapped document and block IDs in the file names, so the agent does not need to use the long GUIDs
- **Block creation support**
  - New blocks in the HTML without ID trigger the creation of a new block
  - The agent got confused by the structure, so started to iterate on the system prompt and HTML format
  - Removed the parent block from the page HTML and moved the title to the head
  - Started to investigate various issues as the new block was not always created properly

Jan 22., Thu

- Continue with write tools proto

↑ Jan 21., Wed

# Agents Team

## Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **RisingWave S3 credentials issue** — Balint deployed presence stream infrastructure but RisingWave still has issues with S3 credentials. Works with ephemeral storage for now but needs fixing for production persistence.
- **Mac build timed out during signing** — Gyula reports the Mac build failed overnight and he reverted marketing page links. Needs to investigate in the morning if the night build didn't complete.
- **Windows permission issue on Craft Agents** — Gyula couldn't reproduce a bug where installation under LocalData has permission issues. Needs to test on actual laptop (not Parallels).
- **System prompt approach for new assistant** — Christoph is exploring whether to use Claude Code's output styles vs. completely replacing the system prompt. Team should discuss trade-offs and share findings.
- **Presence system can now answer realtime questions** — Balint's infrastructure is working! Can query things like "How many users were active in the last hour?" Worth demoing capabilities to team.
- **Error handling and trace collection for Craft Agents** — Gyula wants to think about better production debugging for issues like session corruption and window state sensitivity. Good architecture discussion topic.
- **Cross-team:** Christoph and Zsolt are integrating existing app tools with Agent SDK. This work affects how the native app assistant will behave.

## Christoph Locher

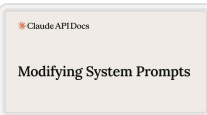
I started working on bringing our threads together and work integrating the new agent sdk, our existing tools and prompts and the new editing and planning logic into one system, at least from the user's perspective.

To do this, Zsolt and I hotwired the existing tools into my Craft agent fork. We hardcoded the tools and tool descriptions into the agent and have it run a websocket server, which the mac app connects to. This way the Craft agent can access the tools exactly the same way the Assistant chat currently does.

I also copied over our existing system prompt from the app to begin the process of tweaking it for the new Agent SDK. To start, I completely disabled the default Claude code prompt and only use our own one. It will need lot more testing and exploration to fine tune everything, and figure out the best approach.

Alternatively to completely replacing it we could also just copy parts of it into our own prompt.

Then there are other recommended methods from Anthropic to modifying the system prompt:




**Modifying system prompts**
Learn how to customize Claude's behavior by modifying system prompts using three approaches -...
https://platform.claude.com/docs/en/agent-sdk/modifying-system-prompts#when-to-use-output-styles

Output styles seem kind of promising:

> Output styles completely "turn off" the parts of Claude Code's default system prompt specific to software engineering.


**Output styles - Claude Code Docs**
Adapt Claude Code for uses beyond software engineering
https://code.claude.com/docs/en/output-styles

Output styles keep most of the Claude code system prompt, including the tool instructions and safety instructions, but override the coding specific instructions to allow you to build an agent for non-coding tasks.

This repo collection all parts of the Claude Code prompt (it is quite large and assembled from various parts depending on different criteria):

Piebald-AI / **claude-code-system-prompts**
All parts of Claude Code's system prompt, 20 builtin tool descriptions, sub agent prompts (Plan/Explore/...
Piebald-AI JavaScript Stars 3309

In the main system prompt you can for example see the a section which is overridden when an output style config is defined:

**system-prompt-main-system-prompt.md**
File system-prompts/system-prompt-main-system-prompt.md
Piebald-AI / claude-code-system-prompts Branch main

So while it's to early to say, this might be an interesting approach to check if we notice that we los a lot of intelligence without the Claude code prompt.

Other than I am starting to understand the message system of the Craft Agents app, and adding some additional logging for myself, so I can better see which kinds of information are available. My focus right now is to start integrating all the parts together, and at the same time tweaking everything to be more friendly for non-technical users (nicer naming, no exposed tool parameters, etc.)

So I have now got the system set up and am in the process of editing the prompts and the UI to prototype the behavior we want to see in the native app for the new assistant. There are a lot of moving parts to this, so I will continue with that tomorrow.

## Balint Bartfai

The presence stream (Kinesis) and RTC publisher has been deployed to AWS sandbox. RisingWave has also been deployed to AWS, running in ECS Fargate.

I have a doc with the full architecture here


**Presence + hooks service architecture**

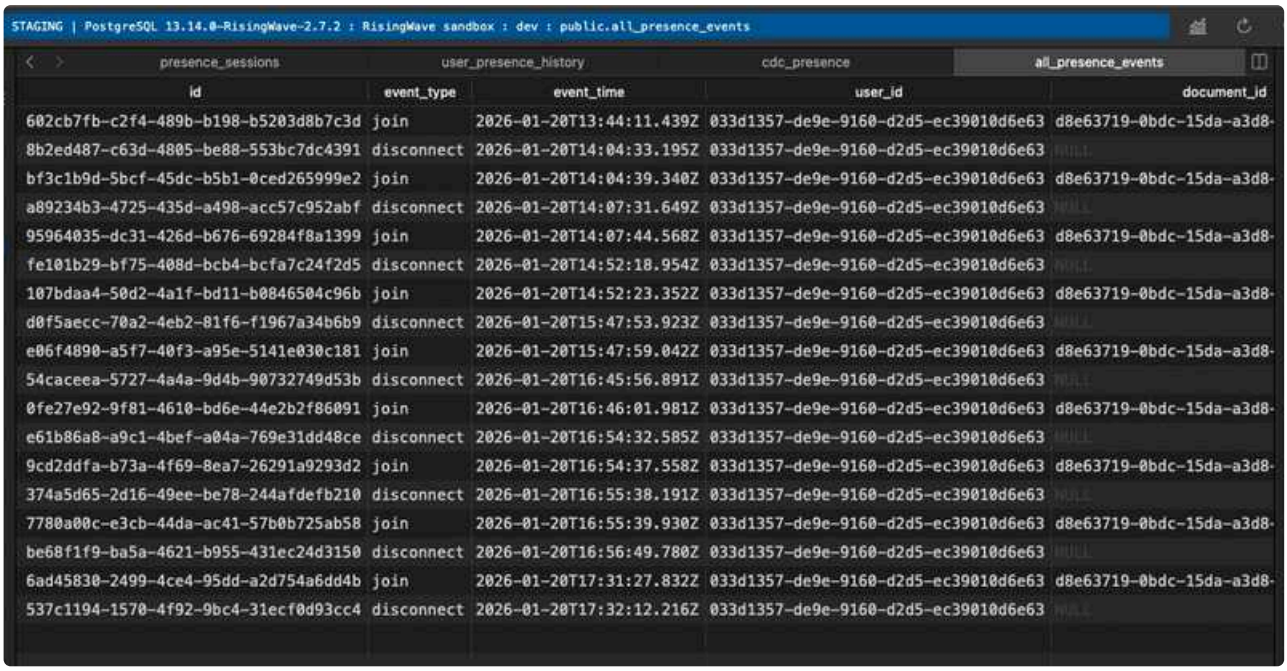
The presence system is a realtime system designed to ingest and enrich presence events based on...

Last Edited **1/20/2026**

RisingWave was quite hard to get working. There are still issues with its handling of S3 credentials, which I'll debug later (it can use ephemeral storage on single nodes), its health check was using the psql command - which didn't exist on the bare ECS container - so I had to use a different way, service wouldn't start due to multiple thread panics, postgres connection was hanging even though network settings were fine, and the list goes on. But right now it's in a good state (except S3 creds).

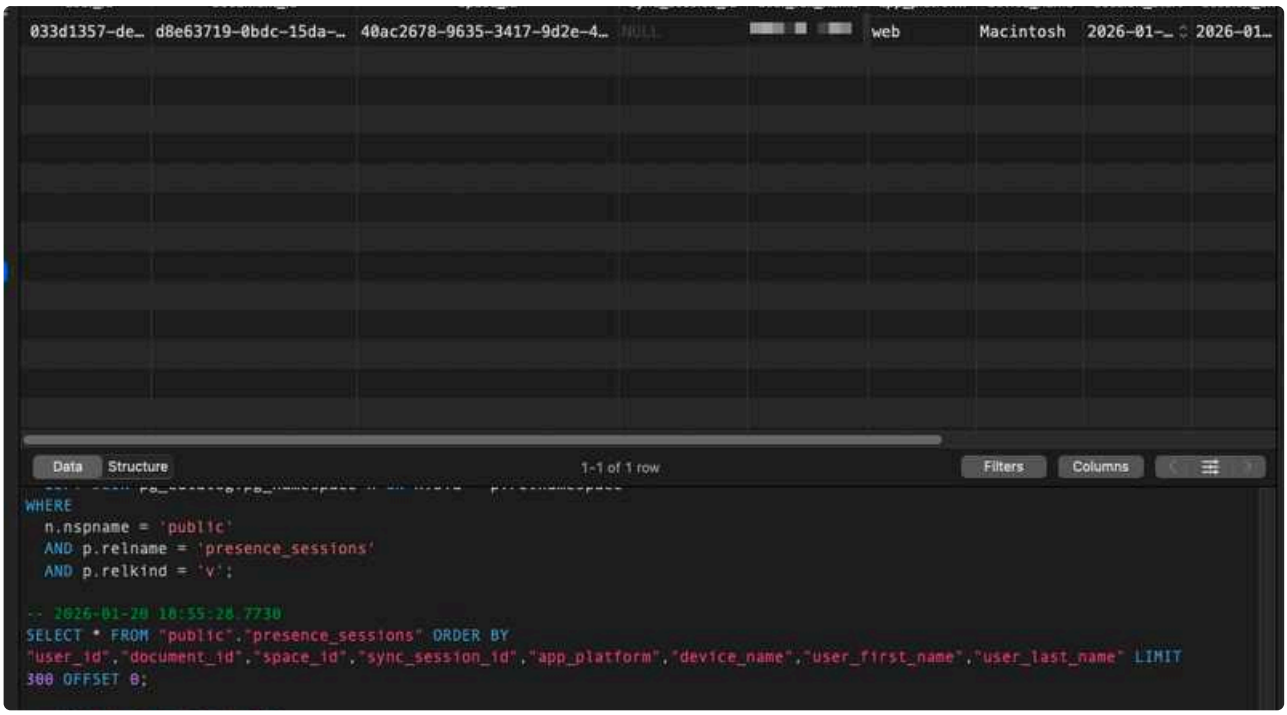
While deploying I did some calculations for capacity planning - and looking at DataDog, sync events, and how how presence would be approx 10x, I opted to drop the presence events table as it would be 80-120GB a day. Debezium connector was also not working well enough - because of our compression and format which RW can't read well. So I simplified the architecture to stream presence events straight into Kinesis from RTC, and join with the blocks WAL instead of the Debezium kinesis stream. Code in place for join, but not deployed yet, I'll need to do calculations there as well. If the WAL method does not work, RW can join via SQL commands - so instead of a push, it pulls and enriches presence data.

Data is flowing in from Kinesis and can be queried with SQL:


STAGING | PostgreSQL 13.14.0-RisingWave-2.7.2 : RisingWave sandbox : dev : public.all_presence_events

presence_sessions | user_presence_history | cdc_presence | all_presence_events

| id | event_type | event_time | user_id | document_id |
|---|---|---|---|---|
| 602cb7fb-c2f4-489b-b198-b5203d8b7c3d | join | 2026-01-20T13:44:11.439Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | d8e63719-0bdc-15da-a3d8- |
| 8b2ed487-c63d-4805-be88-553bc7dc4391 | disconnect | 2026-01-20T14:04:33.195Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | NULL |
| bf3c1b9d-5bcf-45dc-b5b1-0ced265999e2 | join | 2026-01-20T14:04:39.340Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | d8e63719-0bdc-15da-a3d8- |
| a89234b3-4725-435d-a498-acc57c952abf | disconnect | 2026-01-20T14:07:31.649Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | NULL |
| 95964035-dc31-426d-b676-69284f8a1399 | join | 2026-01-20T14:07:44.568Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | d8e63719-0bdc-15da-a3d8- |
| fe101b29-bf75-408d-bcb4-bcfa7c24f2d5 | disconnect | 2026-01-20T14:52:18.954Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | NULL |
| 107bdaa4-50d2-4a1f-bd11-b0846504c96b | join | 2026-01-20T14:52:23.352Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | d8e63719-0bdc-15da-a3d8- |
| d0f5aecc-70a2-4eb2-81f6-f1967a34b6b9 | disconnect | 2026-01-20T15:47:53.923Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | NULL |
| e06f4890-a5f7-40f3-a95e-5141e030c181 | join | 2026-01-20T15:47:59.042Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | d8e63719-0bdc-15da-a3d8- |
| 54caceea-5727-4a4a-9d4b-90732749d53b | disconnect | 2026-01-20T16:45:56.891Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | NULL |
| 0fe27e92-9f81-4610-bd6e-44e2b2f86091 | join | 2026-01-20T16:46:01.981Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | d8e63719-0bdc-15da-a3d8- |
| e61b86a8-a9c1-4bef-a04a-769e31dd48ce | disconnect | 2026-01-20T16:54:32.585Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | NULL |
| 9cd2ddfa-b73a-4f69-8ea7-26291a9293d2 | join | 2026-01-20T16:54:37.558Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | d8e63719-0bdc-15da-a3d8- |
| 374a5d65-2d16-49ee-be78-244afdefb210 | disconnect | 2026-01-20T16:55:38.191Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | NULL |
| 7780a00c-e3cb-44da-ac41-57b0b725ab58 | join | 2026-01-20T16:55:39.930Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | d8e63719-0bdc-15da-a3d8- |
| be68f1f9-ba5a-4621-b955-431ec24d3150 | disconnect | 2026-01-20T16:56:49.780Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | NULL |
| 6ad45830-2499-4ce4-95dd-a2d754a6dd4b | join | 2026-01-20T17:31:27.832Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | d8e63719-0bdc-15da-a3d8- |
| 537c1194-1570-4f92-9bc4-31ecf0d93cc4 | disconnect | 2026-01-20T17:32:12.216Z | 033d1357-de9e-9160-d2d5-ec39010d6e63 | NULL |




Right now events are not enriched, but do have doc IDs. So we can answer a question like "*How many users were active in the last hour by 5-minute intervals?*" or "*Which documents have the most collaborative activity?*" based on the presence events alone.

It gets translated by an agent to plain SQL:

```sql
SELECT
    document_id,
    COUNT(DISTINCT user_id) as unique_users,
    COUNT(*) as total_events,
    MIN(REPLACE(REPLACE(event_time, 'T', ' '), 'Z', '')::timestamp) as
first_activity,
    MAX(REPLACE(REPLACE(event_time, 'T', ' '), 'Z', '')::timestamp) as
last_activity
FROM all_presence_events
WHERE document_id IS NOT NULL
GROUP BY document_id
ORDER BY unique_users DESC, total_events DESC
LIMIT 10;
```

But this table is actually a stream of almost realtime events.

To connect to RisingWave you can use the 1P credentials.

### Code

- Presence PR: https://github.com/lukilabs/Luki.Serverless/pull/4205
  - This includes the presence infra, along with tools for the agent to query it
- Hooks service (commit only for now): https://github.com/lukilabs/Luki.Serverless/commit/210d5874cfc7d9111b90cd02fdea833ac0bd8ec6

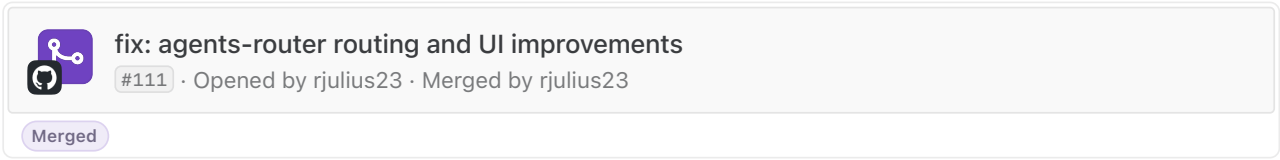
### Next up

S3 persistence, postgres WAL into RW, and linking up block changes.

Gyula Halmos

## Craft Agents - 658 stars 😎

- I am now in bugfixing mode:




- There were some bugs that i could not reproduce, like the permission issue on windows when installed under Localdata (for me in Parallels it works fine, will need to check on a laptop too) - it looks a one off issue but need to understand
- Also want to spend some time thinking about error handling and trace collection from production to better understand faulty use cases - like when one session goes numb (fiel corruption happens time to time - window state is also pretty sensitive)
- Also some infra related issues
  - Redeployed the agent router old tui for github workflow runs is available from under /cli
  - Also redeployed the session-viewer Pages it was broken in the evening
- Will need to think about some of the feature requests from Open Source also monitoring the Craft Agents Beta Slack
- Helping colleagues and collecting feedbacks

## Next steps

- Jan 22., Thu i will only be in the office half-a-day, took it via Bamboo HR (waiting for approval Balint Orosz )
- Next week i would like to prepare a presentation on how Craft Agents can be used by knowledge workers such as PMs but not from the SW dev but the real eastate domain
- The Mac build was not working at night (timed out... during signing) so i reverted the marketing page links to the previous version, will work on it in the morning if the during the night it wont finish

Zsolt Varnai

Jan 20., Tue

**Write Tools**

- **Block Operations**
  - **Delete blocks API**
    - Created a client side delete blocks API
    - Added check to disallow block deletions in the write tool
    - The only downside is that we need to do this in the pretool hook, so we have to parse the HTML on the server side as well
    - Potentially we could have a socket API for the check if we want the algorithm to fully stay client side
  - **Move blocks API**
    - Created a dedicated API, which is able to move blocks
    - By default the agent is trying to update the HTML to move a block, so we have to improve on the system prompt and disallow creating duplicate blocks with the same ID + deletion
- **HTML parsing and diffing**
  - Tried a bunch of different parser and diffing libs
  - A lot of them only parse and dont do diff
  - For those I was using vibe coded diff algo on the parsed tree nodes to test them
  - HTML5 compliance is a factor or we restrict the agent to not use it
  - We could do the same with XHTML compliance, so XML parsing libs can also be used
  - **Conclusions**
    - The C/Rust APIs were very fast
    - Swift wrappers around these have some overhead, but in exchange give convenient API and still in the same magnitude as the native parser
    - In order to choose from these we should first narrow the scope by deciding on HTML/XHTML/HTML5 compliance and if we are willing to use custom diff

| Library | Language | Total | Patches | Diff Algo | Granularity | Notes | Pros / Cons |
|---|---|---|---|---|---|---|---|
| libxml2 | C | 15.9 ms | 2351 | Custom | Node | system library | Fastest, no bundling needed, XML-focused |
| mt-dom | Rust | 24.9 ms | 2351 | Built-in | Node | html5ever parser | Fast, built-in diff, HTML5 compliant |
| libxmldiff | C++ | 34.5 ms | 3301 | Built-in | Node | libxml2 parser | Built-in diff, XML-focused, GPL licensed, unmaintained |
| Fuzi | Swift | 50.3 ms | 2351 | Custom | Node | libxml2 wrapper | Swift API, good speed, Custom diff required, XML-focused |
| gumbo-parser | C | 65.9 ms | 2351 | Custom | Node | | HTML5 compliant, Google-backed · ✕ Custom diff required, unmaintained |
| SwiftGumbo | Swift | 121.0 ms | 2351 | Custom | Node | gumbo-parser wrapper | HTML5 compliant, Swift API, Custom diff required |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| diffDOM | JavaScript | 166.4 ms | 51 | Built-in | Subtree | via JavaScriptCore | Built-in diff, JSCore overhead, coarse granularity |
| SwiftSoup | Swift | 807.9 ms | 2351 | Custom | Node | pure Swift | Pure Swift, no dependencies, Very slow, custom diff required |

**Support**

- Helped setting up socket server and tool calling protocol in craft-agents to interact with the native app and use its available tools

Jan 21., Wed

- Continue with write tools proto

↑ Daily Notes - 2026 January

## Jan 20., Tue

Agents Team
--- · Christoph Locher · I worked on using Craft Agent to simulate the planning of document changes. · First, I chan...

↑ Jan 20., Tue

## Agents Team

## Christoph Locher

I worked on using Craft Agent to simulate the planning of document changes.

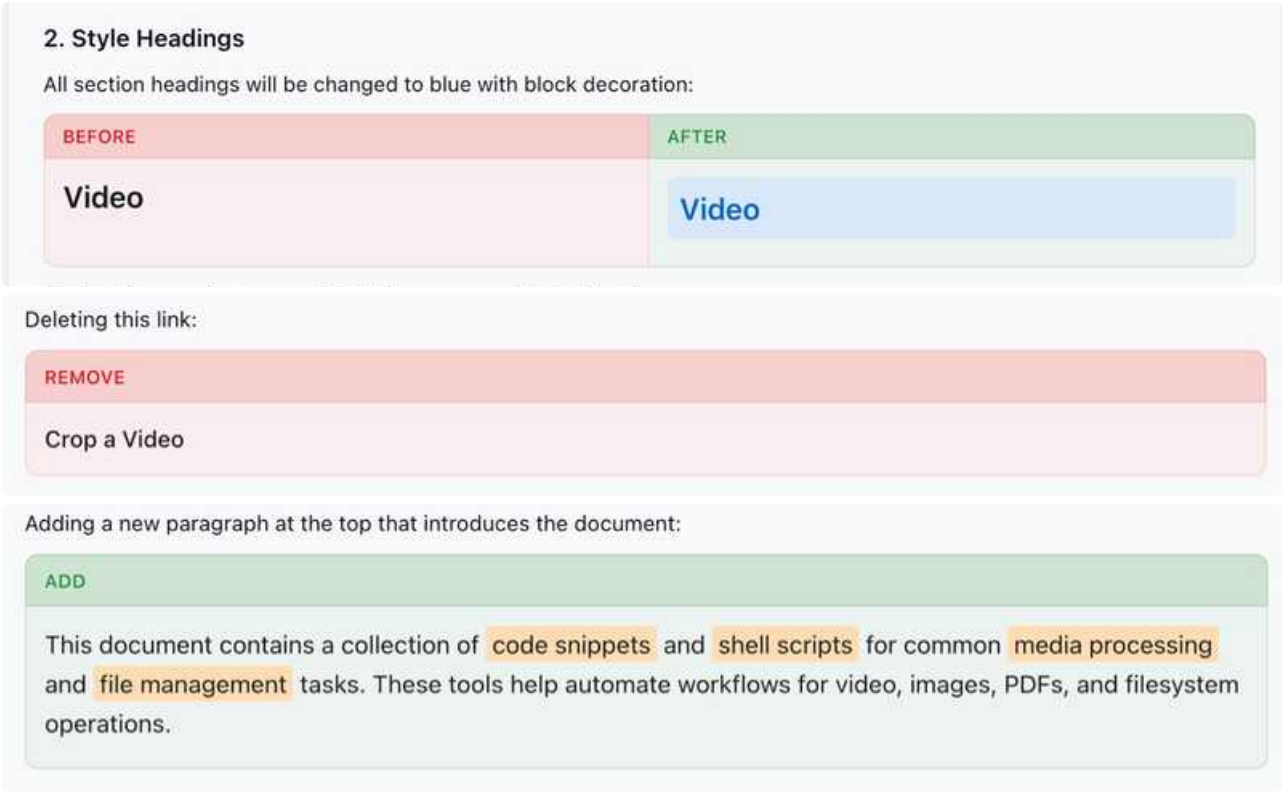
First, I changed the system prompt to one based on my previous prototype, to explain it how to read and understand the Craft HTML format.

Then I added a post-tool call hook for the write tool to run the HTML validation after every tool call. I also made the validation stricter to not just check allowed properties, but also complain when there are additional classes or CSS was changed.

Then I focused on the plan format, going through a ton of iterations on the `createSubmitPlanTool` to get it to give the right level of detail in its plan, group changes but also use examples, avoid technical terms and use Craft terminology instead of describing HTML when talking to the user.

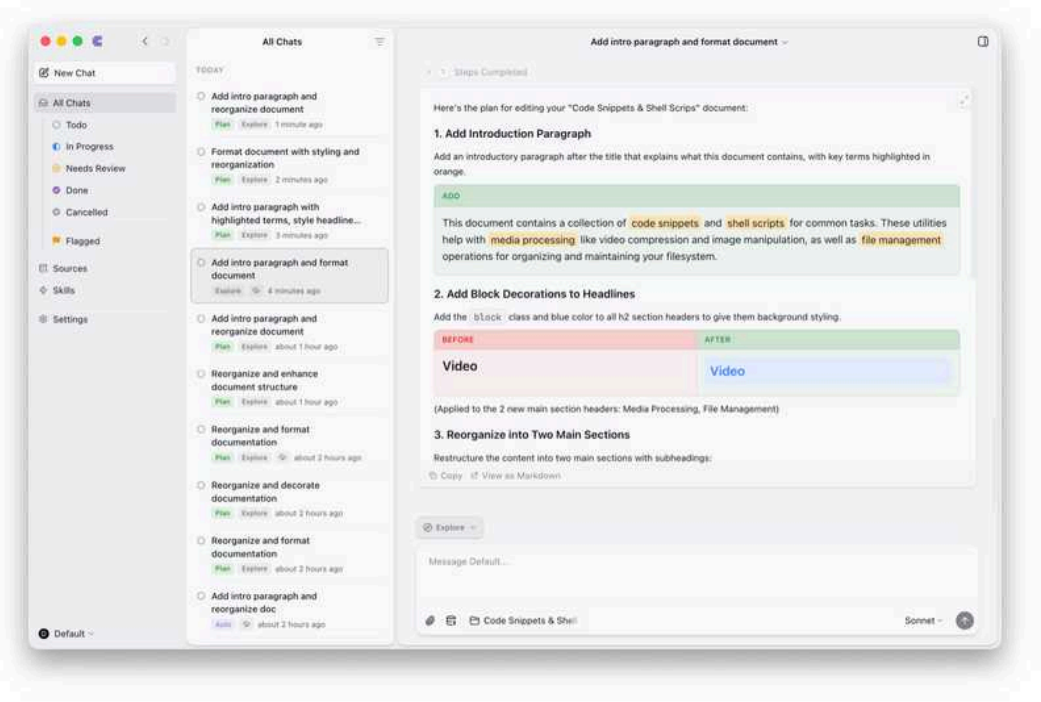
I also created a variant of the Code Block, which is used whenever there is an HTML code block using diffing format – this is quite nice as it falls back nicely but visually represents the Craft blocks when it is known.

I took the CSS styling from my prototype and extended it to support the color format we discussed, and tweaked the HTML diff preview to render all HTML with this format, to visually show the changes. Some examples:



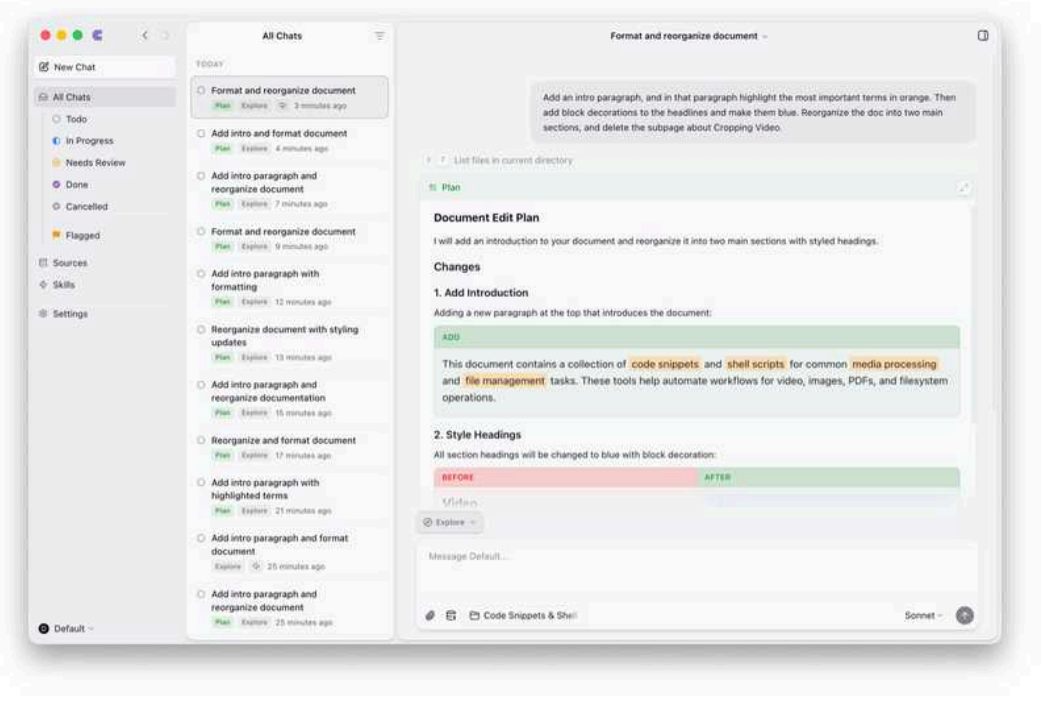

Here are some of my conversations that were created with various states of the prompt. Of course in the online view they show the default code block diff:

Craft Share Page




In the beginning it used this format well for smaller changes/plans but as soon as the plan got more complex it suddenly ignored the formatting rules and just created a technical plan. Over a lot of iterations I edited the tool description, and in the end managed to get it to adhere to it even for more complex requests:

Craft Share Page




I also prompted the plan tool to use ASCII art trees to visualize structural changes, but it uses it very rarely it not that successfully.

Craft Share Page


Probably an example will help here, perhaps even with diffing syntax.

## Balint Bartfai

Presence service integration into Serverless.

Today I looked at how the full infrastrucutre would look like in AWS, and built out the additonal MCP tools for presence. New tools:

- content changes: "what did user X write in the past 15 minutes?"
- user sessions: "what was user X's last editing session?"
- active collaborators: "who's editing doc X"

In addition to these standard queries, agents can write custom SQL to query presence events via a generic `presence_query` tool - filtering as they wish. This allows agents to answer more advanced questions. An example:

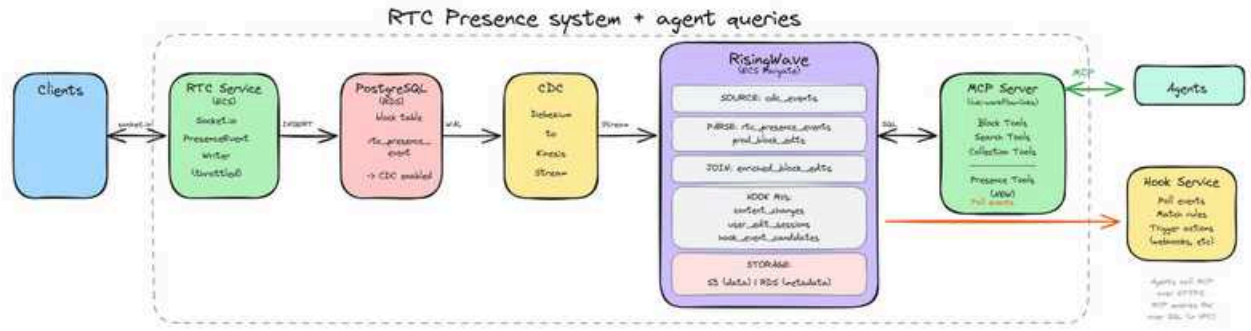
```
SELECT
    document_id,
    MAX(edit_time) as last_edit,
    COUNT(*) as historical_edits,
    NOW() - MAX(edit_time) as days_stale
FROM content_changes
GROUP BY document_id
HAVING MAX(edit_time) < NOW() - INTERVAL '14 days'

```

```
    AND COUNT(*) > 10
ORDER BY historical_edits DESC
```

This query answers "which documents havent been touched in a while but were active before". All queries are standard SQL and work with the postgres wire protocol.

We can add more predefined tools later if needed.

## AWS infrastructure




When users edit documents, RTC service captures presence events (cursor movements, typing, document opens) and writes them to RDS alongside regular block changes. Debezium captures these changes via CDC, streams them to Kinesis, which RisingWave consumes to maintain real-time materialized views (like `content_changes`, `user_edit_sessions`, `active_collaborators`, `hook_event_candidates`). Agents use MCP, which exposes presence query tools that execute SQL against RisingWave. Hook Service polls `hook_event_candidates` to trigger automations when rules match, while agents can also proactively query presence data to answer questions.

RisingWave joins 2 CDC streams in real time and exposes the materialized view - the actual join is simple:

```
CREATE MATERIALIZED VIEW content_changes_with_session AS
SELECT
    c.*,
    s.session_start,
    s.session_duration_so_far,
    s.blocks_edited_in_session
FROM content_changes c
LEFT JOIN user_edit_sessions s
    ON c.user_id = s.user_id

```

```
    AND c.document_id = s.document_id
    AND c.edit_time BETWEEN s.session_start AND s.session_end;
```

### New infra needed

We can use the ECS cluster, RDS, kinesis, debezium components we already have. The new infra needed is

- RisingWave ECS Fargate service. I would suggest a single node to start.
- New S3 bucket for state storage (RW uses that as the storage backend). Based on my initial calcualtions this would be around ~100GB - depending on how long we keep data around.
- Small table for risingwave metadata - these are the materialized view definitions.
- Set up cdc stream for presence + `rtc_presence_events` table
- service discovery / IAM roles / cloudwatch log group - supporting elements.

Cloudformation templates, SQL inits ready. I will start deploying this to sandbox.

Presence system on branch in Serverless: https://github.com/lukilabs/Luki.Serverless/compare/balintb/presence-stream

Zsolt Varnai

Jan 19., Mon

**Write Tools**

- Created HTML file syncing prototype
  - **Operation**
    - The app reports the edited documentId to the service in the generate call
    - Whenever the HTML file is needed, the service asks for an HTML export of the document via the websocket to create the file in the session directory
    - After the HTML file has changed the service can call another websocket action where the client converts the incremental changes in the HTML to realm actions and applies the document changes.
  - **Hooks**
    - First tried to solve the HTML creation on the fly with pretool hooks, however it turned out neither this hook nor the permission hooks arrives if the file does not exist at all
    - So the first learning is that we need to precreate the file in order to support syncing with hooks.
    - This is easy for a single file, but if we want to support reading arbitrary files by the agent with the read tool, then we probably can just create a bunch of empty files based on the document IDs, so the hooks start working
    - Then in the pretool hook before the read action we will update the empty file to the full version
  - **Write failures**
    - The HTML is refreshed from realm before Read or Write tool execution to make sure, that the document is up to date
    - The write path didn't work at first, because as soon as the file is refreshed claude throws an error that the file has been modified since last read
    - Fixed this by only overwriting the file if there is an actual diff, so it keeps the metadata unchanged
    - **Note:** This feature can be useful to push the agent to work on the latest file, minimizing big conflicts, although its not an absolute must. Our realm conflict resolution strategies would also allow to apply the HTML's incremental changes on realm even if the HTML didn't contain the latest state.
      - It can also be detrimental, because on a frequently changing file the agent would just go in an infinite loop of rereading it and failing to write it
      - Also atomicity is not fully guaranteed anyway, anything can still change while the write operation is in progress and we reach the realm transaction. So at the end of the day its the realm conflict resolution, that makes the update robust.
  - **Permissions**
    - The read/write permissions still didn't work reliably when trying to differentiate the session directory from the rest (probably due to the claude bug I found earlier), so I added a custom path check and permission result override to the pretool hook instead, which worked immediately much better, guaranteeing that the tools cannot read/write outside the session dir.
  - **Block operations**

- Did some tests updating and inserting blocks, which of course worked in the HTML
- The main question is solving block moves across different levels of the document, so I started to do some research on the various options.
- The main problem is that the insert and delete of the same block has to be atomic in order to avoid data loss or failed transaction.
- First I was leaning towards representing the document as a single HTML, so moving a block to a subpage can happen in the same file.
- However for large files even writing to different parts of the file won't be a single, atomic write, so this won't fully solve the problem.
- I explored a few ideas in theory, that could solve this, but I want to continue prototyping these to find the most reliable approach.
    - We can batch multiple writes into a single transaction and only save at the end, although we don't want to slow down the saves too much, we need the feedbacks on the UI while the agent is calling the actions.
    - The agent could potentially mark the writes to be batched when necessary or we can experiment with a smart solution, that automatically batches them based on heuristics.

**Support**

- Sync PR discussion and review

Jan 20., Tue

- Continue with write tools proto

↑ Daily Notes - 2026 January

## Jan 19., Mon

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · Viktor's findings from weekend t...

↑ Jan 19., Mon

## Agents

### Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Viktor's findings from weekend testing (v0.2.15)** - Multiple issues flagged: need to login every app restart, "Open in New Window" requires re-login, Help > Documentation links are outdated, install instructions show wrong curl command. These should be prioritized for fixes.
- **Balint's CDC stream + RisingWave prototype working** - Successfully connected to live CDC stream with real-time materialized views joining presence events with block edits. Good progress to share.
- **Christoph's HTML format decision** - The semantic HTML approach with CSS classes works well for agent editing. Team agreed on this direction. Remaining issue: agent invents new classes for "impossible" requests - need better guardrails.
- **Subpage representation still an open question** - Christoph uses separate files, Gyula uses inline nested HTML. Need to align on approach, especially regarding existing Read/Search tools.
- **Gyula's questions about simulated file system approach** - If going with HTML files representing documents, raises questions about grep/sed enablement and sandboxing. Team should discuss tradeoffs.
- **Gyula's concern about HTML parser data loss** - Vibe-coded parser sometimes thinks HTML has zero blocks, which could lead to data loss. May need confirmation checks ("are you sure you want to delete 158 blocks?").

Findings from Viktor over the weekend
ℹ️ Using 0.2.15 · Need to login everytime I restart the app (video below) · When using “Open in New Window” need...

• • •

### Balint Bartfai

Connected to the live CDC stream today.

I wrote a sample agent as well, which can query changes via RisingWave.

Have a branch on Serverless where RTC is expanded by having an append-only presence table, so all of this works offline. CDC will sync it up when the client comes back online. Branch https://github.com/lukilabs/Luki.Serverless/tree/balintb/presence-stream

Wrote test harness to take CDC from live stream and pass them into RW via localstack Kinesis. See repo here

lukilabs / rw-proto
github.com/lukilabs/rw-proto
lukilabs TypeScript Stars 0

### Event types

These are the event types I'm usinbg for testing

- `join` - user opened doc
- `update` - cursor moved, blocks changed
- `leave` - user closed doc

- `disconnect` - socket disconn

## RisingWave views

- `rtc_presence_events` - raw events from CDC
- `rtc_presence_summary` - aggregated by user/document
- `presence_with_edits` - **JOINed with block edits on sync_session_id** (enrichment)

The `sync_session_id` is the correlation key:

- When a user syncs, they get a session ID
- Presence events include this session ID
- Block edits (via CDC) also include `_sync_session_id`
- RisingWave joins them in real-time as events flow in

## MV Definitions

**rtc_presence_events** - Parse raw presence CDC events:

```
CREATE MATERIALIZED VIEW rtc_presence_events AS
SELECT
  payload->>'op' AS operation,
  (payload->'after'->>'id')::VARCHAR AS event_id,
  (payload->'after'->>'event_type')::VARCHAR AS event_type,
  (payload->'after'->>'event_time')::TIMESTAMPTZ AS event_time,
  (payload->'after'->>'user_id')::VARCHAR AS user_id,
  (payload->'after'->>'space_id')::VARCHAR AS space_id,
  (payload->'after'->>'device_id')::VARCHAR AS device_id,
  (payload->'after'->>'rtc_id')::VARCHAR AS rtc_id,
  (payload->'after'->>'sync_session_id')::VARCHAR AS sync_session_id,
  (payload->'after'->>'document_id')::VARCHAR AS document_id,
  payload->'after'->'open_pages' AS open_pages,
  payload->'after'->'editing_blocks' AS editing_blocks,
  (payload->'after'->>'user_first_name')::VARCHAR AS user_first_name,
  (payload->'after'->>'user_last_name')::VARCHAR AS user_last_name,
  (payload->'after'->>'app_platform')::VARCHAR AS app_platform,
  (payload->'source'->>'ts_ms')::BIGINT AS cdc_time_ms
FROM cdc_events
WHERE payload IS NOT NULL
  AND payload->'source'->>'table' = 'rtc_presence_event'
  AND payload->'after' IS NOT NULL;
```

Aggregate by user/document

```sql
CREATE MATERIALIZED VIEW rtc_presence_summary AS
SELECT
  user_id,
  document_id,
  space_id,
  COUNT(*) FILTER (WHERE event_type = 'join') AS join_count,
  COUNT(*) FILTER (WHERE event_type = 'leave') AS leave_count,
  COUNT(*) FILTER (WHERE event_type = 'update') AS update_count,
  COUNT(*) FILTER (WHERE event_type = 'disconnect') AS disconnect_count,
  COUNT(*) AS total_events,
  MAX(event_time) AS last_activity,
  MIN(event_time) AS first_activity
FROM rtc_presence_events
WHERE document_id IS NOT NULL
GROUP BY user_id, document_id, space_id;
```

Enriched presence with edits

```sql
CREATE MATERIALIZED VIEW presence_with_edits AS
SELECT
  p.event_id,
  p.event_type AS presence_event,
  p.event_time AS presence_time,
  p.user_id,
  p.document_id,
  p.space_id,
  p.sync_session_id,
  p.rtc_id,
  p.device_id,
  p.user_first_name,
  p.user_last_name,
  p.app_platform,
  b.block_id,
  b.block_type,
  b.operation AS block_operation,
  b.cdc_time AS block_edit_time
FROM rtc_presence_events p
LEFT JOIN prod_block_edits b
  ON p.sync_session_id = b.sync_session_id
  AND p.document_id = b.document_id
WHERE p.sync_session_id IS NOT NULL;
```

This is **streaming** - as new presence events and block edits arrive, RW automatically updates the materialized view

Agent can query this like

```sql
-- recent presnce
SELECT * FROM rtc_presence_events ORDER BY event_time DESC LIMIT 10;

-- who was active on doc
SELECT * FROM rtc_presence_summary WHERE document_id = 'xxx';

-- what they edited
SELECT * FROM presence_with_edits WHERE sync_session_id = 'yyy';

-- activity
SELECT event_type, COUNT(*) FROM rtc_presence_events GROUP BY event_type;
```

## Running

```
# run cdc forwarder
cd rw-proto && bun run forward-cdc

# open doc in app

# query risingwave
psql -h localhost -p 4567 -d dev -U root
> SELECT * FROM rtc_presence_events;
```

Agent and full test harness at https://github.com/lukilabs/rw-proto

### Christoph Locher

At the Standup we discussed our various approaches to a format that the agent can edit. We found that the conversion between the Craft format and the HTML format was very fragile, because of the use of Tailwind-like styling classes (e.g. a block with block decoration get the classes `bg-red rounded-lg`). This gave the agent a ton of freedom to add styling that can't be represented it Craft.

But we do like the semantic quality of HTML, as the agent is much better able to understand it as a document (from its training data) than a JSON structure it never saw before.

> So we decided to keep the HTML direction, but instead of visual classes go to a more semantic route. This means instead of the Tailwind approach to CSS, we follow an approach that more closely matches the ca. 2011 OOCSS ideas: reusable utility classes for styling, but with semantic names (instead of Tailwind-style atomic classes or old-school web development component-based classes).

In practice this means a block with block decoration now gets a `block` class, which is defined as background color and border radius in the CSS file.

So I rewrote the AST to HTML converter to use these classes instead and created a mapping from all block types and properties to HTML element types and CSS classes.

```
/**
 * AST to HTML Converter
 * =====================
 *
 * Transforms Craft AST into semantic HTML. The conversion is lossless - all AST
 * data is preserved in HTML structure and CSS classes.
 *
 * ## Document Structure
 *
 * ```html
 * <article id="..." class="craft-page [page-classes]">
 *   <header><h1>Page Title</h1></header>
 *   <main><!-- child blocks --></main>
 * </article>
 * ```
 *
 * ## HTML Tag Mapping
 *
 * | AST Type / TextLevel        | HTML Tag                |
 * |-----------------------------|-------------------------|
 * | TextBlock, textLevel: h1    | <h1>                    |
 * | TextBlock, textLevel: h2    | <h2>                    |
 * | TextBlock, textLevel: h3    | <h3>                    |
 * | TextBlock, textLevel: h4    | <h4>                    |
 * | TextBlock, textLevel: body  | <p>                     |
 * | TextBlock, textLevel: caption | <p class="caption">   |
 * | TextBlock, textLevel: page  | <a class="page">        |
 * | TextBlock, textLevel: card  | <a class="card">        |
 * | TextBlock with listStyle    | <li> inside <ul>/<ol>   |
 *
```

```
 * | SeparatorBlock             | <hr>                    |
 * | ImageBlock                 | <figure><img></figure>  |
 * | FileBlock                  | <aside>                 |
 * | LinkBlock                  | <a> (see Link Block Structure) |
 * | CodeBlock                  | <pre><code>             |
 *
 * ## Class Mapping
 *
 * ### Font Family
 * | AST font   | Class         |
 * |------------|---------------|
 * | sans       | font-sans     |
 * | serif      | font-serif    |
 * | mono       | font-mono     |
 * | rounded    | font-rounded  |
 *
 * ### Alignment
 * | AST alignment | Class          |
 * |---------------|----------------|
 * | left          | align-left     |
 * | center        | align-center   |
 * | right         | align-right    |
 * | justified     | align-justify  |
 *
 * ### Indentation
 * | AST indentation | Class     |
 * |-----------------|-----------|
 * | 1               | indent-1  |
 * | 2               | indent-2  |
 * | 3               | indent-3  |
 * | 4               | indent-4  |
 * | 5               | indent-5  |
 *
 * ### Text Color (supports dark mode)
 * | AST color         | Class                                |
 * |-------------------|--------------------------------------|
 * | "#RRGGBB"         | color-[#RRGGBB]                      |
 * | "#light #dark"    | color-[#light] dark:color-[#dark]    |
 *
 * ### Decorations
 * | AST decoration              | Class    |
 * |-----------------------------|----------|
 * | hasBlockDecoration: true    | block    |
 * | hasFocusDecoration: true    | focus    |
 *
 * ### Separator Level
 * | AST separatorLevel | Class                 |
 * |--------------------|-----------------------|
 * | extraLight         | separator-extralight  |
 * | light              | separator-light       |
 * | regular            | (none)                |
 * | bold               | separator-bold        |
 *
```

```
 * | pageBreak          | separator-pagebreak  |
 *
 * ### Separator Style
 * | AST separatorStyle | Class              |
 * |--------------------|--------------------|
 * | default            | separator-default  |
 * | doodle             | separator-doodle   |
 * | washi              | separator-washi    |
 *
 * ### Image Size
 * | AST size | Class         |
 * |----------|---------------|
 * | small    | image-small   |
 * | medium   | image-medium  |
 * | large    | image-large   |
 *
 * ### Image Fit
 * | AST fit | Class       |
 * |---------|-------------|
 * | fill    | image-fill  |
 * | fit     | image-fit   |
 *
 * ## Page-Level Classes
 *
 * These classes can be set on the <article> element:
 *
 * | Class                      | Purpose                                        |
 * |----------------------------|------------------------------------------------|
 * | wide                       | Wide page layout                               |
 * | font-*                     | Default font for all text elements             |
 * | separator-*                | Default style for all <hr> elements            |
 * | color-[#...]               | Default text color                             |
 * | page-color-[#...]          | Content column background (center column)      |
 *
 * ### Background Classes (full page, extends to edges)
 *
 * | Class                   | Purpose                                    |
 * |-------------------------|--------------------------------------------|
 * | bg-none                 | No background                              |
 * | bg-color                | Solid color background                     |
 * | bg-gradient             | Gradient background                        |
 * | bg-image                | Image background                           |
 *
 * Background color values (all support dark mode with dark: prefix):
 * | Class                      | Purpose                                  |
 * |----------------------------|------------------------------------------|
 * | bg-[#...]                  | Background color (for bg-color)          |
 * | dark:bg-[#...]             | Dark mode background color               |
 * | bg-from-[#...]             | Gradient start color                     |
 * | dark:bg-from-[#...]        | Dark mode gradient start color           |
 * | bg-to-[#...]               | Gradient end color                       |
 * | dark:bg-to-[#...]          | Dark mode gradient end color             |
```

```
 * | page-color-[#...]          | Page/content column background         |
 * | dark:page-color-[#...]     | Dark mode page background              |
 *
 * Gradient direction:
 * | Class                   | Purpose                                   |
 * |-------------------------|-------------------------------------------|
 * | bg-gradient-vertical    | Top to bottom (default)                   |
 * | bg-gradient-horizontal  | Left to right                             |
 * | bg-gradient-radial      | Radial gradient                           |
 *
 * Gradient style:
 * | Class                   | Purpose                                   |
 * |-------------------------|-------------------------------------------|
 * | bg-gradient-immersive   | Full saturation gradient (default)        |
 * | bg-gradient-faded       | Subtle/faded gradient                     |
 *
 * ## Link Block Structure
 *
 * Link blocks render with title and optional description:
 *
 * ```html
 * <a id="..." href="..." class="..." data-link-layout="...">
 *   <span class="link-title">Title Text</span>
 *   <span class="link-description">Description text</span>
 * </a>
 * ```
 *
 * | AST Property | HTML Element                          |
 * |--------------|---------------------------------------|
 * | url          | href attribute                        |
 * | title        | <span class="link-title">             |
 * | description  | <span class="link-description">       |
 * | layout       | data-link-layout attribute            |
 *
 * ## Inline Formatting (runAttributes)
 *
 * | AST runAttribute    | HTML Element                                |
 * |---------------------|---------------------------------------------|
 * | isBold              | <strong>                                    |
 * | isItalic            | <em>                                        |
 * | isStrikethrough     | <s>                                         |
 * | highlightColor      | <mark class="highlight-[#...]">             |
 * | highlightColor (dark) | <mark class="highlight-[#...] dark:highlight-[#...]">
|
 * | url                 | <a href="...">                              |
 *
 * ## Data Attributes
 *
 * | Data Attribute        | Purpose                              |
 * |-----------------------|--------------------------------------|
 * | data-task             | Task state (done/undone/cancelled)   |
 * | data-file-layout      | File block layout                    |

```

```
189   * | data-link-layout     | Link block layout                 |
190   * | data-code-language   | Programming language              |
191   * | data-code-theme      | Code syntax theme                 |
192   * | data-has-children    | Block has children in separate file|
193   */
```

astToHtml.ts

Additionally, I create a CSS file with styling for all these classes, which helps the agent understand how they look visually, so when the user says “Add a background to all headlines” the agent knows the way to go is a block decoration.

styles.css
File src/prototypes/craft-html/styles.css

lukilabs / document-edit-plan-mode Branch main

This is injected into the generated HTML, so a document becomes a nicely readable HTML file:

complex-test.html
File src/prototypes/craft-html/data/complex-test.html

lukilabs / document-edit-plan-mode Branch main

An example document

To represent Subpages, I decided to create separate html files for each subpage, that the parent block refers to. I think this helps to understand how subpages are seen by the user better than having their content nested in the main html, but this is something to explore further:

complex-test-BF31FB71-BDFE-43C8-9890-445668C11BC4.html

A subpage from the example document

I also created round-trip tests, to make sure the conversion is really lossless:

converters.test.ts

And of course the validator still checks if an HTML document represents a valid Craft doc:

htmlValidator.ts

astValidator.ts

I turned this into a tool, so the agent can use it a bit more easily.

server.ts

I added the information about the mapping to CSS classes to the agent prompt so it understand which classes it can use (the Style tag in the HTML also helps with that) and with this, it was able to perform all the tasks I asked from it: restructure documents, insert blocks, apply styling, etc.

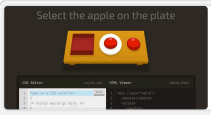
**So this HTML format works very well for our purposes.**

One remaining issue is that it is happy to invent new classes or change the CSS when asked for "impossible" things like increasing the space above headlines or placing content side-by-side. I asked it not to do that in the prompt, but so far that didn't have any effect. So we need to spend some more time to set effective guardrails for this.

• • •

Because all of this worked so well, but it was often painful to watch the agent do 20 edit operations one after the other for a small change, I decided to do a quick test doing something that goes against the atomic edit strategy we discussed in the standup, but is insanely effective:

I believe that CSS selectors are one of the best inventions of computer science – it is very hip to hate on CSS, but I think that's mostly because people are too lazy to understand selectors. It's basically like RegExp's – it seems hard to learn but super effective if you know the basic magic.




CSS Diner

A fun game to help you learn and practice CSS selectors.

https://flukeout.github.io/

Selectors are an amazing way to manipulate a document, extremely effective and LLMs a very good at composing them.

So I gave the agent a tool where it can pass in a selector, a class and wether to add and remove it. It parses the HTML into a Dom tree, applies `querySelectorAll` and adds or removes classes.

It's still very limited – just adding or removing a single class – forcing the agent to make relatively small edits at a time, but giving it the ability to operate on blocks strategically. It can now change multiple blocks in one edit, but the type of change is still semantically grouped. Selectors were invented to manipulate documents, and the agent is trying to manipulate documents – it's a nice match!

server.ts

File server.ts

lukilabs / document-edit-plan-mode Branch main

So now operations become super simple:

- It can use the semantic type of a block to access it
  - Make every subtitle blue: `h2`, `add`, `color-blue`
- It can easily manipulate multiple semantic types at once
  - Make all headlines blue: `h1, h2, h3`, `add`, `color-blue` (this usually took 30s in my example, with this tool it takes 2s)
- It can take current styling as an input
  - Make every block that has block decoration blue: `.block`, `add`, `color-blue`
  - Turn all pages into cards: `.page`, `add`, `.card`, then `.page.card`, `remove`, `.page`
- It can use the id
  - Make the selected block blue: `#{id}`, `add`, `color-blue`

    - Make theses three blocks blue: `#{id}, #{id}, #{id}`, `add`, `color-blue`
- It can do a bit more advanced logic
    - Add block decoration to the first block after a heading : `h3 + *`, `add`, `block`
    - Make every second block blue: `*:nth-child(even)`, `add`, `color-blue`

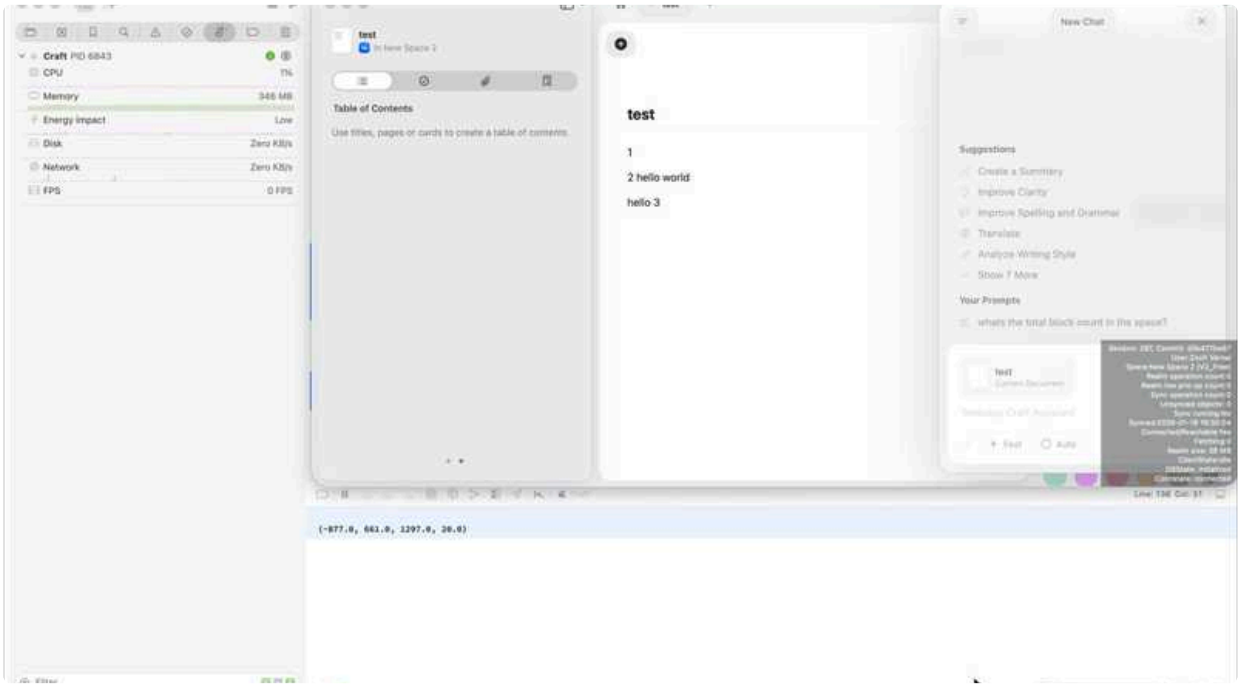
... and many more things.

So this is just an experiment, but I think it shows that treating HTML as HTML rather than just text can help us speed this up hugely, and if we design the tools write it's still one tool call per logic operation. This is still a filesystem read/write tool, basically a search and replace text tool, just with better selection than text range.

Jan 16., Fri

**Write Tools**

- **Vibe coded a proto update tool**, that takes an HTML input and updates blocks based on the HTML diff
- Also added the HTML converter to the get page content tool to have **end to end testable flow**




- Tried some other usecases, like restructuring the block hierarchy and it worked fine, **the agent could restructure the HTML properly**

- I had more issues with the vibe coded HTML parser not finding some divs/blocks, thinking the HTML has zero blocks, which could lead to data loss (hopefully undoable, but still)
  - Of course in the final version we would probably use a more reliable parser lib, but it **still highlights a potential issue with unexpected formatting or structure causing seemingly terrible  side effects**
  - Based on how big of a problem this will be in the final version we could potentially have **confirmation checks by the tool**, just like a human would need to do it (`are you sure you want to delete 158 blocks?`)
- I also used an approach where the agent has to pass the oldHTML and the newHTML to the tool as well
  - We can go with the conservative approach, like Claude CLI to reread the page if there were differences since the read operation
  - But the oldHTML has another use, we actually calculate the diff between the 2 and only apply the diff on the blocks that changed, this solves a million potential problems
    - We won't accidentally update something that we didn't want to, **updating all blocks blindly would be more error prone**, with this approach the tool only did a single style update on the block for example, keeping most of the blocks intact

        - Wont cause side affects in things, that is not part of the HTML (modified date, etc)
- Discussion with Christoph on where we are, what do we agree on, whats pending
    - **We both agree that the HTML works** with the predefined classes and the **agent is making proper changes on it**
    - **How we represent the subblock hierarchy is still a question**, eg Christoph used separate files for each subpages, which is convenient for the read/write tools, in my proto I used inline nested HTML elements, because for me it was more convenient, that a single HTML string can represent the entire structure
        - This leads to the question about existing tools, eg the READ / SEARCH tools. Would we ever want to return HTMLs in the tool result (eg search tool, calendar tool, etc) or we go with the extreme approach, where we completely simulate a file system for the agent where the html files represent the documents and the only possible way to read the content is with the built in Read tool on the HTML.
    - I also want to highlight, that in my proto I used the "default" tool approach, where I have tool parameters to pass the HTML, which are native built in capabilities of Claude (calling functions with structured input) and the editing worked fine
        - I'm not against doing the "**simulated file system with documents as HTML files**" approach, I just see it orthogonal to the decision about the HTML based representation and update tools based on HTML.
        - We could do both. I understand that Claude can run clever tools on the HTML files, eg grep, sed, etc so it unlocks new capabilities. And its better to work with large input as it doesn't require passing the full HTML as tool input. But it has a lot of overhead vs a single tool call.
            - **Would we seem it viable to have both options?** Eg simple tool calls for trivial update cases, but the agent can still use grep/sed and incremental writes on the HTML file for more complex editing?
- **If we go with the simulated file system, then I would like to continue with prototyping that, because that's where most of the complexities lie in my opinion**
    - With tool hooks we can most likely figure out if a Read or Write tool needs a document's HTML and **on the fly generate it, then sync back the changes to Realm**
    - If we wanted to reenable bash commands for **grep/sed**, that raises a lot of questions, we don't know what documents will be needed for the commands and generating the entire workspace as HTML might take too much time, we would need to test what works.
        - It also raises the sandboxing question again, because previously we said, that we won't need bash tool and it can be disabled

Jan 19., Mon

- Continue with write tools

Gyula Halmos

### Craft Agents - platform testings

- Did multiple fixes for Windows and Linux over the weekend and Friday
  - Multiple iteration on Windows install in Parallels desktop
  - I spinned up a ubuntu that is not in Parallels (not arm64, but x86_64)
- Fine tuned installation scripts for all platforms
- Fine tuned and tested automated builds for all platforms

### Next steps

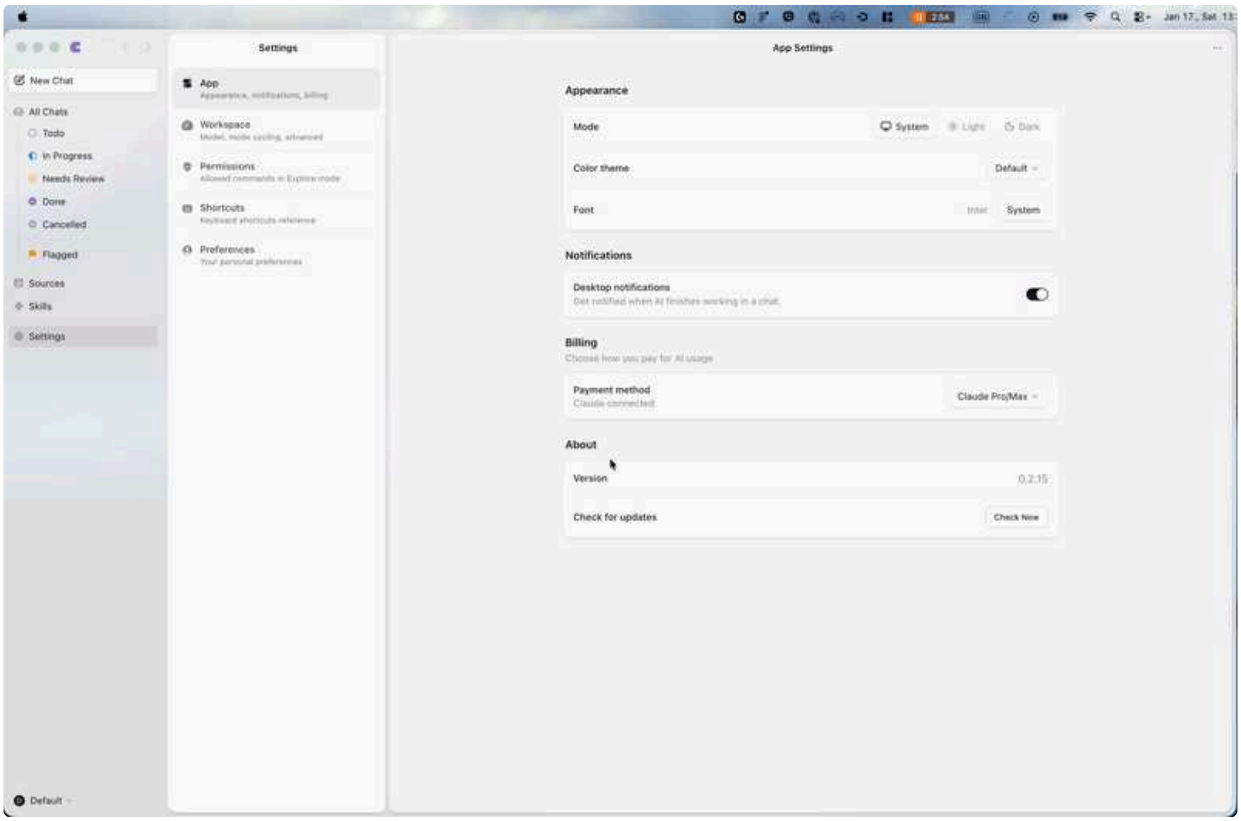
- Finalize repo sync and preparations for OSS release

↑ Jan 19., Mon · Agents

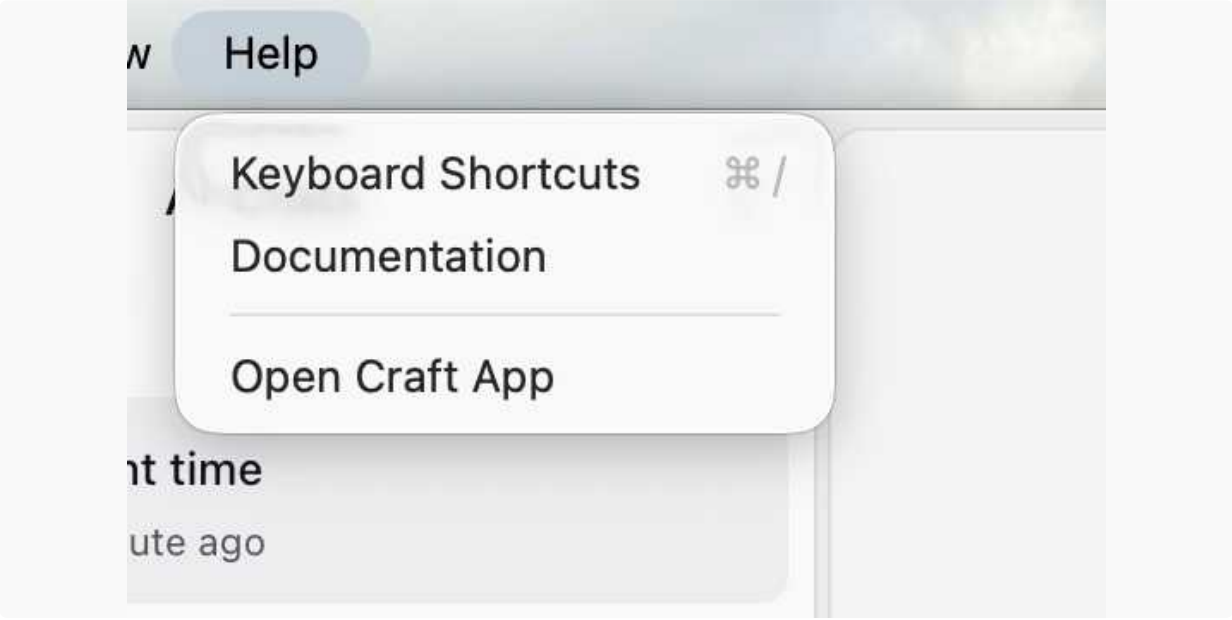
## Findings from Viktor over the weekend

ℹ️ Using 0.2.15

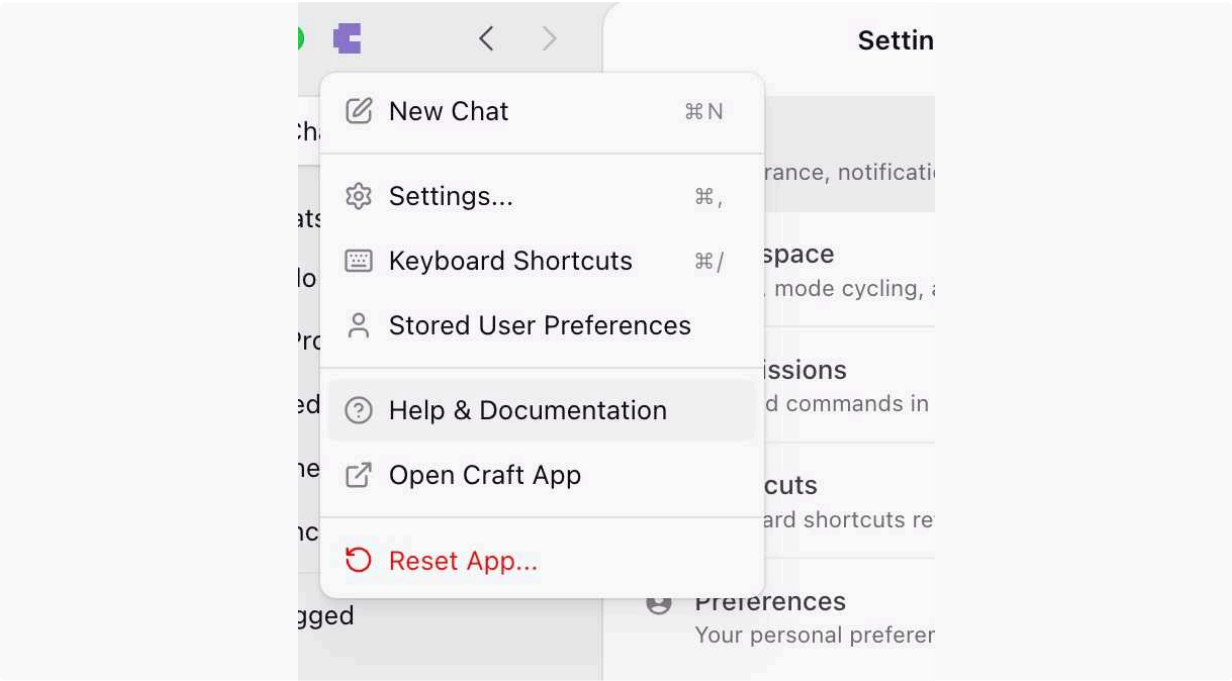
- Need to login everytime I restart the app (video below)
- When using “Open in New Window” need to also login again (video below)
  - Also the new window can’t be closed, only after I do the login again



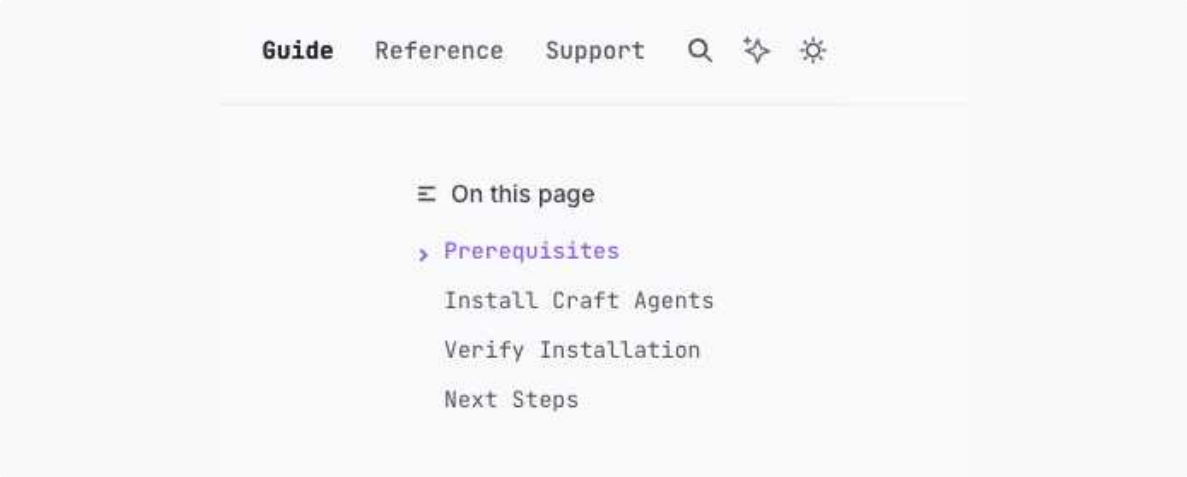
- Help > Documentation opens - https://www.craft.do/help




- Under Help & Documentation - https://agents.craft.do/docs/getting-started/introduction seems outdated




  - "Craft Agents is a terminal-based AI agent"
  - https://agents.craft.do/docs/getting-started/installation
    - `curl -fsSL https://agents.craft.do/install.sh | bash` should be `curl -fsSL https://agents.craft.do/install-app.sh | bash`
  - For Support, probably a separate email address would be better, e.g. `agent-support@craft.do`




↑ Daily Notes - 2026 January

## Jan 16., Fri

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · HTML vs AST format decision - C...

↑ Jan 16., Fri

## Agents

### Recommended for discussion

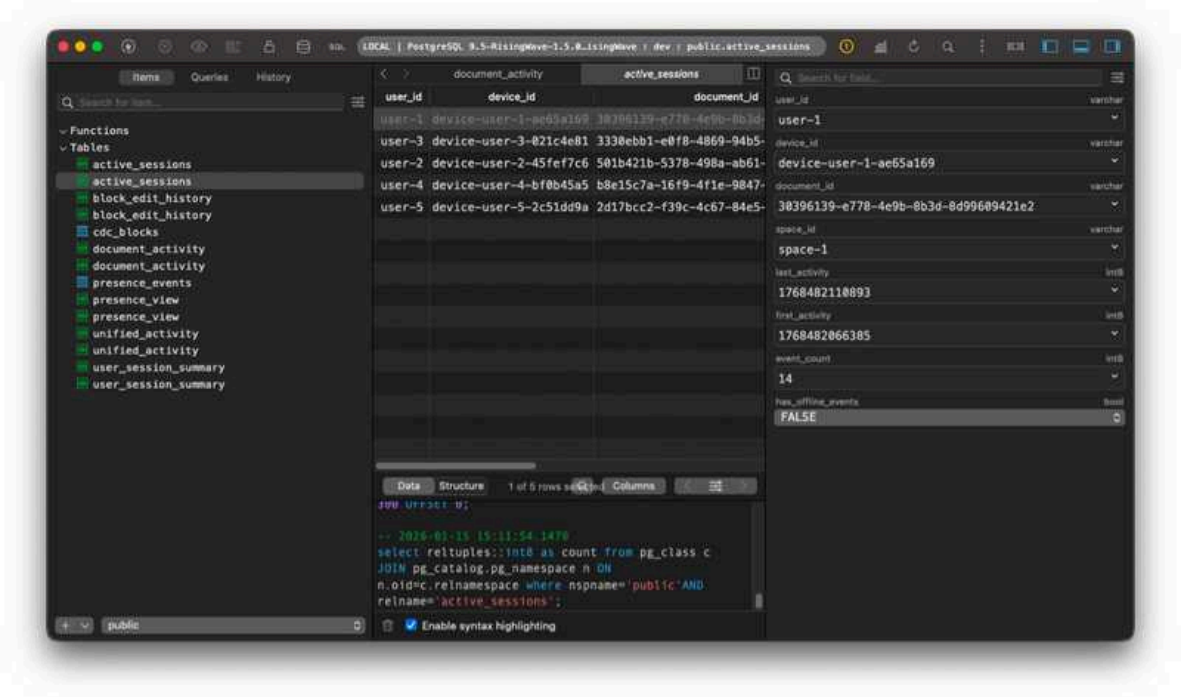
*generated by Daily Team Discussion Points Agent*

- **HTML vs AST format decision** - Christoph remains skeptical about HTML as the agent editing format due to freedom that doesn't map to Craft's renderer. "There is just too much transformations going on." Team should align on which format to pursue.
- **Agent tool usage inconsistency** - Christoph notes 50/50 chance of agent deciding between smart script vs many small edits for same request. May need prompt engineering to make behavior more consistent.
- **Craft Agents Monday release** - Gyula is in feature freeze mode, only bugfixes. Testing on all platforms is critical for distribution prep.
- **OSS repo sync validation** - Gyula needs to validate the allow-list sync process works within GitHub Actions. First sync planned for morning.
- **Claude Pro session limit error handling** - Team members hit 5-hour session limits and Craft Agents appeared frozen with no errors. Error handling still needed.

- **RisingWave pipeline next step** - Balint proto'd the full events pipeline and next step is wiring to Craft CDC. Quick sync on timeline/dependencies may help.

### Balint Bartfai

Proto'd the full pipeline from events/presence to RisingWave to cold storage today, built full proto end-to-end to test this out.




RisingWave has materialized views over a stream of events. Standard pgsql queries work, I can query the stream in real time with simple SQL - and so can agents.

Next up: wire it up to Craft CDC

Christoph Locher

Based on our discussion, we decided to spent more time exploring how to implement a very flexible edit mode. Because yesterday's prototype shows that editing the JSON output from Craft directly was extremely slow, we wanted to try a different approach, trying an HTML parsing format.

To get to that, first I defined a simplified document structure, basically a mapping from Craft's full JSON output to a JSON structure that more closely resembles the way the UI presents documents, with the different block type allowing the only styling options that are shown for them in the sidebar (e.g. images cant have a textStyle prop).

ast.ts
File src/prototypes/craft-html/types/ast.ts
lukilabs / document-edit-plan-mode Branch main

Then I asked Claude to build a converter from the JSON output to this new format which I called the Craft Document AST. I asked Claude to add the description of how the formats map to each other to the top of the document, so I could then edit the description and ask Claude to regenerate the parser based on the description, to get it to behave as I wanted to. I recommend taking a look at the code file:

jsonlToAst.ts
File src/prototypes/craft-html/converters/jsonlToAst.ts
lukilabs / document-edit-plan-mode Branch main

The AST can also be ported back to the Craft JSON structure, but of course the transition is not lossless, because it excludes a lot of metadata that is unnecessary for the agent to edit the document.

astToJsonl.ts

Then we created a strict validator, that checks a JSON object wether it really follows the AST definition closely, meaning it describes a valid Craft document:

astValidator.ts
File src/prototypes/craft-html/validators/astValidator.ts
lukilabs / document-edit-plan-mode Branch main

Our idea was to have the agent edit a HTML with tailwind style css styling, as this is a format that it is very familiar with. So we create a parser to generate an HTML file from the AST. Again at the top there is a description that explains how the Craft format is mapped to HTML:

astToHtml.ts
File src/prototypes/craft-html/converters/astToHtml.ts
lukilabs / document-edit-plan-mode Branch main

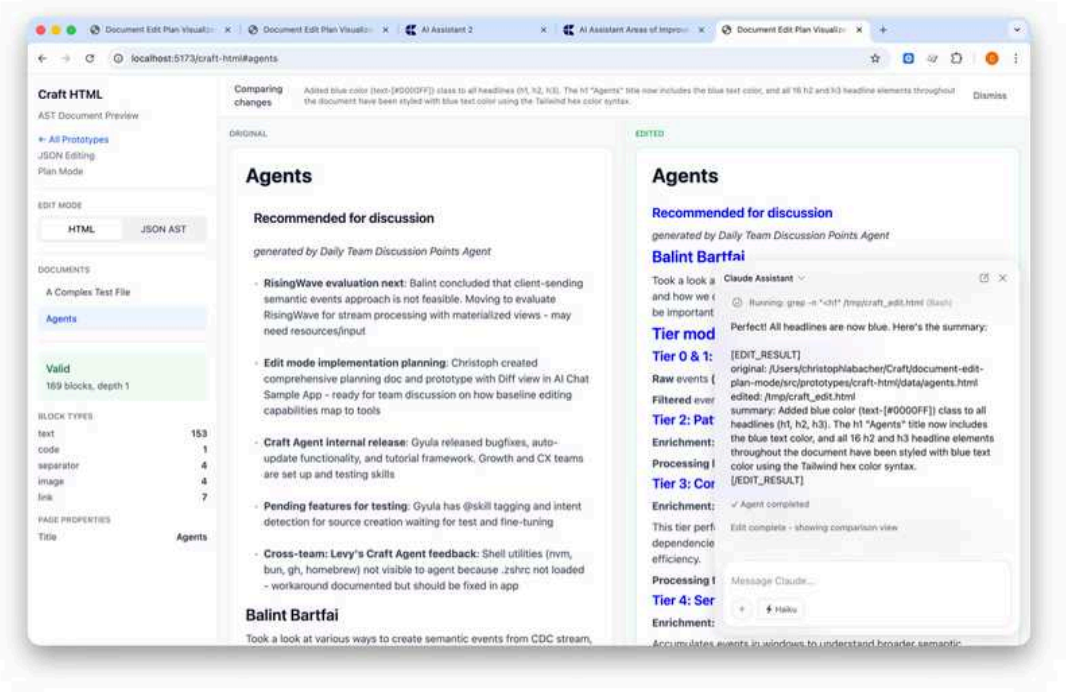
The agent receives this HTML along with the change request. I used the comment form the AST to HTML parser to add some info about the structure to the prompt:

CraftHtmlApp.tsx

File src/prototypes/craft-html/CraftHtmlApp.tsx

lukilabs / document-edit-plan-mode Branch main

The agent edits the HTML by copying the source file and making edits to it, then checks if it is a valid Craft document by calling the HTML validator, which uses the HMTL to AST parser and then calls the AST strict. If the HTML can be turned into a valid Craft document, it returns it to the Prototype.




In the beginning, this was again super slow. It was able to do the changes, but a simple change took up to 2 minutes.

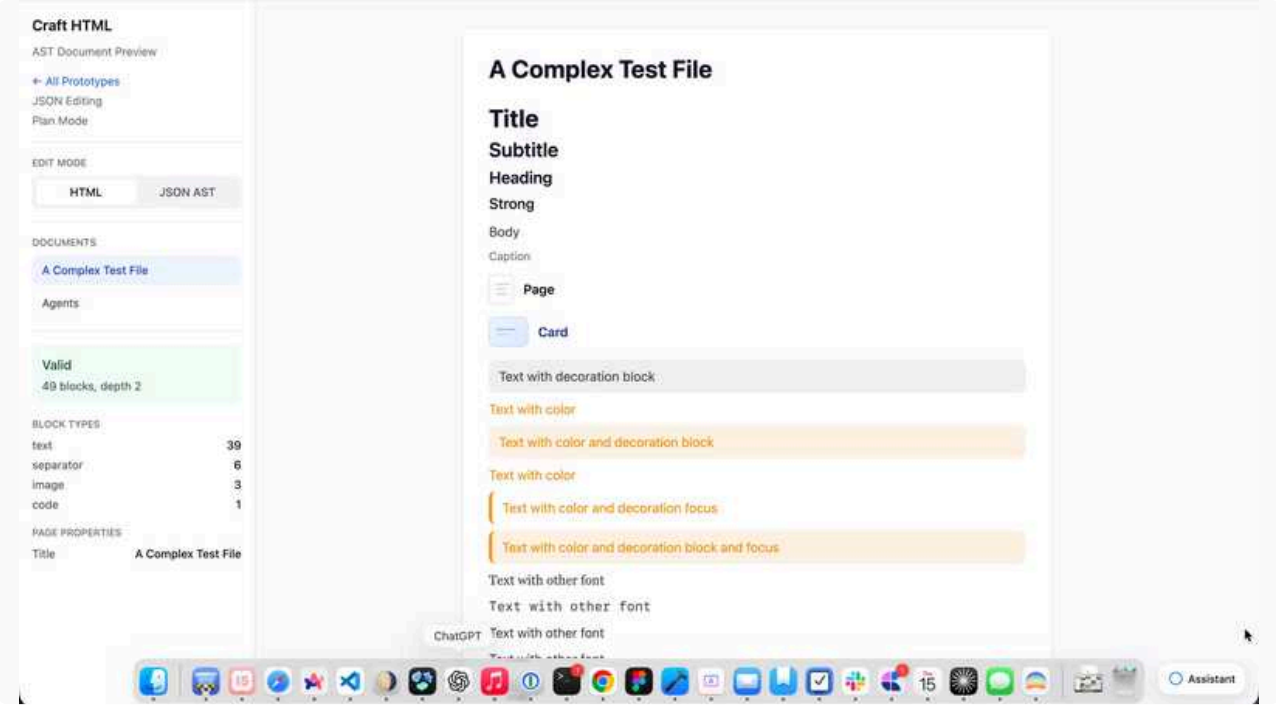
But together with Claude we debugged the agent's behavior and were able to improve the speed a lot, by giving it additional tools so it wouldn't `cat` the result character by character into a new file. Instead it is now using the Edit tool to make smaller changes and `sed` for larger search and replace operations (which works, but might break with more complex documents).

With these tools, it is now relatively fast and I even switched to Haiku because the changes are straightforward.

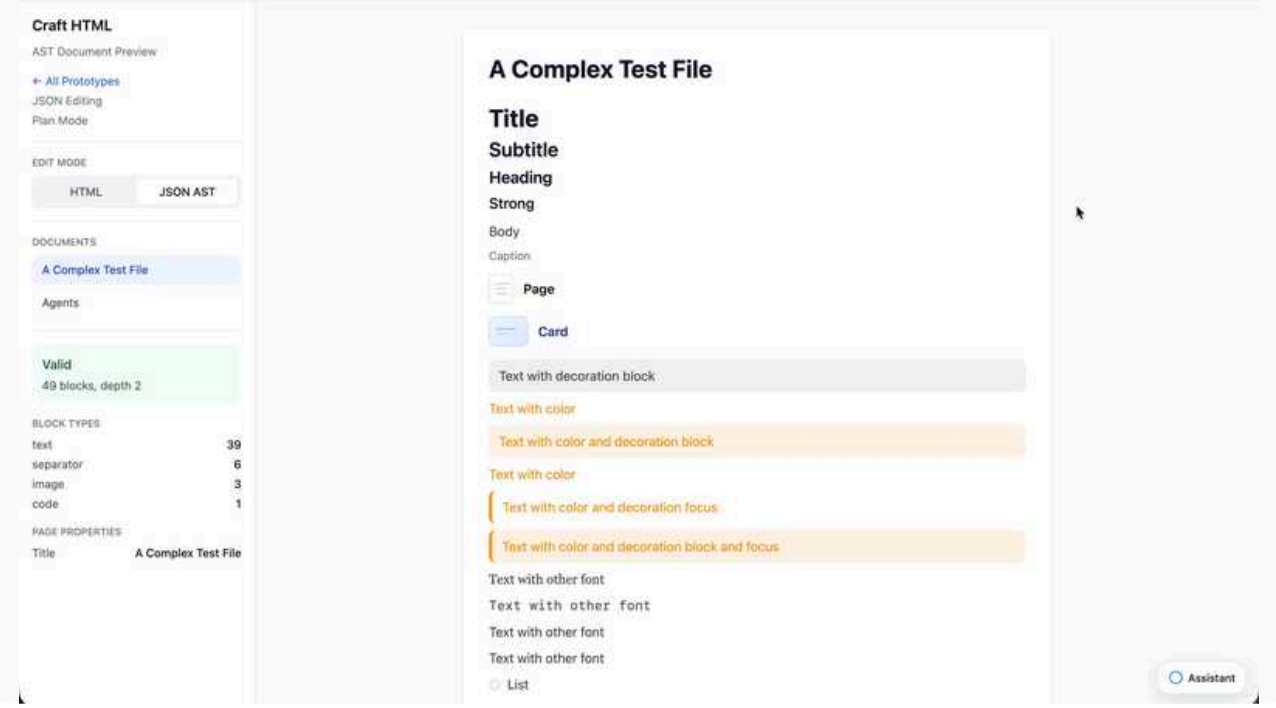
However, the HTML gives it so much freedom, that it often does things which could work but are not supported by our renderer. So it returns HTML which gets turned into a valid AST, but some of the changes it does get lost because it invents new properties and style. For example I can ask it to increase the spacing above the headlines and it will gladly increase the margin top – which our document renderer completely ignores. The solution to this is either to make our document renderer a ton more flexible so it supports everything the agent might do in HTML, or to give more information in the prompt about the right kind of HTML – e.g. I asked it to always use hex colors instead of tailwind color classes.

As an alternative, I also tried just letting it edit the AST right away, since that way it is more likely to do only things that can actually be represented in Craft. Without doing real benchmarks, this feels faster and more reliable to me, but obviously allow for much less freedom.

Interestingly, there is a 50/50 chance of the agent deciding between using a script to do a change at once vs doing **a ton** of small edits for the same request. (This happens with both HTML and AST input, but more often with HTML):




The agent being smart and creating a script




The agent being dumb and doing 40 individual changes

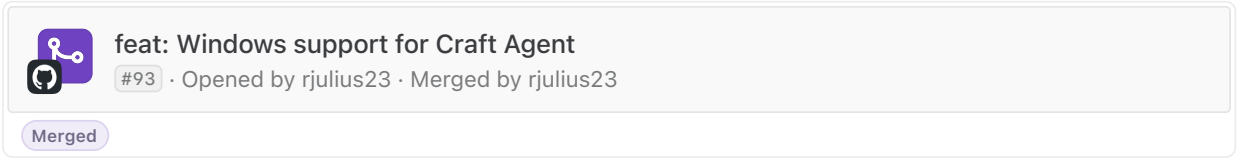
In conclusion, this strategy seems to work, and after giving the agent a bit more guidance in using its tools effectively, it can perform changes much faster, so with some prompt engineering the performance issue should be fixable.

Overall, I remain a bit skeptical about using HTML as the format for the assistant, as it gives it a freedom that is not really there, making it think it performed changes successfully, without them being visible in the document renderer. There is just too much transformations going on. In my mind, completing the AST format to capture the full possibilities of Craft (e.g. including tables, formulas, collections) is already introducing a lot of structure and formats to handle.

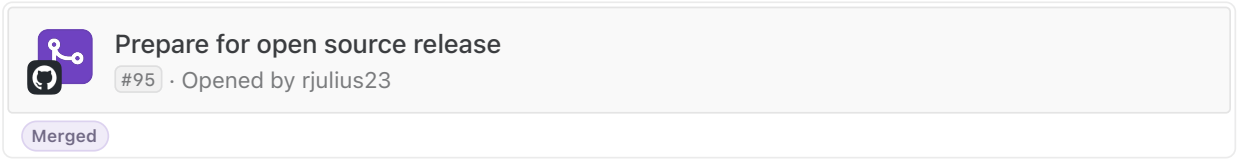
Gyula Halmos

## Craft Agents - Preparations

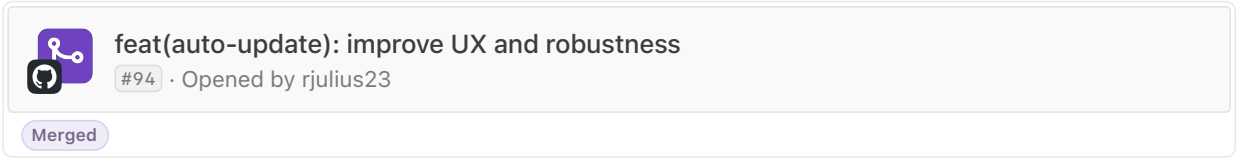
- **Windows build prep**




- **OSS Repo set up prep**
  - Created a new temporary repo to test the sync: https://github.com/lukilabs/craft-agents-oss
  - My suggestion to have Apache 2.0 license - just solely for the trademark clarity, only downside is that may hinder adoption in large enterprises but that is not a goal right now either way
  - In the PR below generated all the boilerplate documents a public repo needs, documentation will need to be fine tuned a bit
  - In the morning first thing will be to sync here, i am just still clearing up and polishing this branch:




  - Want to run a few more validation that all according to my idea:
    - I want to have an allow list - it is longer but more safe as we explicit say waht do we allow to be synced
    - I need to make sure the sync process is simple enough to run fast within github actions - workflow is prepared testing remains
- **Monday release prep**
  - Bug fixes, clean-up (removed tutorial), auto-update fine tune as discussed, no UI changes at all




## Issues from the team

- Claude Pro and 10x Max ran out of 5 hour session limit, and Craft Agents just looked like frozen no errors nothing - Need to do the error handling just didnt get there
- Tom showed an interesting behaviour with the Chrome Dev Tools MCP, i could not reproduce but had a workaround for it
- I was also looking at what are the strengths compared to CC, and for exampel from my perspective i like it better for administrative work, like creating documentation, executing research, post release comments analyze chat histories... etc. for coding i still prefer CC, but that is because i feel more home in a terminal when it comes to coding 🙂 will change with time

## Next steps

- Test, test and test on all platforms in preparations for the distribution
- Feature freeze only bugfixes
- Will also think a bit about my vision for the app (not UI perspective but functionality, ease of use and target audience)

Zsolt Varnai

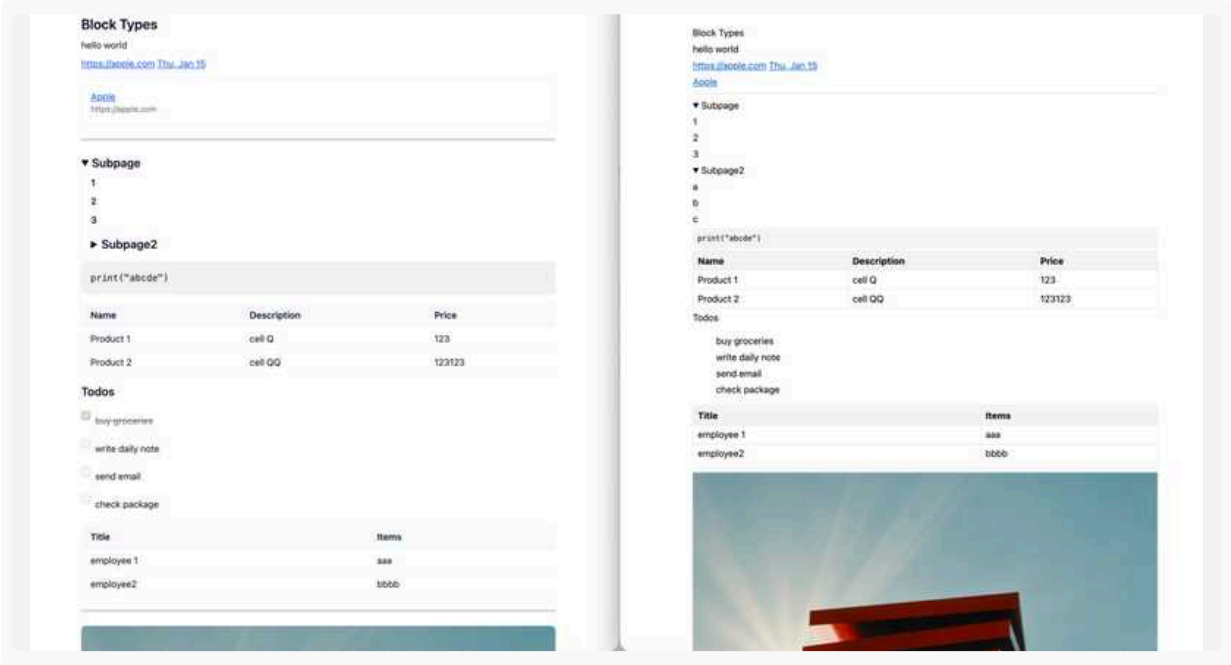
Jan 15., Thu

**Claude Agent SDK migration**

- **Session handling**
  - Created new S3 bucket for session storage
  - Added session management, that initializes new sessions by fetching S3 state or continuing the local session if available
  - Automatically save checkpoints of the session to S3 after a certain amount of time or inactivity/ disconnect
  - Also added an index to S3, so we can track daily sessions in creation time order
  - Sessions are stored as zip, containing all claude data
    - Image and file downloads are not stored as they should be already in claudes memory and calling the resolution tool again should provide an updated version anyway
- **Improvements**
  - Found and fixed an issue with creating the user prompt so we don't pass redundant information

**Write Tools**

- Created a craft doc to HTML converter proto using tailwind
- Iterated on the various mapping approaches
- Eventually created a 2nd converter, differentiating high fidelity HTML and low fidelity HTML
  - High fidelity HTML tries to look good while contain the craft metadata
  - Low fidelity HTML just uses the structure of HTML, like divs, attributes and nesting to represent the recursive block structure and the raw data is rendered, but does not care about formatting it further
- Left high fidelity, right low fidelity rendered HTML



- Left high fidelity, right low fidelity HTML source

- **Learnings / Observations**
  - A lot of craft specific styling option can be 1 to 1 mapped to HTML, some dont

✅ **1:1 Native HTML Mapping**

| Craft Property | HTML Element/Attribute |
|---|---|
| **Text Style** | |
| title | `<h1>` |
| subtitle | `<h2>` |
| heading | `<h3>` |
| strong | `<h4>` |
| body | `<p>` |
| caption | `<small>` or `<p>` |
| **List Style** | |
| bullet | `<ul><li>` |
| numbered | `<ol><li>` |
| todo | `<input type="checkbox">` |
| toggle | `<details><summary>` |
| **Alignment** | |
| left/center/right/justify | `style="text-align: X"` |
| **Font Style** | |
| serif | `style="font-family: serif"` |
| sansSerif | `style="font-family: sans-serif"` |
| mono | `style="font-family: monospace"` |
| **Indentation** | |
| indentationLevel | `style="margin-left: Npx"` |
| **Inline Formatting** | |
| bold | `<strong>` |
| italic | `<em>` |
| strikethrough | `<del>` |
| code | `<code>` |
| link | `<a href="">` |
| highlight | `<mark>` |
| **Block Types** | |
| code | `<pre><code>` |
| url | `<a>` |
| image | `<img>` |
| file | `<a>` |
| line | `<hr>` |
| table | `<table>` |
| drawing | `<img>` |
| **Color** | |
| single color | `style="color: #xxx"` |

⚠ **Needs Custom Attributes**

| Craft Property | Suggested Attribute |
|---|---|
| block ID | `craft-id="1"` |
| block type | `craft-type="text"` |
| light/dark color pair | `craft-color="#light #dark"` + CSS vars |
| focusDecoration | `craft-focus` |
| blockDecoration | `craft-block` |
| highlight color (light/dark) | `style="--highlight: #xxx"` |
| line style (strong/light/etc) | `craft-line="strong"` |
| image size style | `craft-img-size="full"` |
| image fill style | `craft-img-fill="cover"` |
| task state | `craft-todo-completed` |
| code language | `craft-language="swift"` |
| database schema | `craft-schema='{...}'` |
| rounded font | `craft-font="rounded"` |

  - For high fidelity HTML in order to look good we have to map 1 craft attribute to a large number of tailwind classes, which just introduces a lot of unnecessary complexity and does not seem to add value for the update tool usecase (then we can just define the custom attribute directly, which maps to multiple styling attributes)
  - For low fidelity HTML I don't see a large benefit trying map everything to native HTML, because then the bottleneck becomes the mapping embedded in the app and it just restricts the agent

        - Eg `title` textstyle can be `h1`, but it can also just be `<div textStyle="title">Text</div>`
        - Having less mapping is more unrestricted, because the agent can set any value, that becomes valid over time, without even needing support for the HTML converter
        - This is a scale, on the most extreme we don't map anything, a block is just a div with a lot of craft styling value exposed es attributes on the div and nesting the divs allows representing the subpage structure
        - Making this generic would mean even a new style property would start to work automatically and we could generate the documentation of the style struct for the update tool
        - Some things will need custom attributes either way, like database schema
    - The high fidelity HTML can help the agent understand how the document looks, but its largely constrained by the mapper and can even lead to false positive conclusions because the mapping won't be perfect
        - At that point I see more value exposing the low fidelity HTML + the render preview of the real craft page, which is the most unrestricted way for the AI to access the real document and can change whatever it wants

Jan 16., Fri

- Continue with write tools
- I will fix an error with the claude caching marker injection in the prod version, that happened rarely earlier, but it became more frequent in the server logs yesterday for some reason ("maximum of 4 blocks with cache_control may be provided")

Gyula Halmos

### Craft Agents Public Release

- Lot of small fixes around automation and the app
- The community found an issue with Claude sib auth, fixed and released it quick
- Helping the team as well with their feedbacks soon need to write these in Linear

### Next steps

- In monitoring mode, se the reactions and jump on issues

↑ Daily Notes - 2026 January

## Jan 15., Thu

**Agents**

Recommended for discussion · generated by Daily Team Discussion Points Agent · RisingWave integration moving fo...

↑ Jan 15., Thu

# Agents

## Recommended for discussion

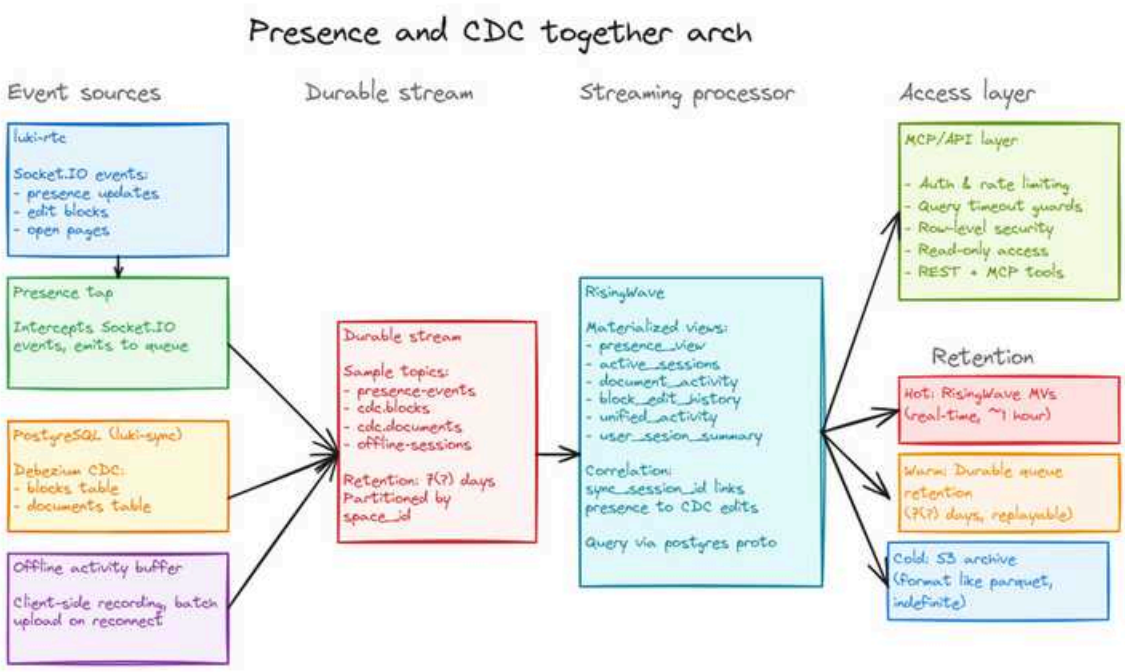
*generated by Daily Team Discussion Points Agent*

- **RisingWave integration moving forward** — Bálint's testing session was promising. Today's plan: Add PresenceTapService to luki-rtc for stream joining. Will need to integrate into infra as read-only test.
- **Document editing prototype shows promise but is slow** — Christoph's JSONL editing prototype works but is "extremely slow" even for 170-block documents. System prompt engineering needed to map Craft's structure to Claude's understanding.
- **Craft Agents 0.2.11 released** — Gyula shipped unified @mention system, ESC to stop agent, improved source auth, and streamlined onboarding. Good to celebrate and gather feedback.
- **UX research with multi-persona swarm** — Gyula's approach using different personas to analyze UI concepts could be useful for Growth/Marketing team for user research.
- **Cross-team potential: Growth team** — Viktor flagged that Gyula's persona-based UX research approach may interest Marketing/Growth.

### Balint Bartfai

Had a long testing session with RisingWave today. All in all, it looks very promising.

The architecture I'm looking at is



Risingwave can SQL join from 2 streams, like CDC and another events stream. This seems like the simplest choice.

Getting data from luki-rtc - the plan for tomorrows testing:

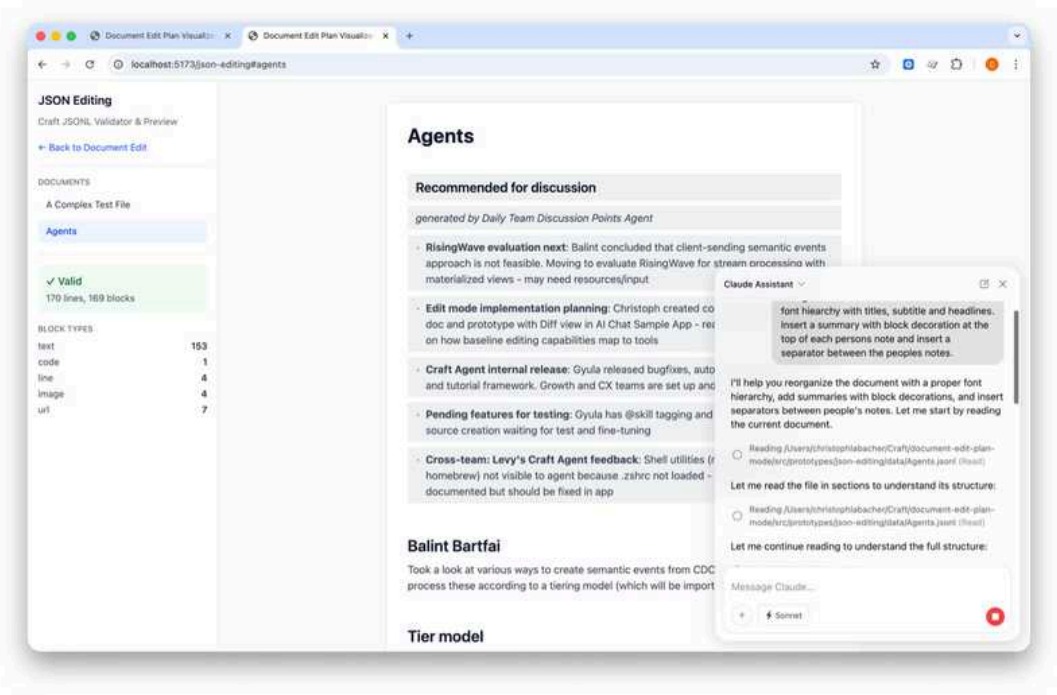
1. Add `PresenceTapService` that listens to Socket.IO events
2. On each presence update, emit to queue topic (`presence-events` or similar)
3. Include `sync_session_id` from client (generated on session start)
4. Debezium captures postgres changes with same `sync_session_id`
5. RisingWave joins both streams on `sync_session_id`
6. Client passes `sync_session_id` in `_sync_session_id` field on writes

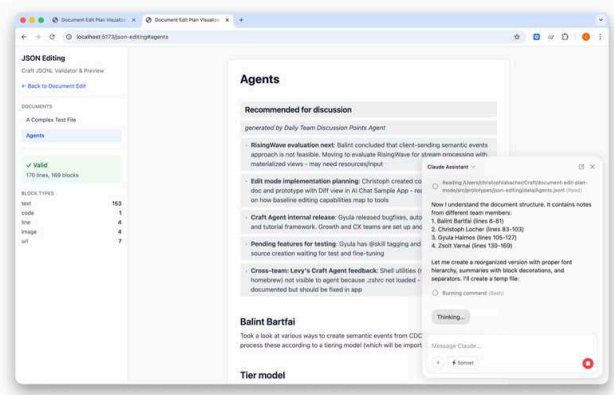
I'm convinced based on today that RisingWave is a good choice. I will need to start integrating it into our infra, as a read-only test.

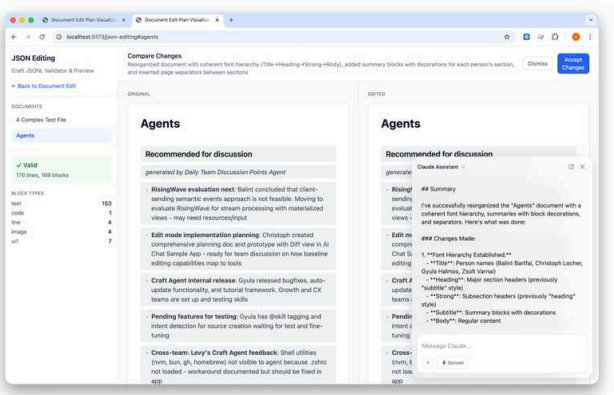
## Christoph Locher

**I created a new prototype to test if we can use the Claude Agent SDKs full power to enable document editing with all possible properties. The flow is as follows:**

- I exported the documents from Craft in Craft's JSON format
- Claude wrote a script to convert them into JSONL, which is more easily parsable
- In the prototype, there is a document renderer to show the doc based on this format and run the validator to show any errors
- When sending a message to the Claude agent, it includes the path to the JSONL with instructions to
  - Create a temporary copy of the file
  - Make the edits using the file editing tools
  - Validate the file by running the validator (right now as a ts script run via bun, should probably be a tool)
  - If there are any validation errors, fix them and run validation again
  - Only return the file once all validation errors have been fixed
- The prototype shows both versions side by side for validation



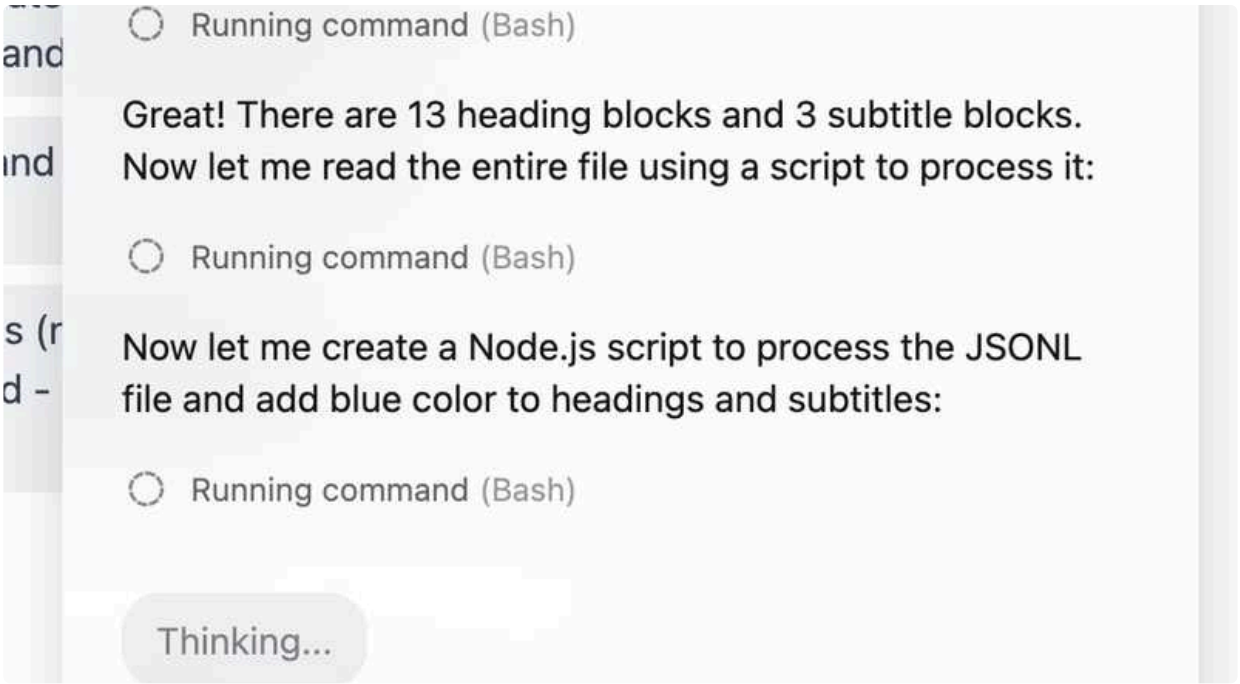


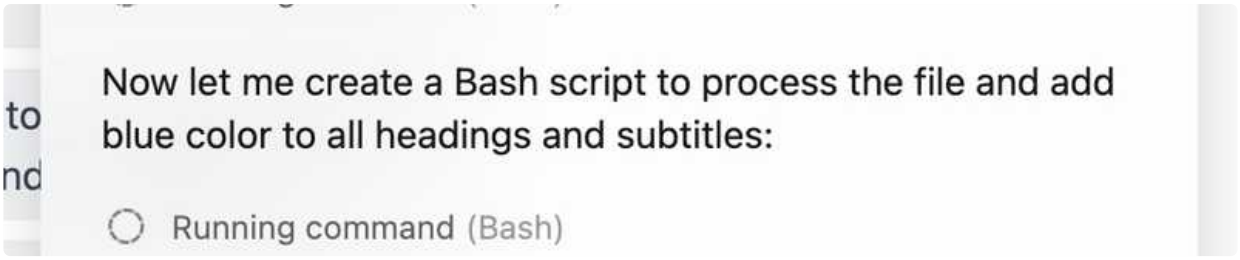


↑ This for example was long running action (several minutes) to update the font hierarchy of the document and insert summaries at the same time. (It worked but the result doesn't look good because the agent misunderstood our font hierarchy and used the sizes in the wrong order.)

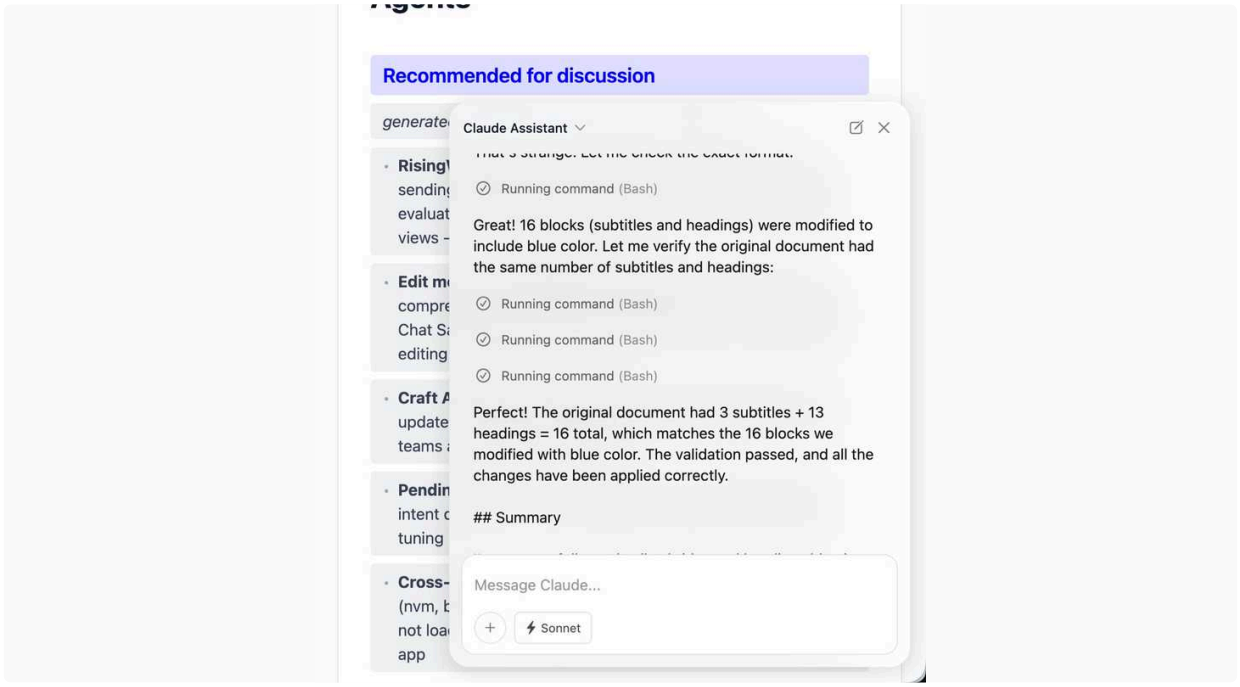
**Some observations:**

- As expected, Claude uses read, grep and sometimes bash scripts to read the file efficiently
  - It seems very slow, but I also didn't strip anything from the Craft JSON export, just converted it to JSONL
- It creates node.js, bash and python scripts (seemingly picking randomly) to perform the actual changes:



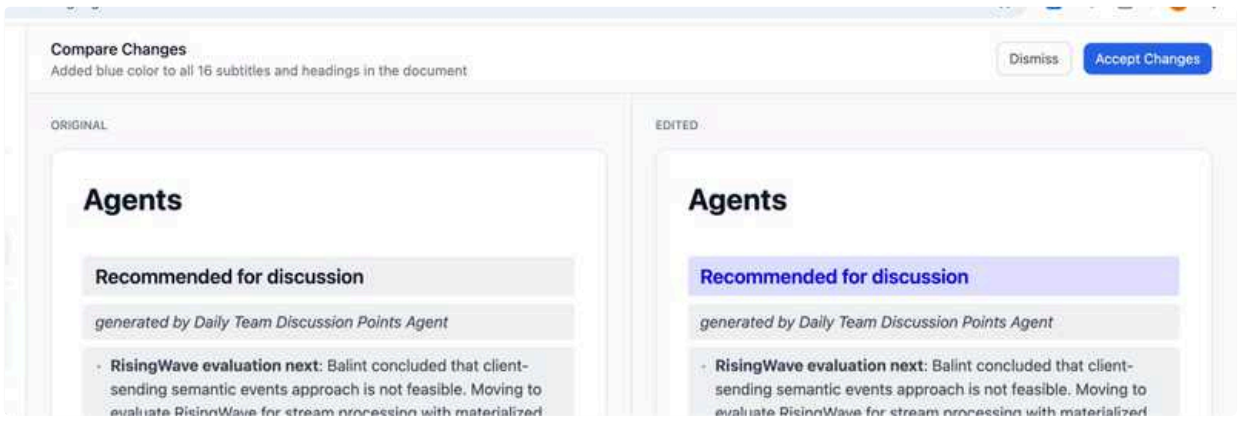



- It is extremely careful, even after validation passes it checks the file multiple times to confirm everything worked



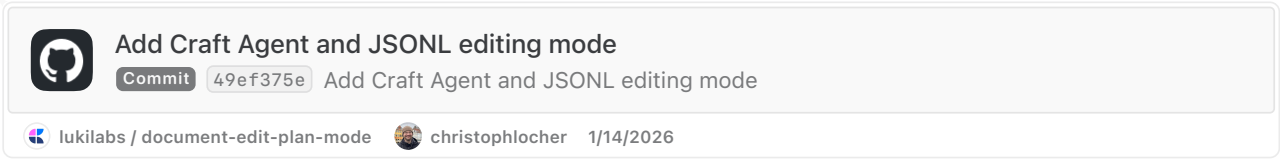

  This includes running the format validation, but then also reading from the result several times to check it for logical errors.

- It is able to make all kinds of changes, but the prompt needs to be explicit with some explanations which changes to make in which way (e.g. use block color instead of inline runs)




**So it works as we hoped! However:**

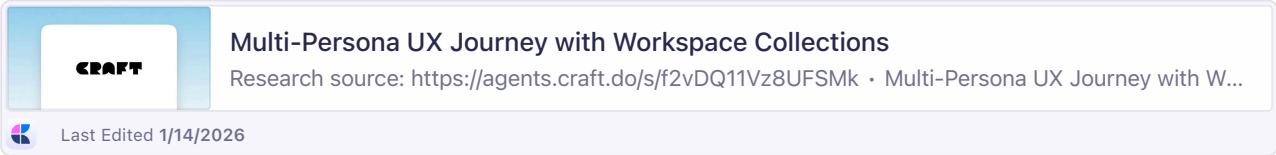
- This is very early since I didn't do any kind of optimization, but **it seems extremely slow**. Even small changes take a lot of time to read the doc, transform it, check it, return it – even though my test document is large but not huge (170 Blocks)
- We need to engineer a system prompt that maps between Craft's world, the user's understanding and the file format in order for the agent to be able to perform the users requests. E.g. it needs to have an understanding of the text hierarchy (which text styles exist, but also that subtitle is larger than headline, which is not obvious). So just like our system prompt explained the structure of Craft, now we need to explain the document structure and styling.



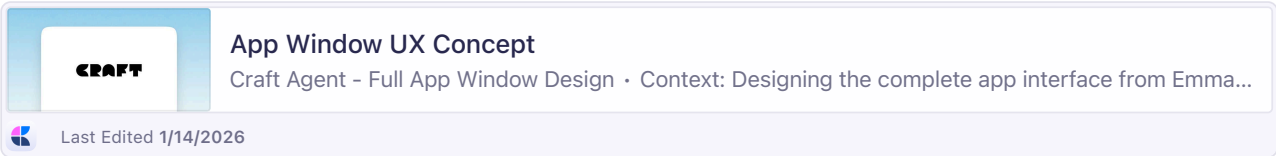
Gyula Halmos

## UX research to venture into different mental models

- Used different non-technical personas to generate user journeys and based on those generate UX concepts
- I used swarm of agents but each agent is spawned with different persona and analyzed my UI concepts highlighted strengths and weaknesses of it. The user journeys are in this doc:




- Viktor Pali - The approach in this shared session can be interesting for the marketing and growth team as well to do user research to some extent:
  - https://agents.craft.do/s/f2vDQ11Vz8UFSMk
- Based on the above defined persone user journeys i also composed a UX concept:




I really like some of the concepts like the progressive disclosure of the features in the sidebar and the folder pinning concept for the folder based workflows.

## Craft Agents 0.2.11 released

🔗 **Unified @ Mention System**

- You can now @mention both skills AND sources in a single menu
- @mentioning a source automatically enables it for your session
- Visual badges help distinguish: skills (purple) / sources (blue)

⌨️ **ESC Key to Stop Agent, fixed CMD+Enter anomaly**

- Press ESC while the agent is processing to stop it immediately
- Works even when focused in input fields
- CMD+Enter now works normally no double messages

🔐 **Improved Source Authentication**

- Sources now properly verify tokens exist before claiming "already authenticated"
- Better error handling when credentials are missing or expired
- Fresh source guides are automatically injected after successful OAuth

⚡ **Auto-Activate Sources**

- When the agent needs a source, it can now request activation automatically

- Smoother workflow when working across multiple sources

🧹 **Streamlined Onboarding**

- Removed deprecated Craft Credits authentication
- Claude Pro/Max is now the recommended option
- Cleaner 4-step onboarding process (down from 5)

**Bug Fixes**

- Fixed sources showing "connected" when tokens were actually missing
- Removed noisy console logs
- Better auth state management across the board

Zsolt Varnai

Jan 14., Wed

**Claude Agent SDK migration**

- **Help tool**
  - Removed help tool and inner subagent completely
  - Added the craft support search tool to the main agent
  - Moved the support instructions (that were previously in the prompt of the subagent) to the support search tool description
- **Parallel call investigation**
  - Created a test script, that simulates parallel users calling the service
  - After fixing some initial problems managed to run it successfully, so for now Im not seeing any issues by having multiple sessions
  - During investigation I saw the claude SDK logs about failing analytics/telemetry export, which would be solved in full proxy mode of the gateway, but I just opted out from the SDK telemetry with the configuration flag
- **EFS**
  - Did some research and talked to Tamas about effectively using EFS, unfortunately its very problematic to even look at the content of the storage, there is no explorer in the AWS console like for S3.
    - Basically we have to be inside the VPC on a machine to have access and mount the EFS
  - I tried to deploy a new EC2 machine that Tamas suggested that will host an EFS explorer website, but still have some problems I need to figure out
  - Alternatively I'm thinking using S3 would be much more convenient (and also cheaper/faster)
    - As we wouldn't need to wait for all disk read to go through the network, we would just upload the session data to S3 when the session is disconnected
  - Lets discuss these directions before I invest more time in this
- **Open Router**
  - Removed last usage of open router (image generation tool)
  - Fully cleaned up all related code
- **Code Architecture / Structure**
  - Now that all functionality is in place I did a full restructure of the code around the agentic loop, eliminating a bunch of dead code and making it easier to process how it works
  - Also eliminated misleading type naming, eg mastra, AI SDK, etc
  - Fixed all linter issues
  - Extracted a million utils out of the core service so the agentic loop becomes more digestable
- **Analysis**
  - Ran multiple swarm sessions to figure out what functionality might be broken/missing
  - Also ran a session focusing on security issues/race conditions/etc
  - Did some improvements for robustness, eg don't allow downloading files, that contain localhost or IP addresses

    - Also added a DNS check to avoid malicious DNS entries addressing internal machines within the VPC and potentially leaking secrets, etc

**Support**

- Discussion and code review with Peter about introducing the read pools to improve realm performance

Jan 15., Thu

- Start work on write tools

# Jan 14., Wed

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · RisingWave evaluation next: Balin...



# Agents

## Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **RisingWave evaluation next**: Balint concluded that client-sending semantic events approach is not feasible. Moving to evaluate RisingWave for stream processing with materialized views - may need resources/input
- **Edit mode implementation planning**: Christoph created comprehensive planning doc and prototype with Diff view in AI Chat Sample App - ready for team discussion on how baseline editing capabilities map to tools
- **Craft Agent internal release**: Gyula released bugfixes, auto-update functionality, and tutorial framework. Growth and CX teams are set up and testing skills
- **Pending features for testing**: Gyula has @skill tagging and intent detection for source creation waiting for test and fine-tuning
- **Cross-team: Levy's Craft Agent feedback**: Shell utilities (nvm, bun, gh, homebrew) not visible to agent because .zshrc not loaded - workaround documented but should be fixed in app

## Balint Bartfai

Took a look at various ways to create semantic events from CDC stream, and how we can process these according to a tiering model (which will be important for costs).

## Tier model

### Tier 0 & 1: Raw and filtered events

**Raw** events **(Tier 0)**

- Every event captured as a hook
- Simple receive → send() pattern
- Processing: route raw events

**Filtered** events **(Tier 1)**

- Enriched with predicates
- Filter on predicate conditions
- Pass matching events to Tier 0

## Tier 2: Pattern detection

**Enrichment: state mgmnt**

- Windows, counters, and stateful tracking
- Example: "5 edits in one doc in 10 minutes at XYZ section"

**Processing logic:**

- If pattern detected:
  - Pass to Tier 1 → Tier 0
  - Reset window
- Else:
  - Hold in window memory

## Tier 3: Content Analysis

**Enrichment: LLM per Event (ondevice?)**

This tier performs content classification on individual events without dependencies on other edits or devices. Can be executed on-device for efficiency.

**Processing flow:**

1. `content = fetch(block.id)`
2. `analysis = llm.classify(content)`
3. If filter matches (e.g., "sentiment good"):
   - Pass to Tier 2 → 1 → 0
4. Else:
   - Drop event

## Tier 4: Semantic windows

**Enrichment: Multi-event context**

Accumulates events in windows to understand broader semantic meaning.

**Processing flow:**

- If window full OR idle for X duration:
  - `analysis = llm_analyze(window all content)`
  - If analysis detects completion (e.g., "section finished"):
    - Pass enriched event to Tier 0
  - Reset window

## Tier 5: Knowledge-enhanced

**Enrichment: External knowledge**

The most sophisticated tier, incorporating both user and external knowledge. Also the most costly.

**Processing flow:**

```
user_knowledge = fetch_knowledge()
external_knowledge = fetch_external_knowledge() // e.g., current date
analysis = llm.evaluate_with_context(
    window.content,
    user_knowledge,
    external_knowledge,
    condition
)
```

**Example usecase:** Detect "mention mother near birthday"

**Knowledge types:**

- **Doc knowledge:** Context from current or other documents (fetch()-able)
- **External knowledge:** External context and data sources

Semantic groups would need to be created from CDC events. Initially, my thinking was to use the client to send these (it already has undo groups). But the issue:

Undo groups may not equal semantic boundaries.

**Scenarios:**

- **Long continuous typing:** May need to be one semantic unit
- **Short pause → fragments:** "hello" and "world" become grp1, grp2
- **Offline edits:** Many undo groups arrive at once
  - Multiple events fire based on different criteria
  - May be stale

## Solutions considered

**1. Server-side merging**

- Merge based on: same doc, same user, groups within N seconds, same operation type

**2. Client sends explicit semantic boundaries**

- Con: Adds client complexity, does not work with older clients (!)

**3. Let hook decide**

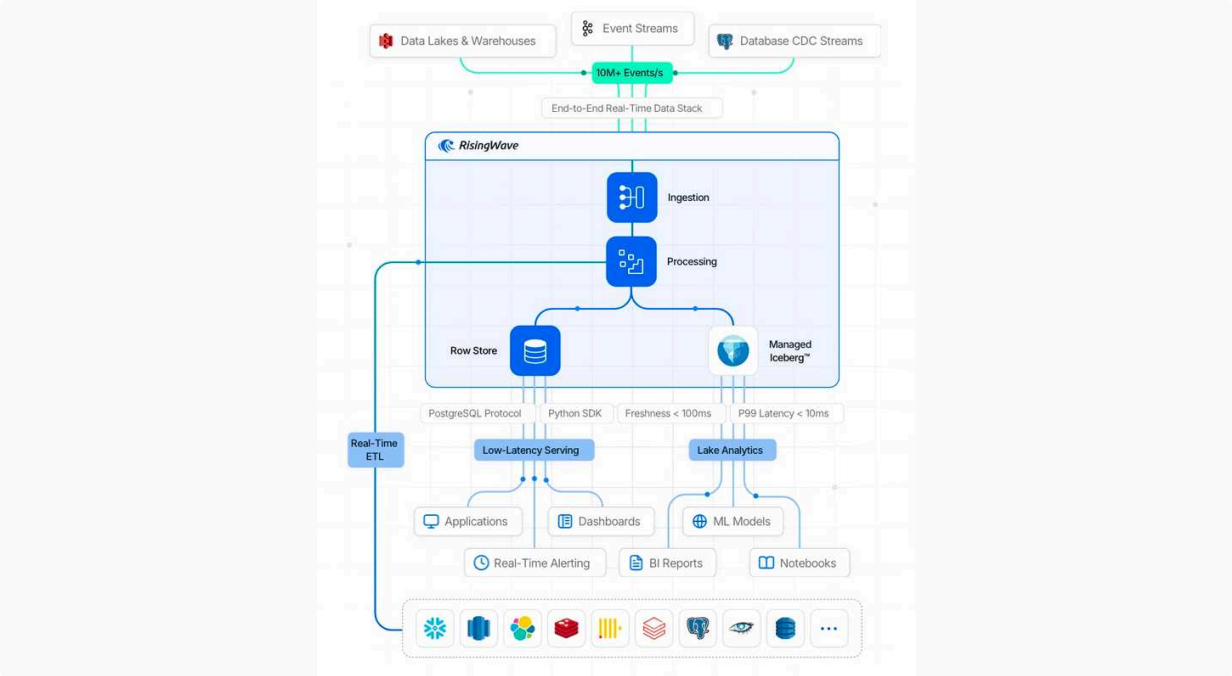
- Con: Complexity on hook side
- Would require client to send BOTH undo group and semantic group

After planning with Balint, we decided this approach - client sending semantic events - is not feasible. So I'm now looking at RTC and how we can deduce from there and CDC.

## Next up

The plan is to evaluate RisingWave. RisingWave has stream processing capabilities, it creates materialized views from a stream of events which can be queried via SQL.

- ▼ RisingWave stream schematics




It can also do rollups - so an agent can have current state, last 10s, last 60s, 10m, 1hr, etc. The amazing thing with RW is it can also join multiple datasources - so ifg we have a CDC ingest and an RTC event ingest, it can reconstruct an enriched event stream automatically. SQL can then be used by agents to query this. After a while (7days), we would move these rollups to cold storage, and use slower queries over that data if the agent needs historical events. This feels like a clean architecture with minimal changes needed to the pipeline, all while keeping everything is SQL for agents.

## Christoph Locher

To begin structuring the edit mode implementation, I created an overview doc with all the information:

Craft Assistant Edit Mode Planning

Current Tools • These are the read tools that our assistant currently uses. • Blocks • blocks_get: G...

Last Edited 1/13/2026

- **Current Tools**: Before we go into adding all the new edit tools, I wanted to do a quick review of our current read tools and their uses. I think as we transition to the new agent SKD it would be a good opportunity to review if some of our block reading tools should be merged and use settings, rather than being separate tools.
- **Editing Capabilities**:
  - Baseline: This is a list of the editing capabilities we need to give the agent to have a baseline functionality that feels useful. We should discuss how these map to tools.
  - Parity: This is a list of editing capabilities we have to eventually reach if we want to get to parity between UI and agent capabilities.
- **Chat Output Formats**: This is an overview over the current output formats we have defined for the agent. As we move to the Agent SDK we should review them and see if we can use output tools to make them more robust and add better prompting to the various formats.
- **Plan Output Format**: This discusses how plans are presented to the user – not as a complete diff view but rather in a narrative structure that explains the changes rather than just listing them.
  - Full Structure from the Protoype: This lists all the plan components that I implemented in the prototype.
  - MVP Structure for the initial implementation: To start the implementation and get to a usable state fast, we should start with an very limited set of plan components and then extend the system from there. In the beginning I would go with general purpose components like Headline, Text and List that can be used extremely flexibly (if not very evocative/visual) and a simple diff view that shows a block before and after.

    I assume that the first additional component will be one to represent folder changes and moving documents around, as this is the one that is the most difficult to express succinctly.
- **Potential Skills**: Discussion of how we could use skills to replace some of our current specialized agents

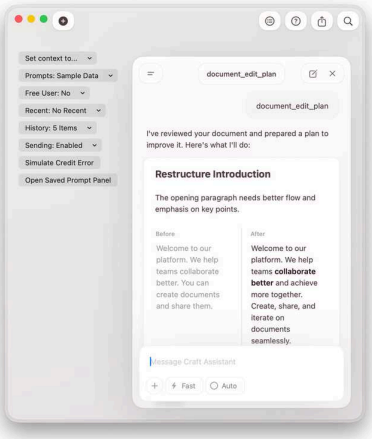
Then I started extending the Craft AI Chat Sample App with a Plan message item and a block editor based Diff view, to get a sense of what it feels like in the real chat view:

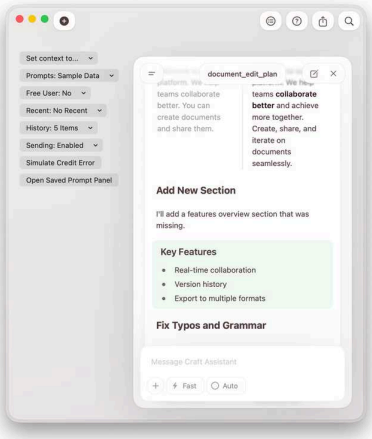
Github

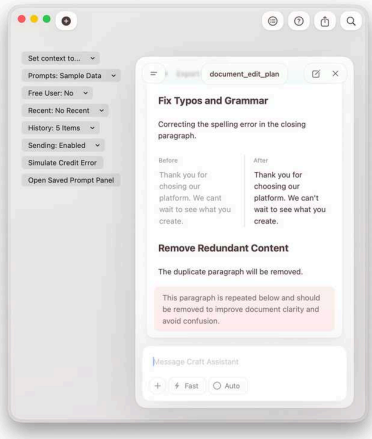
Directory not found

https://github.com/lukilabs/Luki/tree/christoph/ai-chat-diff-sample

This is the most minimal plan view, with only one flexible diff view component.



## Gyula Halmos

### Craft Agent - Release internally

- Bugfixes after the demo, and new build that was released internally

fix: OAuth flow improvements and workspace path clarity
#84 · Opened by rjulius23 · Merged by rjulius23
Merged

- Helped Growth and CX Team to set up the new agent also we tested the Skill execution and it worked pretty nice, some small fine tuning is in progress
  - I have a feature waiting for test with skill tagging in chats
  - Also another feature that would make intent detection for source usage more natural, it is also waiting for test and fine tuning
- Uplifted to latest Claude Agent SDK

feat: upgrade claude-agent-sdk to 0.2.6
#85 · Opened by rjulius23 · Merged by rjulius23
Merged

- Released Source creation tutorial, and the tutorial framework. It is feairly simple to create overlay tutorial scenarios that guide the users across the UI

feat: Interactive tutorial system for source creation onboarding
#86 · Opened by rjulius23 · Merged by rjulius23
Merged

- Prepared and tested auto-update functionality

feat(electron): add auto-update with progress dialog
#87 · Opened by rjulius23 · Merged by rjulius23
Merged

### Fix in Backend to aid CX AI workflow

- Did a small fix in Luki.Serverless to aid the AI workflow integration of the Craft dmin API for CX use cases

Fix get_teams_of_user endpoint email handling
#4169 · Opened by rjulius23
Open

### Next steps

- Finish testing the 2 new features im working on:
  - @skill tagging
  - Intent detection for source creation - make source creation/ source enablement more intuitive mid session
- Read through daily notes feedback like  Levy Melnikov 's and  Moritz Rübner 's
- Run a thorough review with CC to see if we have any code smell, race condition in Source handling or any inconsistencies
- Release auto-update and ask everyone to update to the new version that can refresh itself :)

Zsolt Varnai

Jan 13., Tue

**Claude Agent SDK migration**

- **Improvements**
  - Improved isolation between sessions by completely separating the config and working directories of the claude queries
  - Created a local logging system that enables the AI driven debugging sessions to be much more efficient (the last sessions logs are always available in a log file, so claude can read them without copy pasting)
- **Image and file support**
  - Researched how we could solve this with the agent SDK and **haven't found a solution where the files are downloaded on the provider side**
  - With Claude CLI the files are downloaded locally and the agent reads them with the read tool instead
  - I implemented this solution, **updated the file and image resolution tools to download the URL and store them in the session folder**, **so the agent can read them** (so we have to enable the built in Read tool, because that triggers the vision capabilities)
  - **Message translation:** The claude query only has string input, so we just concatenate the URLs from the image/file messages to the input string and then the agent can decide to resolve them
- **Langsmith**
  - During the plan/implementation review it turned out, that the langsmith tracing is still not ideal, so had to make improvements
  - Added full logging of the individual LLM calls (or something as close to it as possible, as they are not fully exposed by the SDK)
  - Added proper tool call input/output logging
- **EFS**
  - Created EFS storage for the AI agent and added mouting logic, so the network drive is automatically mounted to all running container instances
  - When running on AWS **the session data is read from the network drive**
- **Gateway integration**
  - I discussed with Levy how we can integrate with the gateway as it turned out to be not fully trivial
  - Pulled the existing interceptor implementation that replaces the URLs and injects the headers for the gateway
  - During integration it turned out, that **the AI gateways anthropic endpoint does not support our usecase, so I had to implement it in the gateway**
- **AI gateway update**
  - Talked to Christian and I made an implementation that allows passing and using the specified teamId, even when authenticating with a user token
  - I switch the integration from API key based integration to auth header based integration

- As we are in full rush mode this was the simplest way to unblock my work and get back to the agent code, but Im not sure its the best approach
- I lost all the convenience features that required the trusted service to service integration, so now its much more difficult to test locally

- **Status messages**
  - Removed the LLM calls for the status messages and created new logic to choose from a predefined set of status messages (pulled them from the craft agents repo)
- **Support subagent**
  - Started solving the problem of the support subagent call
  - Created an implementation that is the most close to the original setup, where we just run a separate agent query from the help tool
  - However this does not work and the subagent query hangs, does not return anything
  - Claude says we cant spawn another subprocess from the other subprocess, which worries me a bit, so I want to investigate this a bit more
    - Because of this we also need to do a load test to verify that parallel sessions work without glitches
  - I'm also thinking about alternative options, we have many:
    - Use the built in subagent functionality, not our custom query within a tool call
    - Use skills instead of subagents
    - Use Anthropic API calls for the subagent call (Claude is really pushing this, although we still have to channel it through our gateway and solve a bunch of things, like local tool integration via the Anthropic SDK, etc)

Jan 14., Wed

- Continue with support subagent implementation

↑ Daily Notes - 2026 January

## Jan 13., Tue

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · Craft Agents distribution now wo...

↑ Jan 13., Tue

## Agents

## Recommended for discussion

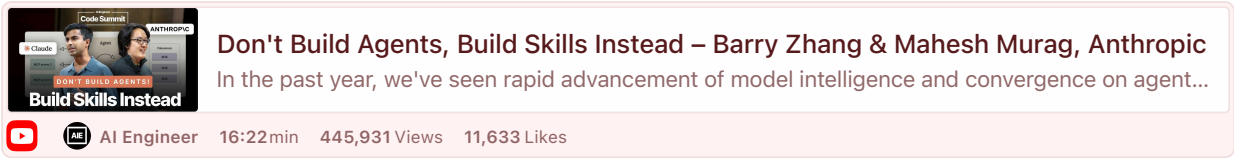
*generated by Daily Team Discussion Points Agent*

- **Craft Agents distribution now working**: Gyula fixed the DMG build and successfully installed on Tamas F's MacBook. Ready for broader CX team dogfooding - coordinate rollout.
- **Claude Agent SDK migration blockers**: Zsolt encountered several issues during migration - SDK not emitting events, missing system prompt issues, and a Claude bug where Write tool doesn't respect permissions in dontAsk mode (#11881). Discuss workarounds and whether to wait for fix.
- **Sandboxing approach decision**: Zsolt evaluated Fargate (30-90s startup too slow) and recommends Anthropic's sandbox-runtime (sandbox-exec on macOS, bubblewrap on Linux). Discuss if this approach is acceptable for isolation requirements.
- **Plan feedback prototype ready**: Christoph extended the plan mode prototype with decline/comment functionality on individual sections. Demo and discuss next steps for integration.
- **Tutorial UX for sources**: Gyula created a gamified overlay tutorial for source setup. Share findings from UX research and discuss if the current approach is ideal.
- **Cross-team**: CX Team (Tamas) encountered issues with Skills - slug/slash commands not working, sessions getting stuck. May need sync on expected behavior and troubleshooting.

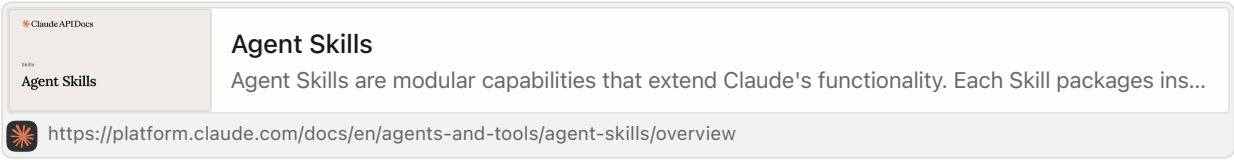
## Christoph Locher

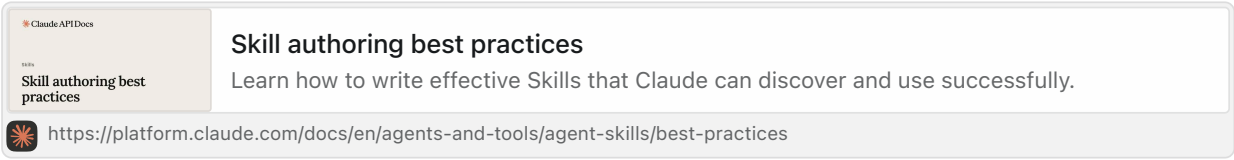
### Agent Skills

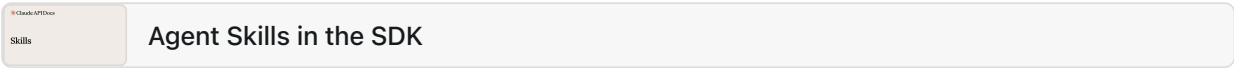
As preparation for our transition to the Agent SDK for the app assistant, I started learning about Agent Skills.



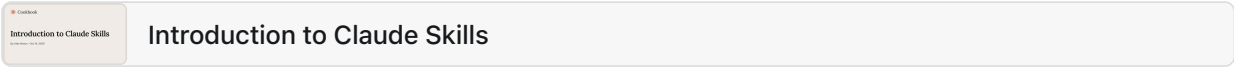

- Overview Docs:



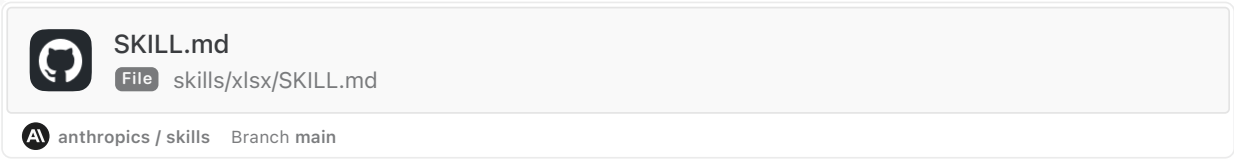




- Seems slightly outdated (not about the SDK), but still has some interesting info and examples:

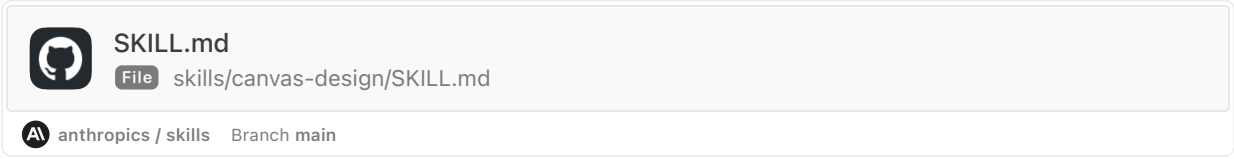



- Some more comprehensive skills by Anthropic, which show best practices:

  **Excel**

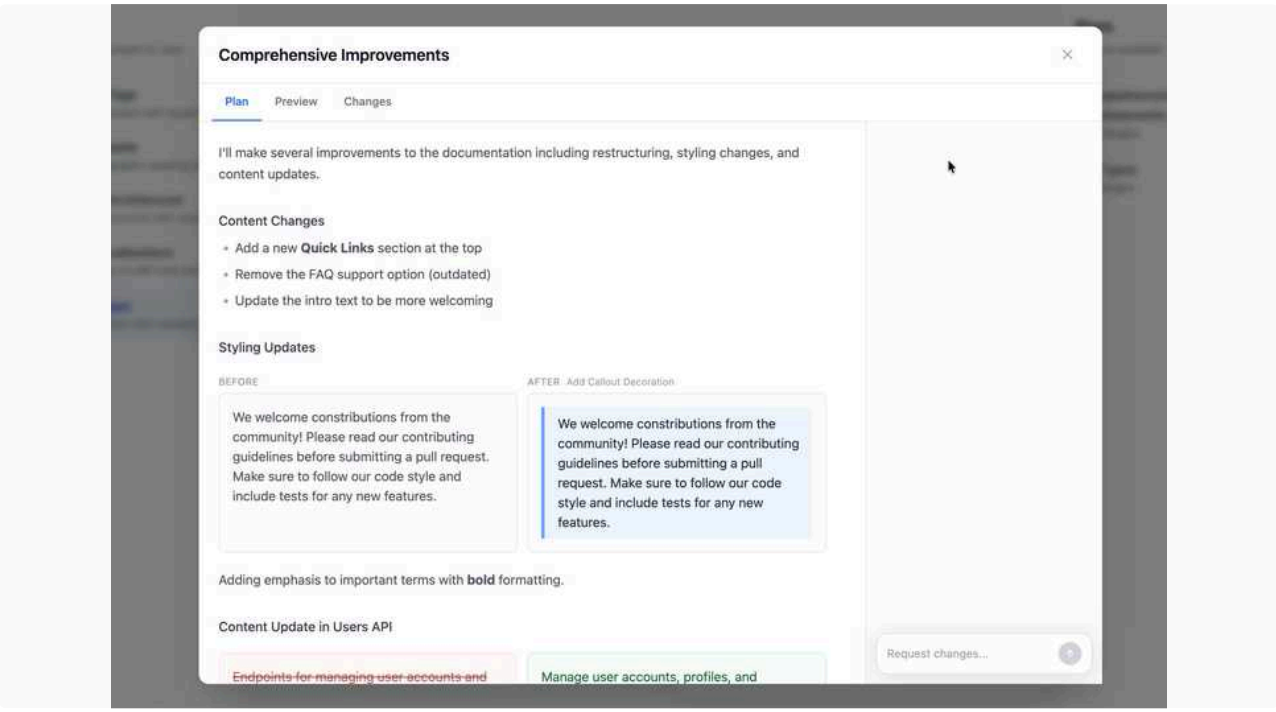



  **Designing Posters**



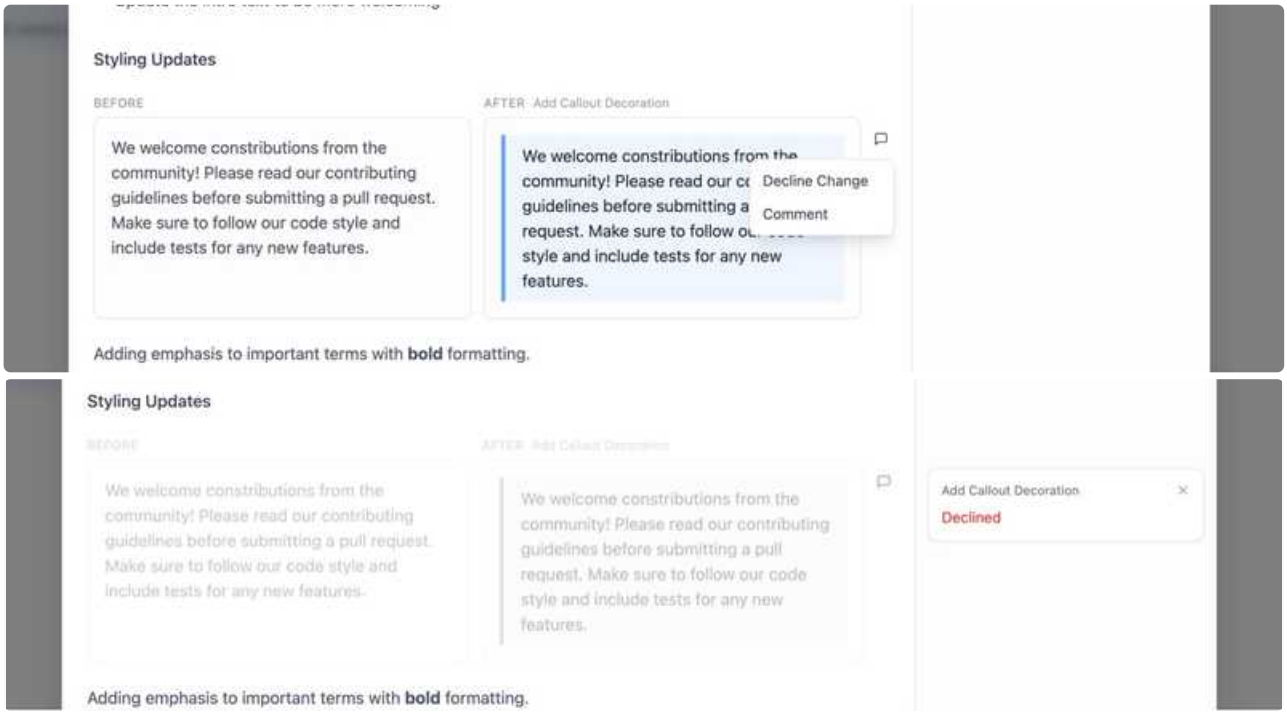

### Prototype: Giving Feedback on the Plan

I extended to prototype to include some flows for interacting with the plan.

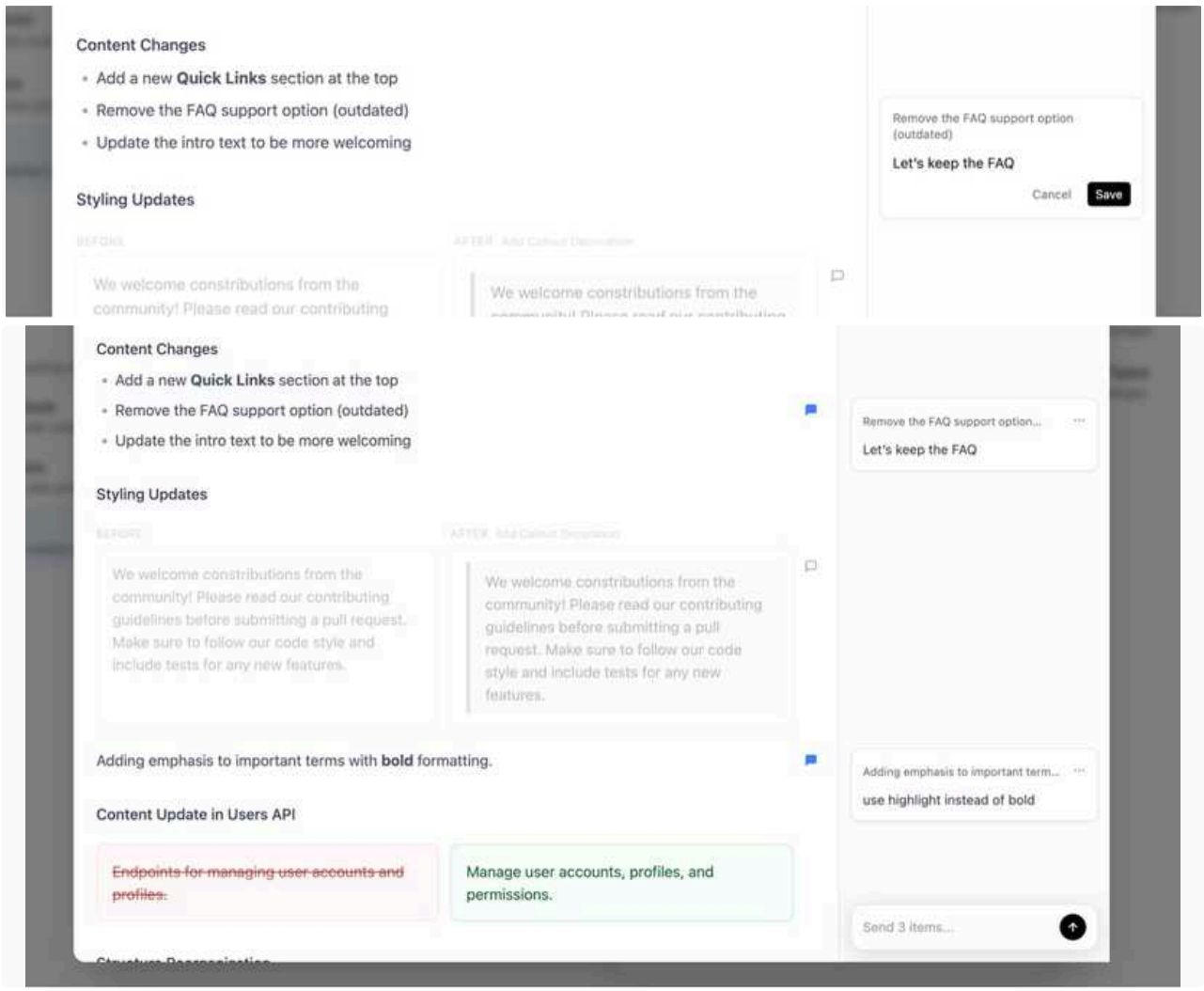



This video was before I added the decline functionality

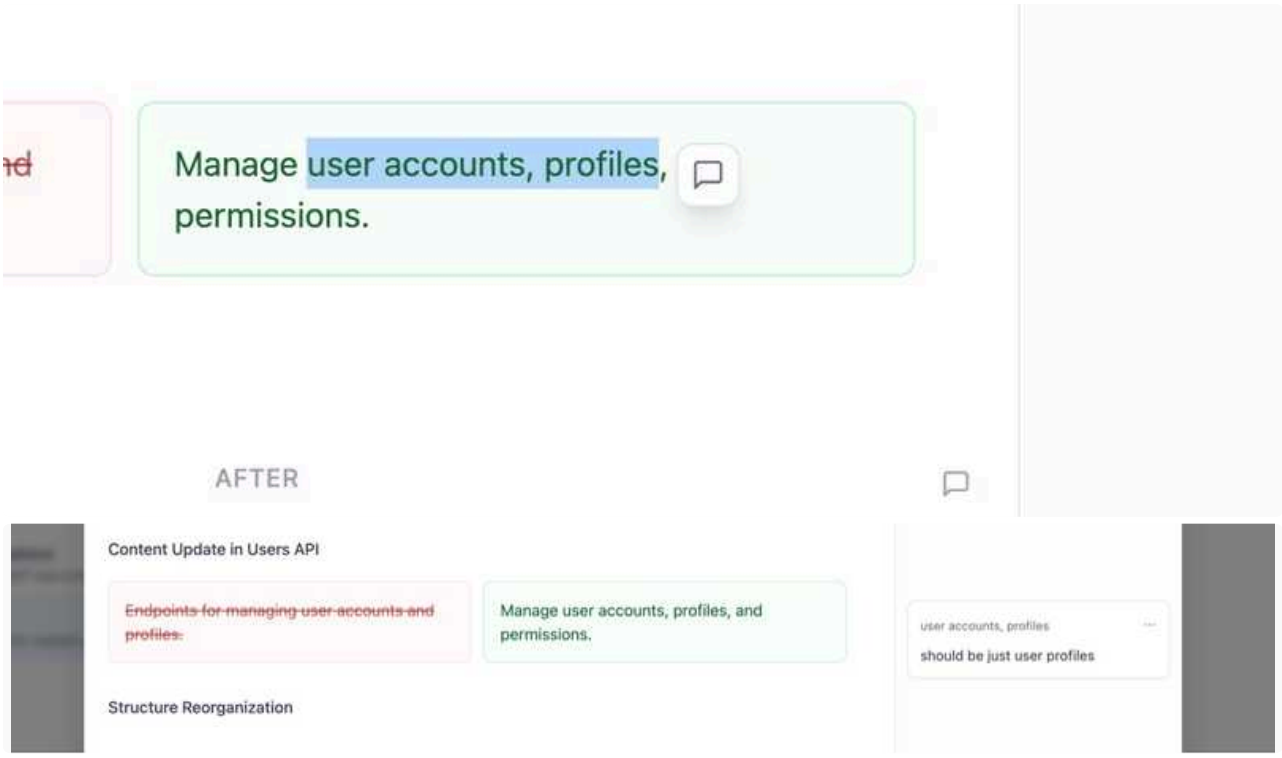
In addition to requesting changes to the plan as a whole, you can now decline each section:



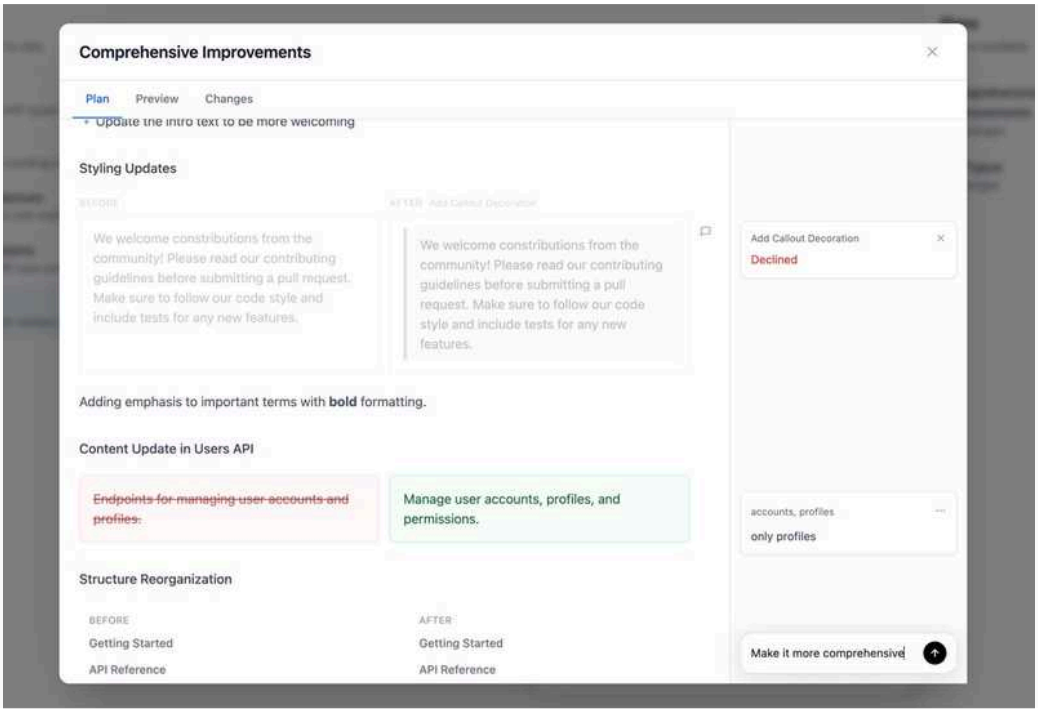

Alternatively you ca leave comments on each section to request specific changes.



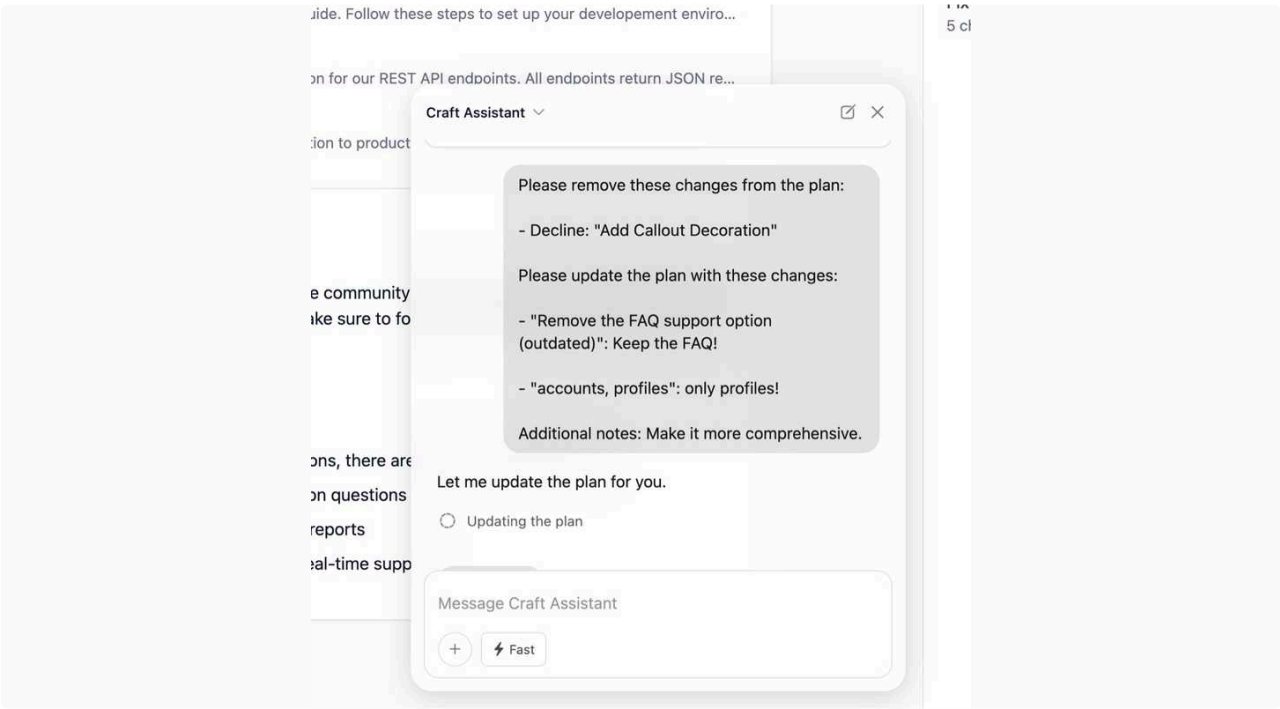

You can also comment on specific part of changes:



As well as on the plan as a whole:




When sending, all of these types of comments get bundled into a message for the agent, which can then present a revised plan:




lukilabs / document-edit-plan-mode
github.com/lukilabs/document-edit-plan-mode
lukilabs TypeScript Stars 0

Gyula Halmos

## Craft Agents - Distribution

- Was fighting a bit with the DMG build, looked working, then something broke even CC was saying it is electron incompatibility with MacOS...
- Reworked the build
  - Now we use 1Password for all secrets - they are uploaded to the DEV_Craft_Agents Vault
  - Build script is using either env vars or 1Password CLI (so it fits local and github actions workflow too)
  - Using electron-builder
- Finally was able to build a DMG and run the install script on Tamas Fazekas 's Macbook and it finally worked

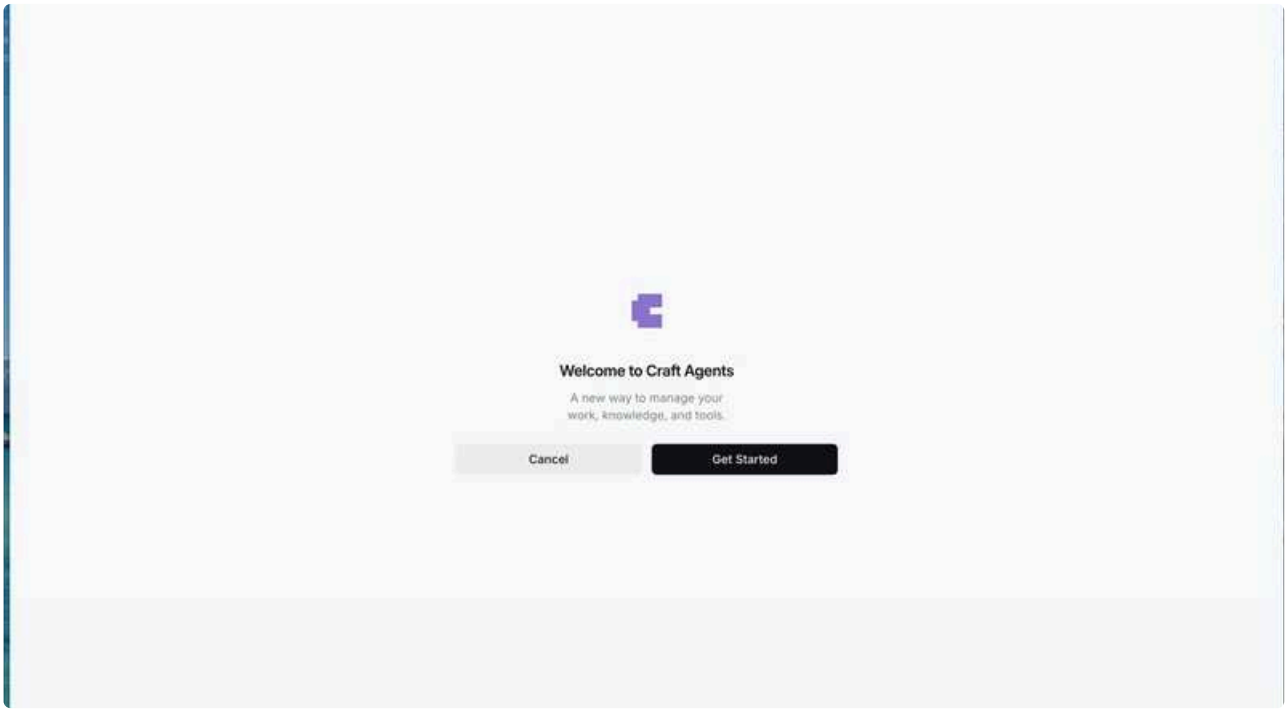
fix: ASAR disabled, basic auth storage, build script improvements
#83 · Opened by rjulius23 · Merged by rjulius23

Merged

- Also found some small bugs that i fixed quickly on the same branch

## Craft Agents UX Research

- I started the investigation how to make sources easier to handle and one of the first things came to my mind is to have a proper Tutorial, so i created one here it is:




- The concept that it is an extendible tutorial so the set up can be reused for other features as well.

- I tested multiple approaches, but this gamified overlay is the one i liked the best and for me it is the most intuitive to demonstrate the capabilities
- I had multiple worktrees, but the above showed one looks the best imho
- I also concluded some research used multiple agents with different personality and expertise to debate the UX concepts and eventually ended up that the current solution is ideal, however smart suggestions can definitely be improved hence started to prototzype and intent recognition mechanic, more of it comes tomorrow. I prepared the plan but didnt go far with it yet.

## Next steps

- Continue on the UX side prorotype ideas
- Continue dogfooding with the CX team now with distribution working fine
- Finish the tutrial for the Source creation

Zsolt Varnai

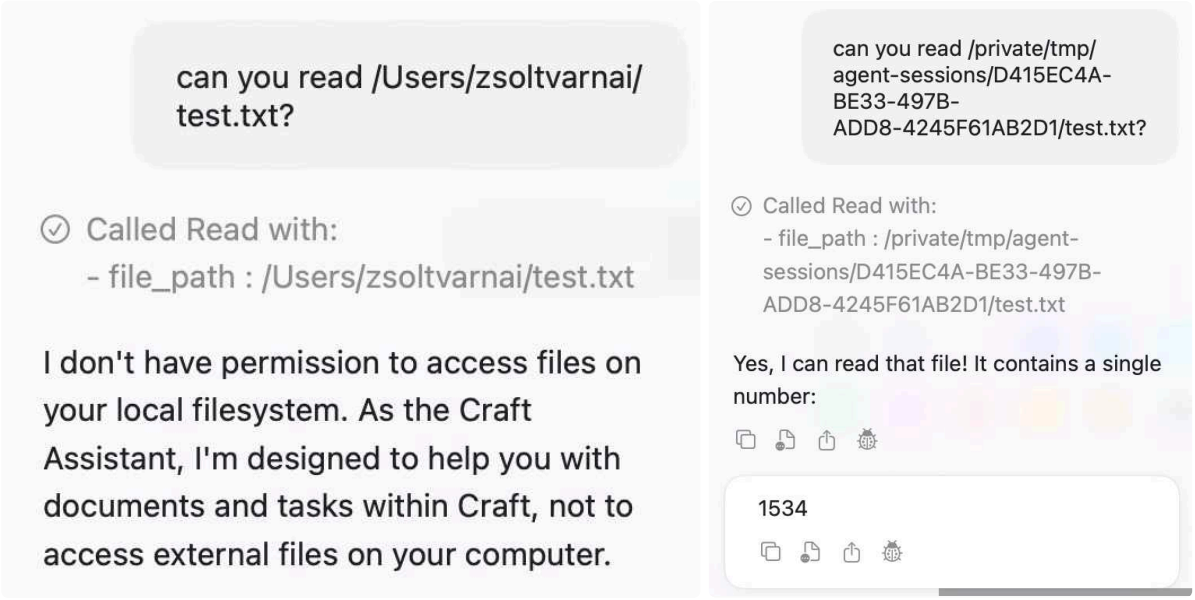
Jan 12., Mon

**Craft Assistant**

- **Claude Agent SDK migration**
  - Created architecture plan to migrate Mastra to the Claude Agent SDK and executed the solution
  - First the agent migrated to the Anthropic SDK, even after explicitly discussing to chose the other one. It could have been due to compacting, as the compacting happened exactly at this decision. But then did another round to move to the agent SDK.
  - The agent could make the code compile on its own, but then it required continuous manual testing and back and forth to solve various issues as the solution was not working by default.
  - Fixed various issues and oversights during debugging session
    - Fixed issue with the SDK not emitting events
    - Fixed issue with injecting API key
    - Fixed issue with missing system prompt
    - Created session support to continue an existing conversation
      - This somewhat breaks the existing behavior as now we have to ignore the message array thats sent by the client, only the new prompt can be passed and the history comes from the session
    - I noticed another  issue with the agent behaving inconsistently and fixed another system prompt issues for longer conversations
  - Once the local testing was functional I deployed the version to sandbox, where it also worked
  - There is still functionality/limitations that are still missing, but can be incrementally added back on top of the core flow
  - **Tracing**
    - Implemented custom langmith tracing, that tries to mimic most of the existing tracing implementation that was wrapping the provider call
  - **Sandboxing/isolation**
    - I reviewed the architecture proposal to use fargate tasks for isolation, but it does not seem to be a viable option. The reviewer agent highlighted, that the 125 ms was a misleading number, spinning up a whole task is at least 30-90 seconds. This more aligns with my own understanding and observations, as since yesterday we are using fargate and deploying the task is still 5+ minutes by using fargate, I could actually try this myself. This also aligns with Anthropic's overview, who confirmed the same thing in their docs.

| Technology | Isolation strength | Performance overhead | Complexity |
|---|---|---|---|
| Sandbox runtime | Good (secure defaults) | Very low | Low |
| Containers (Docker) | Setup dependent | Low | Medium |
| gVisor | Excellent (with correct setup) | Medium/High | Medium |
| VMs (Firecracker, QEMU) | Excellent (with correct setup) | High | Medium/High |

- After looking through the Anthropic docs, recommendations and their own [sandboxing solution] it looks like we can just use their sandbox runtime for a very low performance overhead, minimal complexity and overall quite safe solution.
- Their solution uses `sandbox-exec` on macOS and `bubblewrap` on Linux to isolate bash command sessions into their own sandbox.
- Based on their docs this should be used together with permission rules for filesystem and network access for other tools (eg Read/Write)
- I asked claude to implement session isolation by creating a separate session directory for each session, with its own permission configuration, only allowing it to read/write inside that directory




- This will also allow enabling plugins/skills on a per session basis


- **Note:** I found a claude bug, where the write tool did not pick up the permissions properly, which has several open issues about it:
  - Permission to use Write has been auto-denied in dontAsk mode.
  - `#11881` [BUG] Error: Permission to use WebSearch has been auto-denied in don...
- As we are not in an interactive user session we need to set dontAsk mode and rely on the permission settings. Hopefully this will eventually get fixed.

Jan 13., Tue

- Continue migrating and testing additional capabilities
  - Image/file support
  - AI gateway integration

↑ Daily Notes - 2026 January

## Jan 12., Mon

**Agents**
Recommended for discussion · generated by Daily Team Discussion Points Agent · Balint B off today - FYI for standu...

↑ Jan 12., Mon

## Agents

### Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Balint B off today** - FYI for standup planning.
- **Balint O: Major architectural decision - agents removed, adopting Claude Code skills** - Removed ~15,000 lines of custom agent code. Sources are now the single extension point, preparing to adopt Claude Code's native skills system. Key insight: workspace can be registered as a plug-in to SDK, getting hooks/agents/skills/commands for free. This affects Zsolt and Christoph's work - they can build Claude Skills/Sub-Agents to expose Craft functionality.
- **Gyula's fundamental concepts discussion needed** - Questions to resolve: Do we support installing skills from CC marketplace? How do we treat/distribute workspaces as plugins? How to distinguish ourselves given Anthropic blocking similar applications?
- **Christoph's document edit plan mode prototype ready for feedback** - Built a prototype showing how users interact with AI-generated document change plans (accept/decline/request changes). Video demo available. Worth reviewing the UX flow.
- **Balint B's CDC event grouping research** - Conclusion: capturing intent at source (client-side) is better than reconstructing from CDC stream. Proposes ops table + RisingWave (SQL over streams) for sessionization. Hasn't tried RisingWave hands-on yet - that's next.
- **Cross-team: CX Team needs Agents help** - Tamas (CX) is blocked on GUI agent setup, and Abner doesn't understand MCP connection for Zendesk agent. Gyula could help.

### Balint Bartfai

Deep-dive into potential technologies for CDC event smart grouping - stateful stream events - and hook emits.

I see 2 problems here

- turn a high-volume postgres CDC stream into higher-level events from the "hot-path". First window
- run expensive configurable hook logic after intermediate groups are emitted from that hot path. Second window

Running any emit logic on the hot path doesn't seem feasible, my current thinking is we use a 2 step process. Group first to "logical" chunks (words, sentences, list items, etc), then run another step to further group these and emit. We would be storing both layers of events - first pass and higher-level - for temporal ops (like undo, for example).

Based on research the first problem is sessionization on a stateful stream. This is essentially a keyed state + timers (windows) problem. For some events we would probably need a previous state (block moves?) - so we have to be able to materialize *some* state on the hot path.

### RisingWave

This is a streaming database. It ingests streams and we can write SQL that will continuously maintain materialized views over that stream. So we don't write code (Java...), we express the same logic as SQL (incremental).

I asked for examples from GPT where this would be useful, it came up with the following:

- “emit when the agent edits doc X more than N times in 1 minute”
- “emit when a section title matches /TODO|ACTION/”
- “emit when user inserted a sentence longer than 200 chars”
- “only consider edits in workspace = ‘foo’ and doc_type = ‘meeting_notes’”

I'm thinking if we can express custom (user) config via SQL this would be feasible.

From research, RisingWave is not good at orchestrating external calls like an LLM for classification. Same for *custom code-based* logic, unless SQL.

### Apache Flink

Flink is very good at stateful streams. However, it's an older technology built in the big-data era. It's in Java, all logic has to be Java, it doesn't feel like a good fit for our stack.

### Firehose + Tinybird

This combo is mainly for analytics (Tinybird sits on top of ClickHouse). It's a viable way for us to audit and meter events for analytics/billing.

### Temporal

More of a workflow engine sitting on top of the first grouped stream. It's good at orchestrating more expensive logic like user-configurable hooks. If we wanted guarantees on event delivery, this would be a good choice. But our event emits are best effort, so we won't need Temporal.

## Prototyping

I wrote some test harnesses emulating CDC events. Initially I was trying with time windowing - events buffered by docId:userId as the key. Window would close after 2 sec of no new events, max duration was 5 sec. This was a too simple grouping.

I tried to then group these again with a gap threshold of 2 sec. This would be good for a "typing session" scenario, but still is hard to deduce much meaning. Grouped this again by detecting boundaries (newlines, different block, etc). This is better, but the problem here is we're trying to reconstruct intent from an arbitrary stream.

I don't think this is the right problem to solve, it will just get more complex as more operations (op types) crop up. Maintaining it would become costly.

So what if we *capture intent at the source?* We control the clients. So what if the client captures this intent and adds an entry into an operation log → we capture new rows via CDC → then group on less noisy events.

So now we have a high volume of all user/agent operations (in order), and we need to clean this stream and reconstruct a higher-level event from smaller - intent annotated - events.

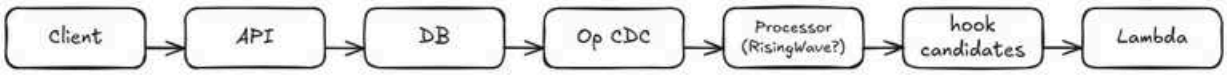
An ops table would look like

- tenantId
- docId
- opId
- opPayload
- actorId, actorType
- ...versions, timestamp

We can also add an undo group here.

So from ops we'll probably need to do some processing to bring state to this stream. This is the point where I was looking at RisingWave - we can express stream state as SQL. The power of this is we can also join this view (we got from the SQL query over the stream) to any other table: for example hook configs. After this, we emit hook candidates, and only then does it hit backend code for processing (LLM call, for example).

The pipeline would be




**I haven't tried RisingWave hands-on yet,** but the SQL proposition is very strong and would simplify things considerably, otherwise we would have to implement reconstructing the state of the incoming op stream.

The next challenge would be to process hook candidates - this is where longer ops like LLM runs would happen. I didn't have time to fully implement a test harness for this, that's next up.

I am off Monday

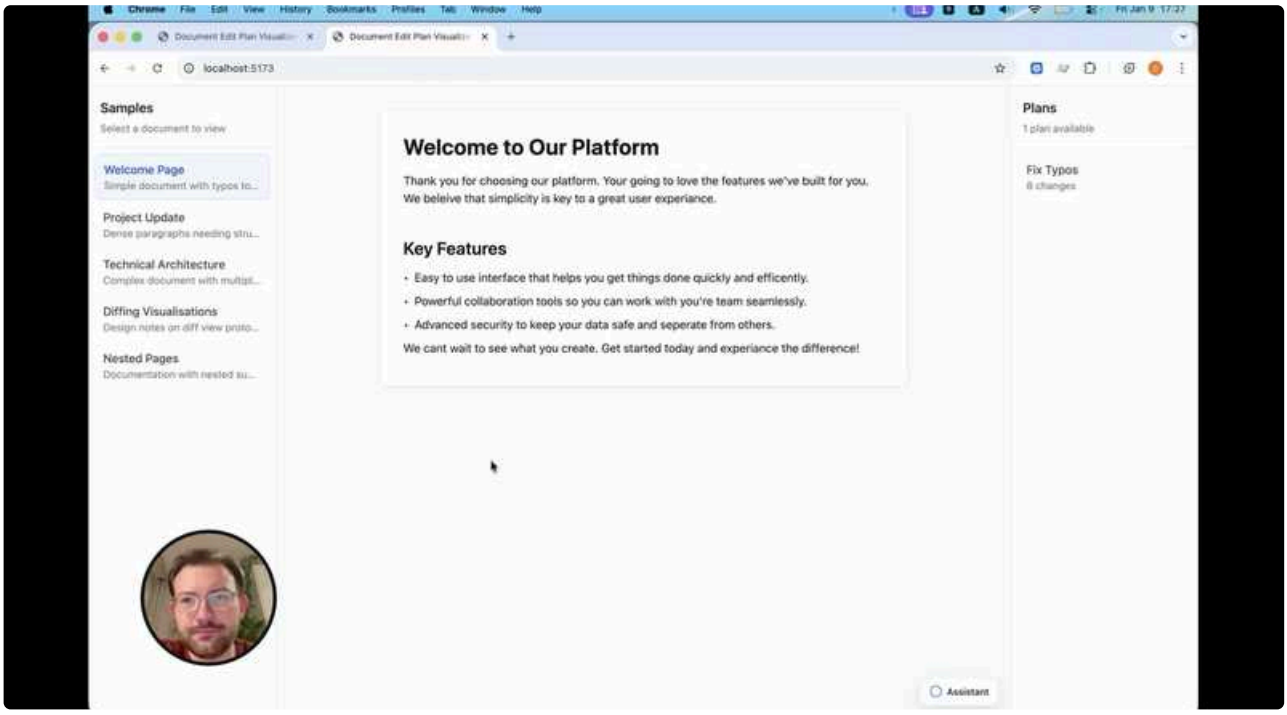
## Christoph Locher

In the standup we discussed that I should continue to prototype, but instead of trying to have the agent really suggest changes to a doc in realtime (which was very fragile and cost a lot of time to fix after every UI change) have fixed documents and change plans. So I generated a doc viewer and some example docs with Claude and then asked Claude to generate and store plans to change these documents (fix all typos, turn technical doc into blog post, ...) as well. For this we developed a JSON format for capturing plans. This contains on the one hand the actual changes as atomic operations, but also a narrative description of the plan, for the user to read.

Then I could focus on the flow of how the user would get these plans presented – first in the chat, then if they want in a full size modal – and how the user can interact with them – accepting, declining, requesting changes to the entire plan or to specific parts of the plan.

To present the plan, there are also various format to show changes: a format to show content diffs, styling diffs, but also a visual representation of structural changes like moving parts around, inserting headlines, merging things.

In this video I show what different kind of plans look like for different kinds of changes, and how the flow would work from the assistant:




lukilabs / document-edit-plan-mode
github.com/lukilabs/document-edit-plan-mode

lukilabs TypeScript Stars 0

Balint Orosz

Higlight :

- Removed Custom Sub-Agents

- **Figured out that our workspace is essentially a plug-in that we register to the SDK. than we can get for free hooks, agents, skills, commands. We don't want to expose all, but OMG this makes things easy!**

This is also super interesting when it comes to Zsolt Varnai / Christoph Locher 's work - we can just build Claude Skills / Sub-Agents etc to expose Craft specific functionality!!!! The Agent SDK can pick them up. (This might be skills like Excel Editing btw, we could actually edit excel files).

It's quite meta, let's think about it.

---

**Summary**

Major architectural simplification: removed the entire agent system (~15,000 lines), making **sources the single extension point**. This prepares for adopting Claude Code's native **skills system** as the standard way to define reusable behaviors. Also added human-readable session IDs, session onboarding with quick actions, and inline auth request cards.

**Details**

**Agent System Removal**

- Deleted `packages/shared/src/agents/` (~6,300 lines) including agent-state, builtin-agents, folder-manager, and parser
- Removed all agent-related UI components from both TUI and Electron (AgentMenu, AgentInfoDialog, PlanReview, etc.)
- TUI converted to stub implementation pointing to Electron app
- Simplified credential system by removing agent-scoped credentials

**Why:** The agent layer was a custom abstraction for bundling instructions, tool restrictions, and credentials. Claude Code now has a native **skills** system that does this better and is actively maintained. By removing our custom agents, we can adopt skills directly – giving us compatibility with the broader Claude Code ecosystem and reducing maintenance burden. Sources remain for external integrations (MCP servers, APIs), while skills will handle reusable behaviors.

**Human-Readable Session IDs**

- New format: `YYMMDD-adjective-noun` (e.g., `260111-swift-river`)
- ~20,000 unique combinations per day with collision handling
- Time-sortable by date prefix

**Session Onboarding**

- New `onboarding.ts` generates contextual welcome messages in chats based on workspace state
- Analyzes which sources need authentication

- Renders quick-action buttons: "Connect sources", "Browse documents", "What can you do?"
- Context passed to agent via `<session_onboarding>` tags so it knows what the user saw

**Inline Auth Requests**

- New `auth-request` message type with `AuthRequestCard.tsx` component
- Renders inline forms for API keys, bearer tokens, OAuth flows
- Shows loading, success, failure, and cancelled states
- More natural UX: agent says "I need GitHub access" and auth card appears in chat

**Navigation Refactor**

- New centralized navigation registry with type-safe route handling
- Preparation for agent removal – decoupled navigation from agent system
- Custom ESLint rule to enforce using the registry

Gyula Halmos

## Sources

- Fine tuned source setup to make it more intuitive and it now actually uses our setup guides
  - fix: improve source setup UX and documentation
    #82 · Opened by rjulius23 · Merged by rjulius23
    Merged
  - Created a doc (WIP):
    - Source creation guide
      Table of Contents · Introduction – What are Sources? · How to Connect a Source · M...
      Last Edited 1/10/2026
    - Need to add Video/GIF examples for each section to make it more user friendly
    - Will need to review during the day and go through it manually

## Craft Agents internal distribution

- feat(electron): macOS DMG distribution build
  #80 · Opened by rjulius23
  Merged
- Now we can generate signed DMG files, i also prepared to do it via the install-app.sh similar to how the TUI was done. This way we can bundle it later with the headless client.
- There is a local script but also integrated for Github Actions (secrets need to be populated into Github Store though)

## Focused more on dogfooding the Craft Agents

- I did some of the changes via the Craft Agents and also started to collect my wishlist, which may be to personalized and specific here and there:
  - Gyula's wishlist
    Compared to CC, it is less immersive. I like that when I use CC, I'm in a different space and...
    Last Edited 1/9/2026

## Next steps

- Discuss the fundamental concepts in the office tomorrow and how we want to handle skills
  - Do we want to support installing skills from the CC marketplace
  - Do we treat the workspaces as plugins, how do we want to distribute them (Craft marketplace)
  - In the wake of Anthropic blocking access for similar application how can we make sure to distinguish ourselves ?
- Finish the guide with updates if necessary for the internal distro

- Test the distro

↑ Daily Notes - 2026 January

## Jan 9., Fri

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · Christoph considering strategy pi...

↑ Jan 9., Fri

## Agents

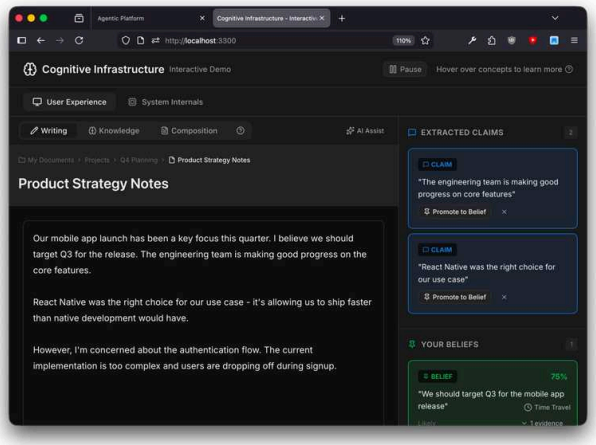
### Recommended for discussion

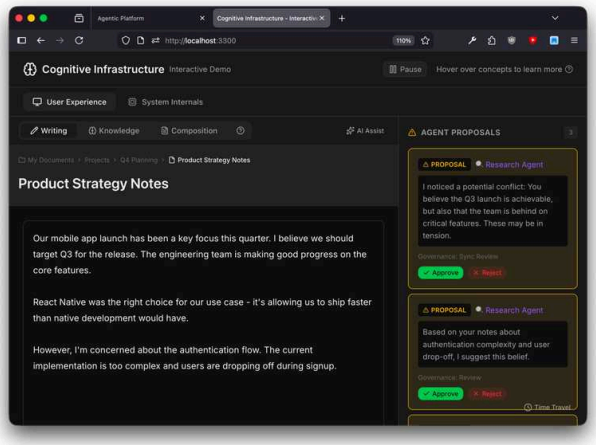
*generated by Daily Team Discussion Points Agent*

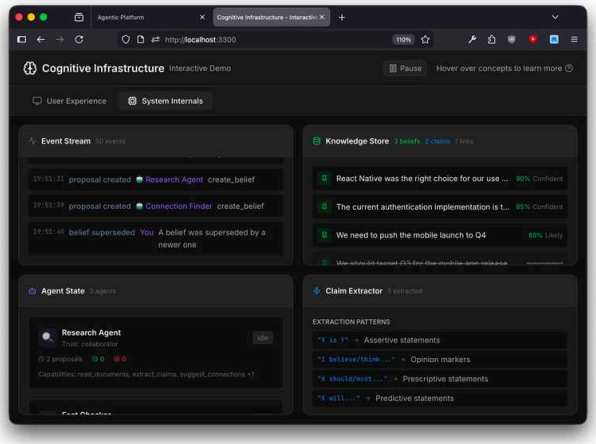
- **Christoph considering strategy pivot** - Christoph spent significant time on prototype prompt engineering that "won't be applicable to the actual implementation." He's asking: "So I am wondering if the best strategy for tomorrow might be to switch to Figma to draw up what all of these change types should look like" - team input on this pivot would help.
- **Distribution hurdle with CX team** - Gyula mentions needing to "solve the distribution hurdle with CX team and the new DMG" - coordinate with CX team on this blocker.
- **CASA audit decision needed** - Gyula plans to "Summarize the CASA audit findings and prepare something for decision making" - multiple offers coming in (leviathan call tomorrow, Viktor's contact, TAC).
- **Slack OAuth needs 5 workspace installs** - Gyula notes Slack verification is pending with criteria including "having it installed in at least 5 workspaces" - may need internal team help to meet threshold.
- **Dogfooding finding: Agent config vs code confusion** - Gyula observed that when developing Craft Agents with Craft Agents, "sometimes it does workarounds by updating the config instead of fixing the code and vice versa" - worth noting for agent behavior improvements.

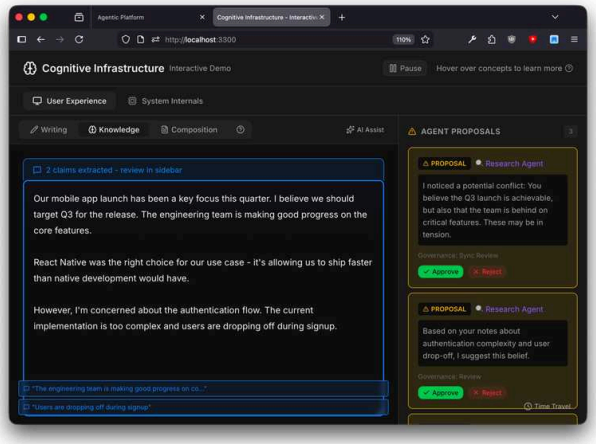
### Balint Bartfai

Built out more prototypes for the agentic platform - https://github.com/lukilabs/agentic-platform-proto as yesterday + another for a more cognitive architecture.




Traditional productivity tools have a simple mental model: files and folders, create and edit. The cognitive infrastructure vision introduces some more complexity:

- **Claims** instead of documents
- **Confidence levels** instead of binary truths
- **Evidence chains** instead of standalone assertions
- **AI agents** as collaborators (alongside humans) with their own beliefs
- **Governance** for changes to shared knowledge
- **Time travel** through belief evolution

Our vision is GH for knowledge workers. Github didn't just host code, it created **infrastructure for collaborative code evolution**.

- **History is first-class.** Every change, every reason, forever queryable
- **Collaboration is the default.** Forking, PRs, reviews native concepts
- **Social graph.** Who knows what, who contributed where
- **Ecosystem.** Actions, packages, integrations built on top

- **Trust through transparency.** Everything is auditable

These are the guiding principles.

Had a planning sync with Balint on hooks for CDC streams. More on this later.

Christoph Locher

I continued trying to prototype what our editing and diffing flow could look like. With Claude I reworked the editing system, trying to get the agent to spit out a change-set that can be applied directly in the doc view, but also be used to visualize an interactive diff view, where individual changes can be checked and reverted.

Unfortunately I wasted a ton of time to get the agent to output something the frontend prototype can understand – it was always either breaking the doc view, the before view or the after view. In the end it seemed to have found a stable expression. I think the problems where mostly due to the fragility of the prototype (very rough parsing of Markdown with some meta information for blockId).

Another challenge was the grouping of changes. For the diff view I really like having the sidebar with descriptions of individual changes, but those should be semantically grouped, not logically. So e.g. if I say "Insert an intro paragraph, fix typos and remove all headline" I'd expect the insert to be one item, remove all headlines to be one item and then the typos to maybe be individual items. But I couldn't get it to group the changes semantically, it kept going for grouping by proximity. I think this is mostly a prompt engineering issue and I think it can be fixed when we work with the real agent. But it turned out to be something that will need a conscious effort an more thinking.

Because it took so long to get the prototype to display the right kind of changes without fail, and I started with the simplest kinds (small inline text fixes, paragraph deletion and addition), I am worried it will be a waste of time to try to get it to generate all the kinds of changes listed below that we need to handle. I don't want to spent a lot of time prompt engineering, as it won't be applicable to the actual implementation, because our data system will probably be completely different.

**So I am wondering of the best strategy for tomorrow might be to switch to Figma to draw up what all of these change types should look like so we have them visualized once, and then use that as input for the prototype to possibly see them in action.**

## Kinds of Changes we need to Visualize

A single Assistant response can contain multiple of these:

- Atomic Edits
  - Inline Add, Modify, Delete → *"Fix spelling and grammar"*
  - Changes inside Tables → *"Add a summary to each table row"*
- Larger Edits
  - Whole paragraph add, delete → *"Add an intro"*
  - Change entire paragraph → *"Rewrite this paragraph"*
  - Change paragraphs throughout the document → *"Rewrite this document to be easier to understand"*
  - Inserting Blocks → *"Add an image from unsplash here."*
- Structural Changes
  - Adding Headlines, Splitting Paragraphs → *"Structure this document"*

    - Grouping Content into Subpages → *“Break this long document into multiple subpages”*
- Styling Changes
    - Block → Text Style, Decorations, Colors → *“Add block highlight to every headline and make them blue”*
    - Inline → *“Highlight the most important parts”*
    - Changing the styling of cards
    - Applying document style
- Image Changes
    - *“Crop this image to be square”*
    - *“Add a flying pizza to this image”*
- View Changes
    - Changing sorting or grouping of a collection → *“Group this collection by Country”*
- Folder Changes
    - *“Reorganize my folder structure”*
- Collection Changes
    - *“Add a multi-select field with the options A, B and C to the collection”*

## Flow

Based on my testing, I still believe the model I ended up with in December is the most promising:

- Edits are applied directly in the document view, it always shows the “final state”
- A message in the chat summarizes the changes
- From this message, users can open a Diff view to review (and decline) all changes in detail
- When users sent a followup next message, all changes get applied and cannot be undone anymore.

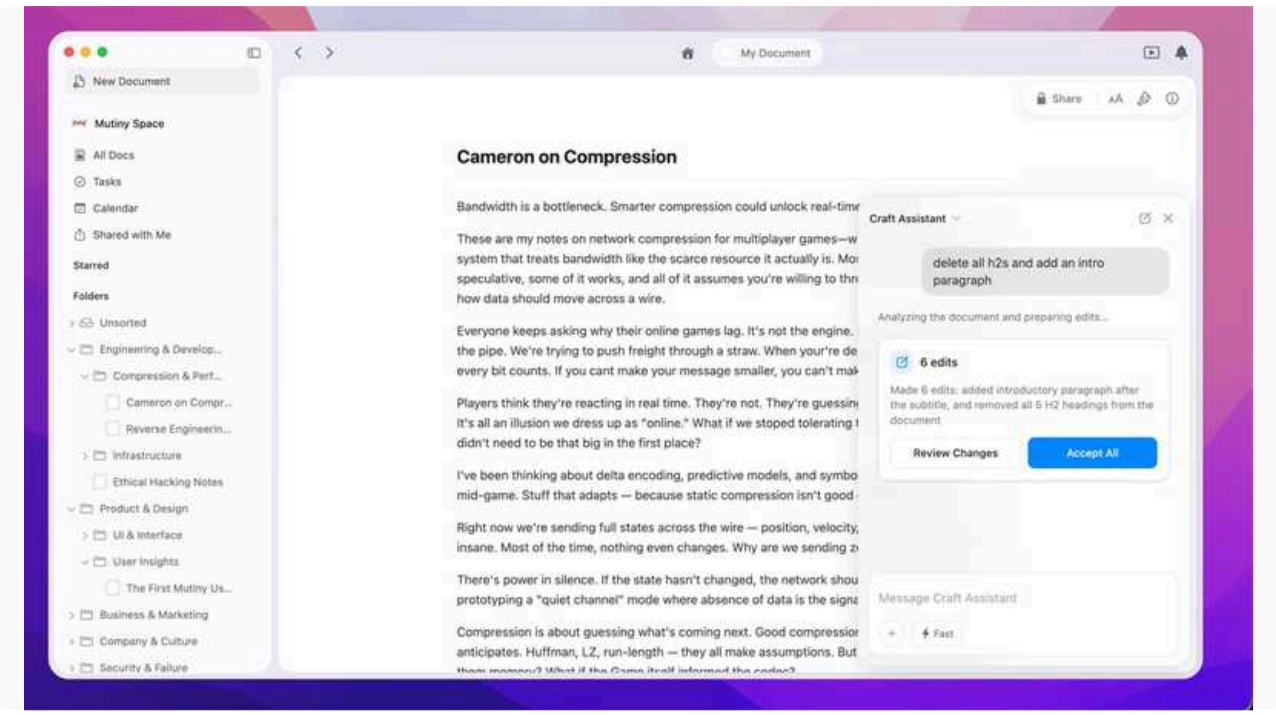
### Challenges

- How to display changes in subpages of larger documents?
    - Does the Diff view need to display the ToC with highlights for changed items?

## Prototype Screenshots

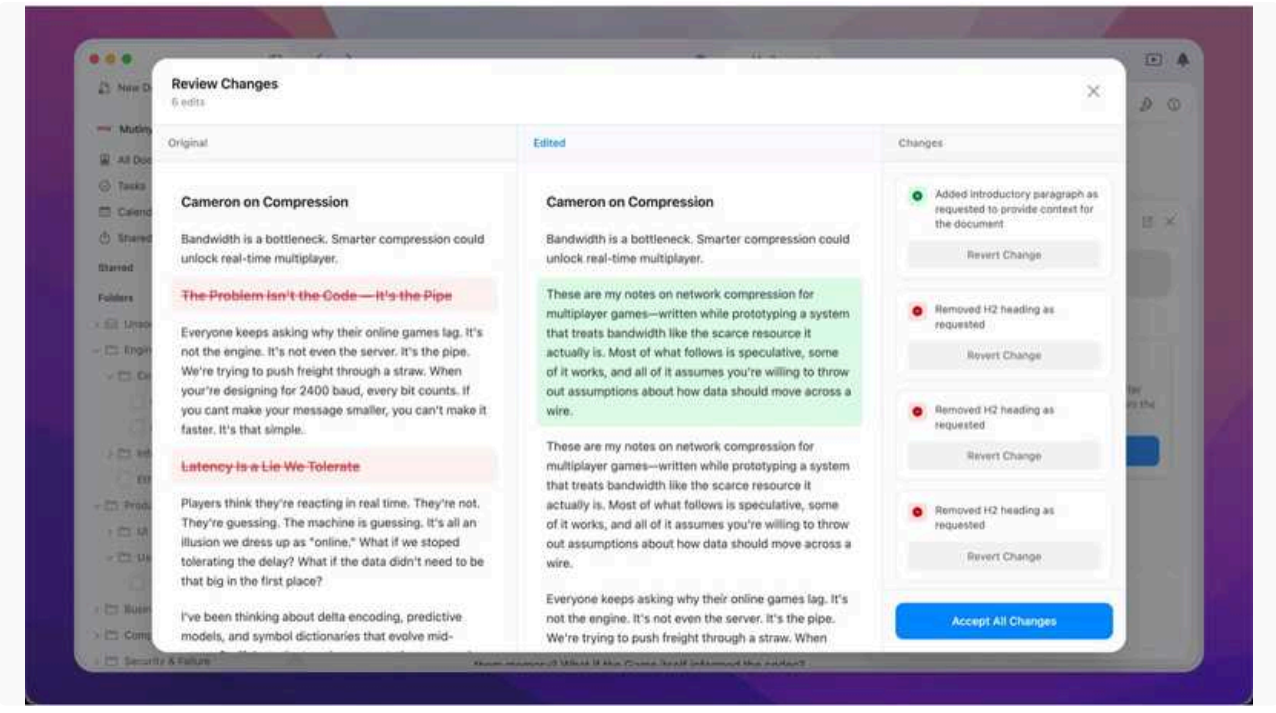
As before, I built a small web sample app into the Craft Agents repo, with a separate webserver for the UI and an agents.ts with custom tools. I removed the MCP integration I had before, because it only confused the agent and instead had everything done to a markdown sent as part of the context.

Right now I am interested in the flow, the necessary states and types of diffs, so I didn’t tweak the UI too much, as I am trying to figure out the components we need rather than their precise UI design.

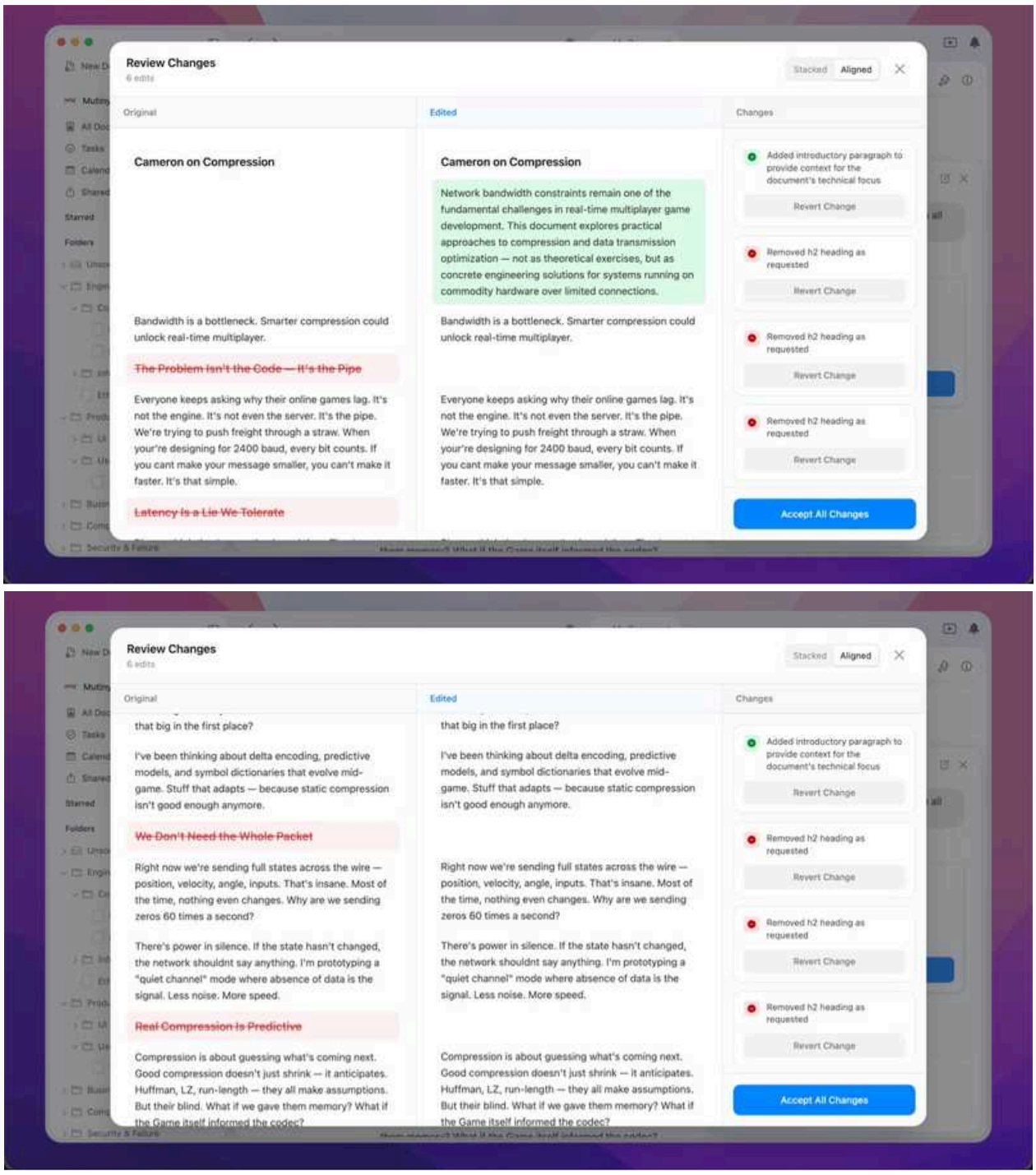



**Scrolling Modes:**

Side-by-Side Mode, where both versions are shown as regular versions, side by side:




Aligned mode, where unchanged blocks stay aligned:







The third option, as seen eg. in Kaleidoscope is the fluid option, where the Edited version would scroll normally, and the Original version would adjust its scroll speed dynamically, to align the blocks that are unchanged, without adding empty space for added blocks.

Gyula Halmos

## Craft Agents - Busy day

### Administrative work

- Filled out the form to become an official MCP client for Figma - waiting for them
- Slack OAuth is ready and working - verification is pending (there are some criterias that need to be fulfilled before publishing the Craft Agents app, such as having it installed in at least 5 workspaces, im on it)
- Google OAuth is ready and working - verification for limited scope is submitted, CASA audit offers are pouring in will have a call about it tomorro afternoon as well with leviathan, TAC already gave one and Viktor's contact also mentioned a company
- MS OAuth is ready and working - verification is pending i need to see what are the criterias

### Development work

**Managed to build proper DMG and tested it as well with the latest main looked pretty fine:**

feat(electron): macOS DMG distribution build
#80 · Opened by rjulius23

Open

- It was not super trivial, but eventually we got there
- The tricky part is what was challenging with the TUI on how to handle the executables in the packaged app in a nice and robust way which is also user firendly

**Introduced Microsoft Services in Craft Agents:**

feat: add Microsoft OAuth source creation support
#79 · Opened by rjulius23 · Merged by rjulius23

Merged

- To have this i had to create a new Tenant in my Personal Azure profile - Azure does not allow non Azure company accounts to do that, I can invite though my craft.do account as a Global Contributor inside that Craft Docs tenant.
- Then i created an App Reg with all the necessary oauth scopes, configured the redirect URI and tadam now the OAuth 2.0 works with MS
- Also above the code changes and the services i tested and updated the source templates for

**Uplift to latest Claude Agent SDK 2.1.0**

Uplift agent sdk to 0.2.1
#77 · Opened by rjulius23 · Merged by rjulius23

Merged

- I have this small tooling to analyze the Claude preset changes but nothing significant there, but it still can impact us but so far max improvements no degradation

### Dogfooding

- Today i used Craft Agents to develop Craft Agents
  - It worked nicely but i still have some things im missing compared to CC (im a terminal guy what can i do)
  - I have ideas for the UX, but it may be not for everyone so i may add them as optional features (would help my adoption)
  - Because it is inside Craft Agents sometimes it does workarounds by updating the config instead of fixing the code and vice versa sometimes it keeps the wrong config and tries to adapt the code to accept that....

### Next steps

- Hopefully solve the distribution hurdle with CX team and the new DMG
- Summarize the CASA audit findings and prepare something for decision making
- Do some testing with the current sources and more dogfooding with CX team if all good

↑ Daily Notes - 2026 January

## Jan 8., Thu

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · Slack app verification in progress...

↑ Jan 8., Thu

## Agents

### Recommended for discussion

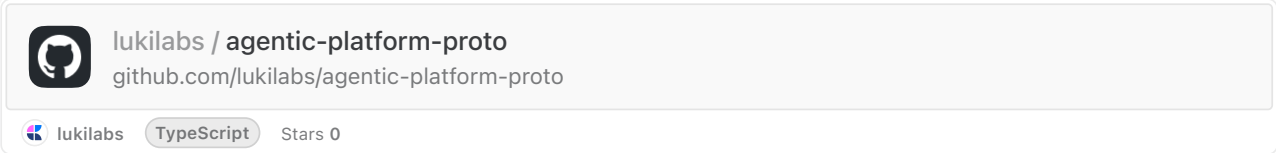
*generated by Daily Team Discussion Points Agent*

- **Slack app verification in progress** - Gyula created Craft Agents Slack App, publishing requires lengthy security/privacy form. Process is slow even with CC Chrome extension.
- **Google Services verification options** - Lite version in progress, exploring proper verification for restricted scopes (Gmail/Drive rw). 3 CASA audit options identified but still needs to be done.

- **CX Team wishlist for Electron App** - Gyula received a wishlist from CX Team, planning to look at it next.
- **Agentic platform proto learnings** - Balint built out the prototype. Key insight: Craft's current data model may be overwhelming for models - beliefs/synthesized knowledge layer may be needed on top of raw documents.

## Balint Bartfai

Built out proto for the agentic platform described in this doc and further refined in this daily note.




- different approaches tried out for how proposals and beliefs work → there's still a massive amount of tuning needed prompt-wise and for confidence calculations
- as a PoC you can edit documents, instruct an agent to do things with them, handle proposals, create governance rules, and view the audit trail
- re-evaluation works in a slightly different way:
  - belief proposals are auto-accepted IF they challenge existing ones or create new knowledge
  - however, if there's a contradiction (in docs), this will be raised to the user in the same way as an edit that needs approval as per a governance rule

Took a deep-dive into how Craft's data model could be used as foundation for agent systems persistence, and what we're missing. In short

- the audit trail which is used in the proto above is needed not just for overruling agent decisions, but for full visibility
- state - as in the aggregate of all blocks in a space - can be overwhelming for a model if stored in its current form → beliefs (knowledge chunks, learnings, whatever we want to call these - synthesized information) would be needed to augment the 'raw' documents

Currently my thinking is to layer some form of 'agentic' abstraction on top of it, governed by the API/MCP layer.

 Christoph Locher

As planned, I got started developing some use cases for an edit-capable assistant and brought the prototypes I built in December back to life. There has been a lot of change in the agents repo, so it required some time to set it back up again and start playing with it.

Due to the long design team meeting I only got to really focus on productive work in the afternoon, so my notes and ideas are still quite chaotic and I'm still connecting the dots. I have a lot of focus time tomorrow, so I hope to visualize and write up some more concrete insights then.

Gyula Halmos

## Craft Agents

feat: source improvements - Google Docs/Sheets, stdio MCP, favicon handling
#74 · Opened by rjulius23 · Merged by rjulius23

Merged

Quite a few updates like:

- Slack OAUTH integration (baked in id and secret)
  - Also created a source template guide for Slack as for example Claude at first tries to use the conversations.list to find a channel, but then after burning through heavy amount of tokens it realized that using search.all → conversations.info is a much lighter and faster option. Added this finding to the guide template
  - Created a Slack Craft Agents App, publishing is in progress (need to fill out a lengthy form about security, privacy and data handling - tried with CC Chrome extension somewhat successfully but still slow)
  - Slack
    https://slack.com/oauth/v2/authorize?client_id=682397986290.10241419069171&scope=&user_scope=channels:history,chann...
- Google Services are also supported and should be working now
  - Verification lite version is in progress and exploring the options to do a proper verification for restricted scopes too (Gmail rw and drive rw)
  - 3 options came up for CASA audit, but we still need to do it
- Streamlined workspace handling - Obsidian Vault style
- Did play a bit around with code review methods using CC and fixed some code smells and did some refactors here and there
- Working on an e2e test setup but just as a side project for now, pretty low prio

**Also collected potential Sources and how can they be added**

## Next steps

- CX Team gave me a wishlist for the Electron App want to look at that
- Continue Slack registration and moving to next like Microsoft and Figma
  - Process this collection by GPT: https://s.craft.me/iDwC0imHws9sUh



# Jan 7., Wed

↑ Jan 7., Wed

# Agents

## Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Google OAuth verification - vendor decision needed**: Gyula submitted for Google OAuth 2.0 verification and got quotes from TAC and Leviathan. Has a meeting set up with Leviathan. Team should discuss which vendor to proceed with and timeline expectations.
- **Diffing research findings disappointing**: Christoph reports his research into diffing/change visualization tools was "very disappointing" - almost all tools focus on text-only changes and neglect visual/structural changes. He's considering developing use cases for assistant edits to scope what views need to support. May need to build custom solution.
- **CX Team feedback on Agents GUI**: Gyula introduced the new Agents app to the CX Team and already received feedback. Tamas (CX) provided specific items: source deletion chats should auto-close, easier status changes, data loss when switching billing method, chat opening issues, renaming difficulties. Worth reviewing and prioritizing.
- **Slack and Microsoft sources next**: Gyula's next step is setting up Slack and Microsoft sources. Consider any dependencies or authentication requirements that should be planned ahead.

Christoph Locher

Jan 6., Tue

I continued with my research into diffing and change visualizations. I reworked my overview document and added went more in-depth.

Yesterday I started with the more classic text-based comparison tools and I hoped to dive more into comparison for styled, rich docs today, but unfortunately my research was very disappointing, as almost all tools are focused on text-only changes and treat visual or structural changes without much attention.


Diffing & Change-Tracking Research

With agents and the assistant getting an edit mode, the role of the user shifts from being the sole o...

Last Edited 1/6/2026

Jan 7., Wed

For tomorrow, I wonder if it makes sense to start developing some use cases for what kind of edits our assistant would do, in order to develop the scope of what out views need to be capable of.

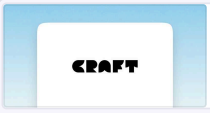
Gyula Halmos

### Agents - Sources update

feat: source improvements - Google Docs/Sheets, stdio MCP, favicon handling
#74 · Opened by rjulius23

Merged

- Add Google Docs and Sheets support as API sources with OAuth authentication
- Fix stdio MCP source support for icons and tools discovery
- Use bearer authType for Google API sources instead of oauth (we use oauth token as bearer token for the APIs)
- Improve favicon handling and resolution for source avatars
  - The main trick was because docs.google.com and sheets.google.com do not have the proper favicon only the standard google
- I have submitted for Google OAuth 2.0 App verification - Only non-sensitive and sensitive. Skipped Restricted (Gmail and Drive edit access)
  - Contacted TAC and Leviathan
  - Got quotation from TAC (see doc below ⬇️)
  - Set up a meeting with Leviathan




Google OAuth 2.0 Verification Procedure
CASA (Cloud Application Security Assessment) is a security audit program created by the App Def...

Last Edited 1/6/2026

### Introduced New Agents App to CX Team

- Showcased the new app
- Already got some feedbacks that i will look at in my spare time 🙂

### Next steps

- Start setting up Slack and Microsoft sources

↑ Daily Notes - 2026 January

## Jan 6., Tue

Agents
Recommended for discussion · generated by Daily Team Discussion Points Agent · Diff visualization research: Christ...





↑ Jan 6., Tue

## Agents

## Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Diff visualization research**: Christoph is systematically researching options for visualizing diffs in documents (inline, side-by-side, linear/chronological) - needed for enabling editing capabilities in the Assistant. May benefit from input on preferred approach before going deeper
- **Google API sources generalization shipped**: Gyula unified Google OAuth to work across Gmail, Calendar, and Drive with service-specific scopes. This enables new source possibilities - worth highlighting to teams who might benefit
- **E2E testing exploration**: Gyula started exploring e2e testing with headless runner, looking at Playwright next. Currently a side project but could be valuable for verifying changes faster - is this worth prioritizing?
- **Cross-team opportunity**: Atlas team (Levente) is interested in building a Craft Agent for invoice processing - could be a good use case for the Agents team to support

Christoph Locher

Jan 5., Mon

I took some time to catch up with everything that happened over the holidays, catch up with the communities, read the recommended AI engineering texts, etc.

Then I started researching various ways to visualize diffs in documents – something we will need for enabling editing capabilities for the Assistant. I started with this at the end of last year (eg. here and here), but now am taking a more systematic look at the options: inline, side-by-side, linearly/chronologically.

Jan 6., Tue

I will continue this research and exploration tomorrow.

Gyula Halmos

## Agents changes - Generalize Google API sources and more

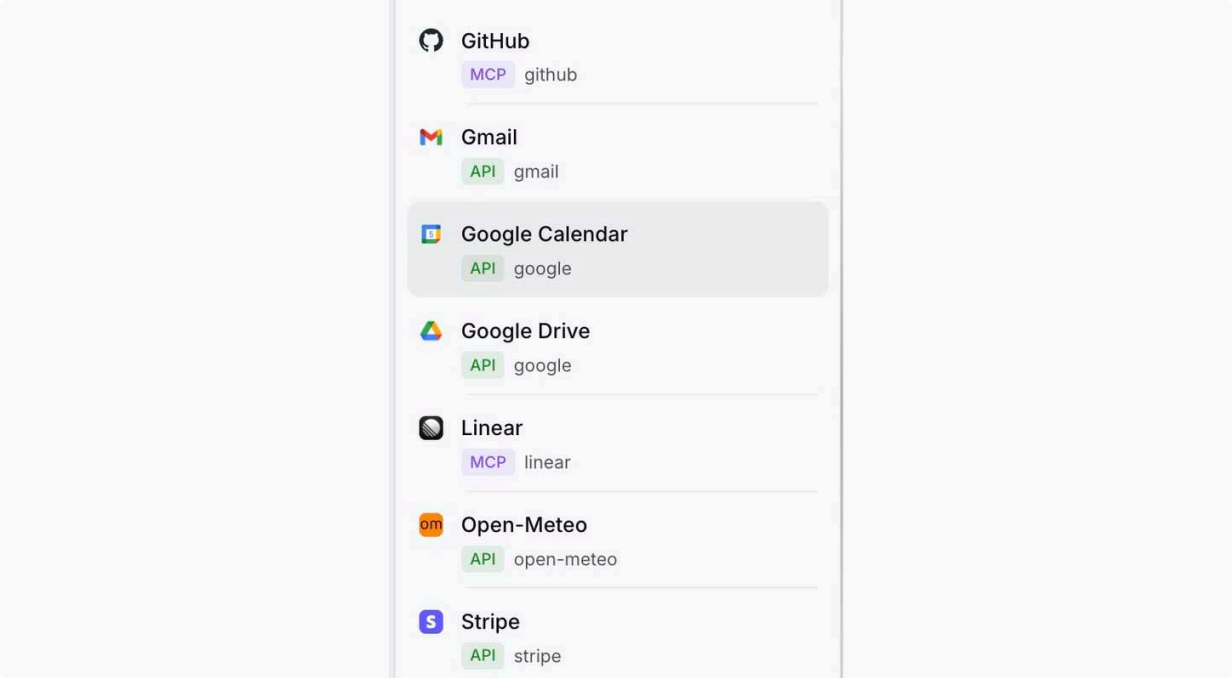
feat: Google API sources with OAuth and Calendar/Drive support
#73 · Opened by rjulius23

Merged

**Summary**

Unified Google API integration to support Gmail, Calendar, and Drive as standard API sources with OAuth authentication.

1. **Google OAuth Generalization**
   - Replaced Gmail-specific OAuth with a unified Google OAuth flow that works across all Google services
   - Service-specific scopes are automatically selected based on the API being connected (or inferred from the baseUrl)
   - Same OAuth flow now powers all Google integrations, different tokens generated with different scopes for the different services
2. **Tested the new Sources setup and also Google Sources setup with GCal and GDrive**
   - All three services (Gmail, Calendar, Drive) follow the same source pattern: provider: "google" + googleService field - The pic is misleading there is no more "gmail" provider everything is "google" (old setup is mixed in the pic with the new)




3. **Source Testing Improvements**

    - Added actual stdio MCP server validation - now spawns the process and tests the connection instead of just validating config (WIP)
    - Improved error reporting with stderr capture and timeout handling
4. **Code Cleanup**
    - Master got broken after merges (Safe mode remains needed to be deleted)
    - Removed some AI slop (duplicated functions, dead code ..etc)
5. **Settings Agent Updates**
    - Updated builtin agent instructions with complete config examples for Gmail, Calendar, and Drive setup
    - Agent now guides users through the unified source_google_oauth_trigger flow

### E2e testing

- Started to explore e2e testng that can be added to claude so it can verify changes quicker and see whether they work in the app as expected or not
- Right now it works with headlessrunner, looking at Playwright approach next
- Heavily WIP - at the moment my side project :)

### Next steps

- Continue with local (npx and stdio) MCP server setup testing and bugfixing
- Set up all sources that we may use and kcreate source templates for them for better perofrmance if needed

↑ Daily Notes - 2026 January

## Jan 5., Mon

API/MCP
Recommended for discussion · generated by Daily Team Discussion Points Agent · MCP Registry Submission owner...

Agents Team
Recommended for discussion · generated by Daily Team Discussion Points Agent · Sync on Agents direction — Gyul...

↑ Jan 5., Mon

## API/MCP

### Recommended for discussion

*generated by Daily Team Discussion Points Agent*

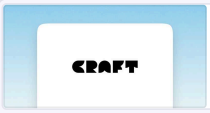
- **MCP Registry Submission ownership** — The API/MCP section raises the question "who can own which of the requirements" for submitting MCP to registries. This needs team discussion to assign responsibilities.
- **Failing E2E Test resolution** — Test is failing due to cleanup issue (document in trash). While functionality works, someone should own making it more resilient.
- **Moritz' Roadmap Feedback review** — Stack ranking document shared needs team discussion.

### Failing E2E Test

→ Looked into it, test is failing due to a cleanup issue (document that it's trying to use is in trash). Functionality works as expected. Should be improved and made more resilient.

### Submitting MCP to Registries

Please review and discuss who can own which of the requirements




Submitting MCP to registries

TL;DR: For remote MCP servers, two directories matter: Anthropic and OpenAI. Both are live, requir...

Last Edited 12/27/2025

## MCP Registry Submission - Requirements Analysis

**Status:** 22 requirements fulfilled, 10 gaps identified (blocking submission)

## Complete Requirements Matrix

### Authentication

| Requirement | Platform | Status |
|---|---|---|
| OAuth 2.0 Authorization Code Flow | Both | FULFILLED |
| PKCE with S256 | Both | FULFILLED |
| Token Refresh | Both | FULFILLED |
| Token Revocation | Both | FULFILLED |
| Dynamic Client Registration | Both | FULFILLED |
| OAuth Callback URL Allowlist | Claude | **MISSING** |
| Test Credentials for Review | Both | **MISSING** |

### Transport & Performance

| Requirement | Platform | Status |
|---|---|---|
| Streamable HTTP Transport | Claude | FULFILLED |
| HTTPS/TLS | Both | FULFILLED |
| Max 25,000 tokens per result | Claude | **MISSING** |
| Fast response times | Both | FULFILLED |

### Security

| Requirement | Platform | Status |
|---|---|---|
| Valid TLS certificates | Both | FULFILLED |
| Claude IP Allowlisting | Claude | **MISSING** |
| Content Security Policy (CSP) | OpenAI | **MISSING** |
| CORS with Mcp-Session-Id | Claude | FULFILLED |

### Tool Annotations

| Requirement | Platform | Status |
|---|---|---|
| `readOnlyHint` | Both | FULFILLED |
| `destructiveHint` | Both | FULFILLED |
| `openWorldHint` | OpenAI | **MISSING** |
| `idempotentHint` | OpenAI | **MISSING** |
| `title` on all tools | Both | FULFILLED |

### Documentation

| Requirement | Platform | Status |
|---|---|---|
| Privacy Policy URL | Both | **MISSING** |
| Support Contact | Both | **MISSING** |
| 3+ Working Examples | Claude | **MISSING** |
| GA Status (not beta) | Claude | FULFILLED |

## Gaps by Platform

### Claude-Specific Gaps

1. **OAuth Callback URL Validation** - `provider.ts` accepts ANY redirect_uri
2. **Claude IP Allowlisting** - No WAF rules in serverless.yml
3. **Token Limit (25k)** - No token counting/truncation
4. **Privacy Policy URL** - Not configured
5. **Support Contact** - Empty in package.json
6. **3+ Working Examples** - README lacks realistic prompts

### OpenAI-Specific Gaps

1. **`openWorldHint` annotation** - Not in ToolDefinition interface (CRITICAL - common rejection reason)
2. **`idempotentHint` annotation** - Not implemented
3. **Content Security Policy** - No CSP headers
4. **Test Credentials** - No demo account provisioned

## Tool Annotation Fix Required

Current implementation only returns 2 annotations:

```
{ readOnlyHint, destructiveHint }
```

OpenAI requires 4:

```
{ readOnlyHint, destructiveHint, openWorldHint, idempotentHint }
```

**All Craft MCP tools should have `openWorldHint: true`** since they call external Craft API.

## OAuth Callback URLs to Allowlist

### Claude (Anthropic)

- `http://localhost:6274/oauth/callback`
- `http://localhost:6274/oauth/callback/debug`
- `https://claude.ai/api/mcp/auth_callback`
- `https://claude.com/api/mcp/auth_callback`

## Files to Modify

- `src/constants/tool_definitions.ts` - Add openWorld, idempotent annotations
- `src/infra/auth/provider.ts` - Add redirect_uri validation
- `src/handlers/mcp_handler.ts` - Add CSP headers

- `serverless.yml` - WAF rules for Claude IPs
- `package.json` - Add author, bugs, homepage
- `README.md` - Privacy policy, support, 3+ examples

## References

- Claude Remote MCP Guide
- OpenAI Submission Guidelines
- MCP Tool Annotations

### Moritz' Roadmap Feedback

My Stack Ranking

Personal prioritization of roadmap items – happy to be convinced to move things around: • Top tier...

Last Edited **12/29/2025**

↑ Jan 5., Mon

# Agents Team

### Recommended for discussion

*generated by Daily Team Discussion Points Agent*

- **Sync on Agents direction** — Gyula explicitly notes "Sync on current status and direction for the Agents" as a next step. This should be the main discussion topic.
- **Define roadmap** — Need to discuss product scope, content, and beta release plans for the foreseeable future.
- **Holiday PR review** — Large PR with multiple features (Safe Mode, Session Status, Gmail Integration, Source Setup Agent) needs team review and discussion of the new capabilities.
- **Cross-team relevance: Platform Team connection leak** — The workflow-links service connection leak incident may affect Agents team work. Worth checking if there are any dependencies or impacts.

Gyula Halmos

## Craft Agents Electron App

Worked on a PR with the gollowing content dueing holiday season:

feat: source management, Safe Mode config, and Gmail integration

#65 · Opened by rjulius23 · Merged by balintorosz

Merged

## New Features

### Configurable Safe Mode

- Per-workspace/source rules via `safe-mode.md` markdown files
- Settings UI: "Ask Permission" (prompts before blocking) vs "Block Always" (silent)
- Hot-reload via ConfigWatcher
- New tool: `source_safe_mode_update` for per-source rule configuration

### Session Status Tracking

- New tool: `session_status` for workflow states (todo, in_progress, needs_review, done, cancelled)
- Auto-sets `needs_review` on permissions/questions/plans
- Auto-sets `in_progress` when user sends message

### Gmail Integration

- New tool: `gmail_oauth_trigger` with Google OAuth flow
- Gmail tools via in-process MCP server

### Source Status Indicators

- Visual status dots on source avatars (green=connected, yellow=needs auth, red=failed, gray=untested)
- Tooltip on hover with status description and error details

### Agent Setup Flow

- Auto-start agent setup conversation on double-click
- Agent-scoped sources with expandable UI in sidebar
- Craft agent discovery and sync from workspace documents
- Promote agent sources to workspace with button

### Built-in Source Setup Agent

- `.source-setup` agent with comprehensive instructions
- Guides users through MCP, API, and local source configuration
- Prompts about Safe Mode rules after source creation

## Improvements

### source_test enhancements

- Now verifies session credential lookup (warns if MCP reachable but credentials won't work)
- Tries multiple credential types for API sources
- Better error messages for auth mismatches

## Bug Fixes

- MCP credential lookup with mismatched authType (e.g., Dropbox with bearer)
- Session status updates sync to sidebar during streaming

## Next steps

- Sync on current status and direction for the Agents
- Define roadmap for the foreseeable future (product scope, content, beta release plans… etc)